

University of Camerino

School of Science and Technology

Master's degree in Computer Science (LM-18)

Adaptive Multi-Hazard Emergency Response Framework for Indoor Environments

Author
Nicol Buratti

Matricula 128003

Advisor
Prof. Massimo Callisto De Donato

Coadvisor
Luca Mozzoni

A.Y. 2025/2026

Abstract

Indoor emergencies such as fires, hazardous material leaks, and structural failures put people, buildings and the surrounding environment at risk. Traditional evacuation plans rely on fixed routes and static signage, which cannot adapt when a hazard changes shape, spreads, or blocks the planned path. This thesis presents a framework that reacts to hazards as they develop, instead of relying on a route decided before the emergency starts. The building is represented as a graph, where each room and corridor is a node, and Internet of Things (IoT) sensor data is used to keep the state of the building up to date. A Multi-Agent System (MAS), built with Large Language Model (LLM)-based agents, interprets the incoming sensor data, detects hazards, and enriches the graph accordingly. An environment-aware Business Process Model and Notation (BPMN) engine orchestrates the response: it computes safe evacuation routes on the graph and turns on physical actuators, such as dynamic lighting, to guide occupants and inform the security warden. A prototype of the framework was built and tested on a real building, the Department A of the Computer Science division at the University of Camerino. When hazards appear, the system consistently finds a valid route that avoids the hazardous areas, at the cost of a longer path when the shortest one is no longer safe. The results suggest that combining a graph-based representation of a building with hazard-aware routing can help people evacuate along a path that is both safe and efficient. The current prototype is a simplified version of what a deployed system would need to be, since it does not yet integrate Building Information Modeling (BIM) data, does not model occupant behavior, and leaves human oversight, security and regulatory compliance outside its scope.

Contents

Abstract iii

List of Listings vi

List of Figures ix

List of Tables xi

Acronyms xiii

1 Introduction 1
- 1.1 Problem Statement . . . 1
- 1.2 Project Description . . . 2
- 1.3 Objectives . . . 3
- 1.4 Thesis structure . . . 3

2 Background 5
- 2.1 Emergency Management . . . 5
  - 2.1.1 Occupant Behavior and Evacuation Dynamics . . . 6
  - 2.1.2 Situational Awareness . . . 8
  - 2.1.3 Active Evacuation Guidance Systems . . . 10
- 2.2 Digital Twin . . . 10
  - 2.2.1 Spatial Digital Twin . . . 11
  - 2.2.2 Interplay of Business Processes, Environment and IoT . . . 12
- 2.3 AI Agents . . . 14
  - 2.3.1 The Agentic Workflow . . . 15
  - 2.3.2 Memory Architectures . . . 15
  - 2.3.3 Tools . . . 17
  - 2.3.4 Multi-Agent Systems (MAS) . . . 17
  - 2.3.5 Knowledge Graphs . . . 19

3 Literature Review 21
- 3.1 Methodology . . . 21
  - 3.1.1 Database and Query Search . . . 21
  - 3.1.2 Inclusion and Exclusion Criteria . . . 22
- 3.2 Findings . . . 22
  - 3.2.1 Observations . . . 25

3.2.2 Results and Gap Analysis 26

4 Architecture Design **29**
4.1 Design Goals 29
4.2 Conceptual Architecture 30

5 Prototype **33**
5.1 Architecture 34
5.2 Technology Stack 35
5.2.1 Evaluation of AI Agent Frameworks 35
5.2.2 Node-RED 37
5.2.3 Evaluation of Graph Databases 39
5.2.4 Memgraph 40
5.2.5 Camunda Process Engine 41
5.3 Data Integration Middleware 42
5.3.1 Node-RED Flows 43
5.3.2 Emergency Agent 47
5.4 Environment Engine 54
5.4.1 The Physical Layer 54
5.4.2 The Logical Layer 57
5.5 Environment-aware BPMN Engine 59
5.5.1 BPMN Extension and Custom Tasks 59
5.5.2 Emergency Process Modeling & Implementation 61

6 Evaluation **69**
6.1 Evaluation Objectives 69
6.2 Evacuation Routing Evaluation 70
6.2.1 Metrics 70
6.2.2 No-Hazard Scenario (S0) 72
6.2.3 Static Route Hazard (S1) 72
6.2.4 Dynamic Hazard (S2) 73
6.3 MAS Evaluation 75
6.3.1 Experimental Setup 75
6.3.2 Metrics 76
6.3.3 Effectiveness 77
6.3.4 Efficiency 78
6.3.5 Reliability and Cost 80

7 Discussion **81**
7.1 Research Questions Revisited 81
7.2 Limitations 82
7.3 Future Work 83

8 Conclusion **87**

Bibliography **89**

List of Listings

1 The graph states 47
2 The Agent's structured output 48
3 The Triage structured output 49
4 `get_room_data` query 51
5 `get_adjacent_rooms` query 52
6 The graph compilation code 53

List of Figures

2.1 The behavioral process of occupant response [11]. 7
2.2 A bottleneck situation [12]. 7
2.3 Data flow levels between physical and digital spaces [20]. 11
2.4 Core Set of BPMN Elements [24] 13
2.5 Comparative representation of conventional Generative AI (GenAI) (left) and Agentic AI (right) [29]. 16
2.6 The architecture of LLM-based MAS [34] 18
2.7 An example of Knowledge Graph [40] 20

4.1 High level view of the proposed architecture. 30

5.1 System architecture 34
5.2 LangChain and LangGraph logos 37
5.3 Node-RED logo 38
5.4 Node-RED architecture [61] 38
5.5 The Memgraph logo 40
5.6 Camunda Logo 41
5.7 Camunda architecture [64] 42
5.8 The place graph initialization flow 43
5.9 IoT caching flow 44
5.10 Visual IoT aggregation in Node-RED. Non-technical users can easily trace data flows by hovering over the output nodes to reveal filtering parameters like `== ‘‘temperature’’`. 45
5.11 The agent output flow 46
5.12 Agent architecture 47
5.13 Floor plans for Department A of the Computer Science division at the University of Camerino 56
5.14 Visualization of the populated physical layer graph in Memgraph. Red nodes represent static `Place` entities, while yellow and orange nodes represent dynamic `EnvironmentState` and `Hazard` nodes, respectively. . 57
5.15 A process example involving new tasks [25]. 60
5.16 The Security System Process 61
5.17 The Emergency Warden and Occupant Processes 62
5.18 Example of a simple flow implementation 64
5.19 The occupant evacuation flow 65

5.20 The occupant signal area flow 66
5.21 The Emergency Conditional Flow 66
5.22 The Process Deployment Flow 67

List of Tables

3.1 Results for each database with given search string. 22
3.2 Eligibility Criteria. 23
3.3 Comparison of state of the art approaches (✓): Fully match. (✗): Doesn't match. 27

5.1 From the conceptual architecture (chapter 4) to the modules and technologies implementing it. 33
5.2 Comparison of AI Agent Frameworks for IoT Hazard Detection 36
5.3 Comparison of evaluated Graph Databases 40

6.1 Summary S0 72
6.2 Illustrative example 73
6.3 Safe Path Cost Overhead (SPCO) distribution across the origins affected by the hazard 73
6.4 Step-by-step evolution 74
6.5 Concrete examples of adaptation 74
6.6 Comparison of Route Success Rate (RSR) Between S0, S1, and S2 . . . 74
6.7 Cross-Scenario Summary Table 75
6.8 Effectiveness per scenario. 77
6.9 Efficiency per scenario 78
6.10 Tool-use behaviour and latency composition 79

7.1 Summary of the research questions and how they were addressed. The full text of each research question is given in section 1.1. 81

Acronyms

AI	Artificial Intelligence 1–3, 5, 9, 24, 36
API	Application Programming Interface 14, 17, 77, 80
BIM	Building Information Modeling v, 5, 11, 21–23, 26, 34, 54, 82, 83, 87
BPM	Business Process Management 12, 41
BPMN	Business Process Model and Notation v, 1–3, 12, 14, 27, 28, 33–35, 41, 42, 54, 57, 59, 60, 64, 65, 67, 81, 82, 84, 85, 87
BPS	Business Process Simulation 13, 14
CFD	Computational Fluid Dynamics 23, 26, 28, 82, 84
COP	Common Operating Picture 8, 9
CoT	Chain-of-Thought 14, 15, 17
DAG	Directed Acyclic Graph 36
DSA	Distributed Situational Awareness 9
DT	Digital Twin 2, 10, 11, 23–25, 27, 28
EM	Emergency Management 5, 6, 8
EWS	Evacuation Wayfinding System 10
FDS	Fire Dynamics Simulator 26, 28, 82
FSM	Finite State Machine 36
GenAI	Generative AI xi, 15, 16, 25
GIoT	Generative IoT 25, 28
HE	Hazard Exposure 71–73
HFR	Hazard-Free Route Rate 71–75, 81
IFC	Industry Foundation Classes 11, 22
IoT	Internet of Things v, xi, xiii, 1–3, 5, 8–12, 16, 21, 23–25, 27, 28, 30, 33, 35, 36, 39, 41, 43–45, 47, 48, 52–55, 57, 82, 87, 88

KG Knowledge Graph 16, 19, 20

LLM Large Language Model v, xi, 14–20, 24, 25, 36, 37, 41, 48, 70, 75–80, 83–85, 87

MAGE Memgraph Advanced Graph Extensions 41
MARL Multi-Agent Reinforcement Learning 26
MAS Multi-Agent System v, xi, 1, 17, 18, 21, 23, 70, 83
MCP Model Context Protocol 24, 41, 51, 53
ML Machine Learning 23

OLTP Online Transactional Processing 39

PW Path Weight 71–74

RAG Retrieval-Augmented Generation 24, 25, 36, 84
RAR Route Adaptation Rate 72, 74
RoG Reasoning on Graph 19, 20
RRI Route Risk Index 23, 26, 27
RSR Route Success Rate xiii, 71–75
RTLS Real-Time Locating System 11, 12

SA Situational Awareness 8, 9
SDT Spatial Digital Twin 11
SES Static Evacuation Signage 25, 26
SLM Small Language Model 14–17
SPCO Safe Path Cost Overhead xiii, 71, 73, 75
SPOF Single Point Of Failure 74, 75, 81

TTL Time to Live 45, 46

1. Introduction

Indoor environments present diverse operational challenges during emergency events. Space is limited, egress points are not many, and panic among occupants often leads to evacuation bottlenecks. Relying on static escape routes introduces a real vulnerability, since these pre-calculated paths cannot account for what is actually happening in the building at that moment. In the worst case, a static route can send people directly toward the hazard instead of away from it. Securing indoor environments during an emergency therefore requires a shift toward routing that can react to what the building's sensors are reporting in real time.

This thesis looks at how a graph representation of a building, IoT data and a MAS can be combined into a working emergency response framework. The building is modeled as a graph of places and connections, enriched with sensor readings coming from the IoT layer. Specialized Artificial Intelligence (AI) agents read this data, classify the hazard, and update the graph accordingly. A BPMN process then uses this updated graph to fetch a safe evacuation route and to trigger the actuators, such as directional lighting, that guide occupants out of the building. The goal is a system that keeps occupants away from a hazard while adding as little extra travel as possible compared to the normal, static route.

1.1 Problem Statement

Indoor emergencies come in different forms, from fires and gas leaks to structural failures. What they have in common are the closed spaces, few ways out, and very little time to decide what to do; occupant panic makes this worse, and it is one of the main causes of evacuation bottlenecks. According to the National Fire Protection Association, 65% of deaths in indoor fires are linked to failed evacuation routes [1]. The building itself plays a role too, since its geometry can make an evacuation harder or easier [2].

Today, most buildings still rely on static evacuation routes, printed on evacuation maps and they do not change once an emergency starts. This is the core problem this thesis works on, a static route has no way of knowing what is happening in the building at that exact moment. If smoke fills a corridor or a hazard blocks an exit, the route stays the same, and in the worst case it can lead people straight into the hazard [3]. Dynamic routing is meant to fix this, by reacting to real conditions such as fire, toxic gas, or blocked corridors, and by updating the route while the emergency is still unfolding [3].

Sensing this kind of condition is no longer hard; buildings are increasingly installed with IoT sensors that can pick up temperature changes, smoke, and other signs of danger. What is still missing is the step that comes after: turning this stream of raw sensor data into a decision fast enough to be useful, and turning that decision into an updated route and clear instructions for the people still inside. Closing this gap between

the sensors that already exist in a building and the response that should follow from them, is the need this thesis tries to address.

To satisfy those needs, three things are needed. A representation of the building that can be read and updated while the emergency is happening. A way to interpret raw sensor data on its own, instead of relying on a person to read it first. A way to define what each type of actor in the building, occupants and emergency wardens among them, should do once a hazard is confirmed. The prototype just described is built around these three needs.

Starting from this problem, the thesis is guided by three research questions.

RQ1. Can specialized AI agents reliably turn raw IoT telemetry into a hazard classification that is accurate enough to drive routing decisions, without a human interpreting the sensor data first?

RQ2. Can BPMN be used to define a response strategy that generalizes across different hazard types without changing the core logic of the system?

RQ3. Does an adaptive approach keep evacuation routes safe, avoiding hazardous areas, while adding only a reasonable amount of extra travel distance compared to a static route?

These questions are addressed through the design and implementation of a prototype, and through its evaluation on the layout of a real building.

1.2 Project Description

Getting a reliable, up-to-date picture of a building during a crisis is not trivial, and acting on it in time is even harder. Without a live view of the environment, even a good routing algorithm risks sending occupants toward a hazard that is still expanding rather than away from it. The approach explored in this thesis represents the building as a graph, where each node is a place such as a room, a corridor or an exit, and each connection represents how occupants can move between them. Every node carries both static attributes, describing the physical space, and dynamic attributes coming from IoT sensors, such as temperature or smoke readings. This graph is built directly from a floor plan, and is closer to a lightweight spatial representation than a complete Digital Twin (DT). On top of this graph, a BPMN process defines what the different actors, occupants and emergency wardens among them, are expected to do once a hazard is detected.

Recent work has started to combine agentic AI with IoT data to turn raw sensor streams into concrete action plans. An example is Morsali and Khajavi [4], who use autonomous AI agents to turn physical layer data into a DT for wildfire disaster management. Their agents convert multisource imagery and weather telemetry into actions such as Unmanned Aerial Vehicle redeployment or crew reallocation, matching emerging fire conditions against precomputed tactics instead of relying on manual analysis. This kind of agentic pipeline, going from raw telemetry to an actionable response, is the same idea this thesis applies to indoor evacuation.

The prototype built for this thesis monitors IoT telemetry and, once a hazard is suspected, triggers a triage step among a set of specialized agents. Each agent is an expert on one type of hazard, currently fire and earthquake, and is responsible for confirming the hazard and estimating its severity for the room under analysis. The

resulting assessment is written back onto the building graph, marking the affected places as unsafe. A BPMN process and then fetch the evacuation route from the graph for every occupant and drives the available actuators, at this stage the building's directional lighting, to guide people along the updated safe path. The system is designed so that, as new hazards appear or existing ones spread, the guidance keeps adapting instead of staying fixed to a route computed before the emergency started. To validate this idea, the framework was evaluated on the layout of a real building, the Department A of the Computer Science division at the University of Camerino, comparing the routes produced under normal conditions against those produced once one or more hazards are introduced.

1.3 Objectives

The main objective of this thesis is to move indoor evacuation guidance from static, pre-computed routes and toward a system that adapts as a hazard develops. This means representing the building as a graph that can be queried and updated in real time, rather than relying on a fixed map handed out in advance. A second objective is to use agentic AI to turn raw IoT telemetry into a hazard classification that other parts of the system can act on, without requiring a human to interpret the sensor data first. A third objective is to use BPMN to define, separately for each type of actor, what should happen once a hazard is confirmed, so that different hazard types and different response strategies can be added without rewriting the core logic of the system. The thesis aims to show that this combination produces routes that stay safe, keeping occupants out of hazardous areas, while adding only a reasonable amount of extra distance compared to the shortest available path. This last point is verified experimentally, by comparing the framework's routing decisions against a static baseline across a set of scenarios with no hazard, a single hazard, and hazards that evolve over time.

1.4 Thesis structure

The thesis is structured as follows. Chapter 2 introduces the concepts the rest of the thesis builds on. It covers emergency management and evacuation behavior, digital twins and how a building can be represented as one, and AI agents, from a single agent's workflow up to multi-agent systems and knowledge graphs. Chapter 3 reviews existing research on the topic. It explains how the literature was searched and filtered, and then goes through the findings to point out the vulnerabilities of static evacuation and the gaps this framework addresses. Chapter 4 steps back from the details and looks at the problem from a higher level. It lays out the design goals behind the framework and presents the conceptual architecture, before Chapter 5 shows how it was actually built. Chapter 5 breaks down the prototype. It covers the overall architecture and technology stack, how IoT data reaches the system through the data integration middleware, how the environment engine keeps the building graph up to date, and how the BPMN engine turns that graph into an evacuation route and a set of actuator commands. Chapter 6 tests the framework through a set of hazard scenarios. It looks at two things separately: how safely and how efficiently the routing behaves, and how well the agents classify hazards from the telemetry they receive. Chapter 7 reflects on these results, going over the limitations of the current implementation and what could be done next. Chapter 8 closes the thesis with a short summary of what was achieved.

2. Background

The following chapters will explore technologies, methodologies, and the current state of the art. This review establishes the necessary theoretical foundation for the proposed framework, detailing how the integration of AI Agents, BIM, and IoT can be synthesized to overcome current limitations in emergency management. Specifically, these technologies will be leveraged in the design phase to solve the fundamental challenges of real-time hazard identification and the autonomous activation of building actuators. By understanding how AI agents can decompose complex emergency objectives into actionable steps and how BIM can provide the necessary semantic structure, this study will demonstrate how to transition from static, pre-calculated evacuation plans to dynamic, adaptive response systems.

2.1 Emergency Management

An **hazard** is defined as a potential source of danger that may precipitate an emergency or disaster, often categorized according to the specific event it initiates [5]. **Risk** is defined as the susceptibility to death, injuries, property damage, destruction, operational disruption and so forth [5]. A **disaster** is defined as an event necessitating a crisis response that requires the mobilization of governmental authority and resources exceeding the capacity of a single agency or service [5].

Emergency Management (EM) is the discipline concerned with risk assessment and mitigation to safeguard human life, property, and environmental integrity. Hazard identification represents the primary objective within the scope of EM activities. Comprehensive risk analysis is a prerequisite for effective strategic planning and mitigation. The realization of a risk may culminate in a disaster, thereby triggering emergency response and recovery protocols [5]. To facilitate systematic disaster administration, EM is traditionally divided into four interrelated phases [5]:

1. **Mitigation**: Actions done to minimize or eliminate long-term risks to individuals and infrastructure.
2. **Preparedness**: The operational state of readiness maintained by a system in anticipation of imminent hazards.
3. **Response**: Immediate actions executed to preserve life, secure property, and address essential requirements during and after a disaster event.
4. **Recovery**: A complex, long-term process designed to restore the system to its pre-incident state.

While all four phases encompass distinct strategic frameworks and operational mandates, this thesis specifically focuses on the response phase. Within this scope, two types

of emergency response can be defined: pre-event and post-event response. The first one consists of prediction and analysis of potential risks and preparing to mitigation. The second one instead starts when the disaster is in progress, it consists to allocate, coordinate and manage resources [6].

Disaster identification is a fundamental first step in the disaster management response phase, enabling decision-makers to rapidly trigger and coordinate interventions. Since these events occours in highly dynamic, uncertain, and resource-constrained environments, the ability to accurately and rapidly identify the occurrence, scope, and specific nature of a disaster dictates the effectiveness of all subsequent rescue and relief operations [7, 8].

2.1.1 Occupant Behavior and Evacuation Dynamics

Because indoor environments can change rapidly during an emergency, such as an earthquake triggering a subsequent fire or structural collapse, EM must consider both human psychology and dynamically changing physical obstacles [9].

An important aspect to take into account is how the occupants respond to indoor emergencies for developing effective evacuation plans and emergency systems. Occupant behavior during emergencies can be classified in [10]:

1. **Pre-evacuation behavior**: When an emergency occours, occupants rarely start evacuating immediately.
2. **Wayfinding behavior**: This is the process of moving toward a safe destination, different emergencies can indicate different safe zones. For example moving to upper floors during a flood, evacuating the building during a fire or sheltering in place during a terrorist attack.
3. **Interaction behavior with others**: Occupants engage in social behaviors such as grouping with known associates, queueing at exits, helping vulnerable individuals or competing for personal space.
4. **Interaction behavior with the environment**: Individuals may exhibit risky behaviors, such as taking shortcuts through hazard zones or attempting to fight the hazard (e.g., trying to extinguish a fire) to protect their property, which often exposes them to greater danger.

In figure 2.1 we can see the occupant behavior during an emergency. The actions, and process flow, taken by the occupants is indeed influenced by diverse factors, for example if the occupant perceives the alert or not, how the individuals interpret the situations and the associated risks or factors that influence the decision about the action. The Phase 1 corresponds to the perception phase, where occupants perceives external alerts, physical or social, from the environment, for example during a fire he/she can see smoke or perceive heat, otherwise can hear discussions or see others in action. During this phase occupants perceive more information, have more uncertainty and time pressure. The Phase 2 is the interpretation phase, in this phase the occupant tries to interpret the situation, considering whether there is a false alarm or a hazard and if there are risks to themself or others. In the third Phase the behavioral process is activated, where the occupants make decisions about what to do next based on their interpretation. In the last Phase occupants may perform the actions that they decided in the previous step. Each time that an occupant interrupts its daily routine with cues or new information, a new behavioral process begins.

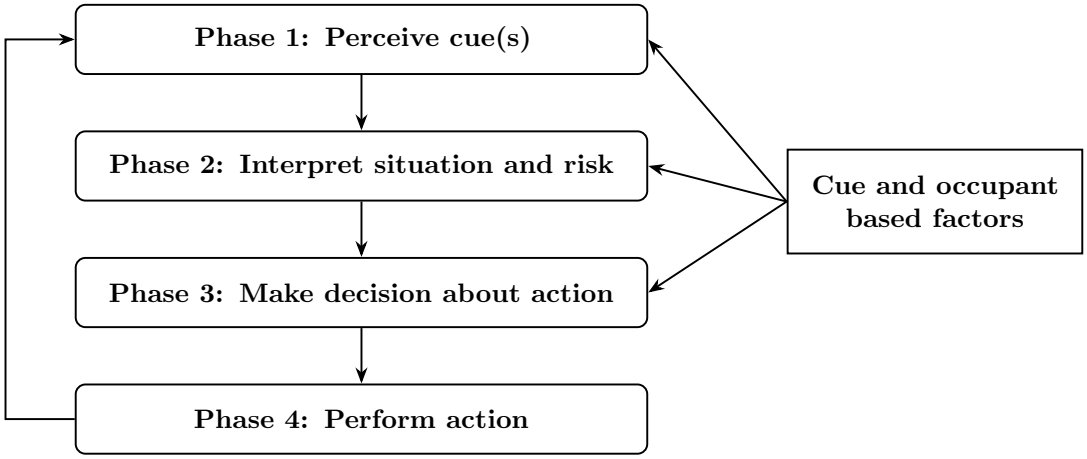


Figure 2.1: The behavioral process of occupant response [11].

Panic and Bottleneck Dynamics

The physical dynamics of a human crowd becomes critical when spatial constraints reduce the available area, forcing pedestrians to move, flowing like a river [12]. A bottleneck is formed when a continuous flow of individuals converges upon a narrowing space, such as a door or corridor, which physically restricts the number of people who can pass simultaneously and creates a funneling effect [12]. As the crowd compresses against an obstacle, the physical density threshold becomes a critical determinant of collective safety. Lujak and Ossowski [13] indicate that safe crowd densities should remain below 4 persons per square meter to prevent accidents; when densities exceed 7 persons per square meter, the crowd becomes too dense, leading to extreme turbulence and uncoordinated force propagation, causing people to fall [12].

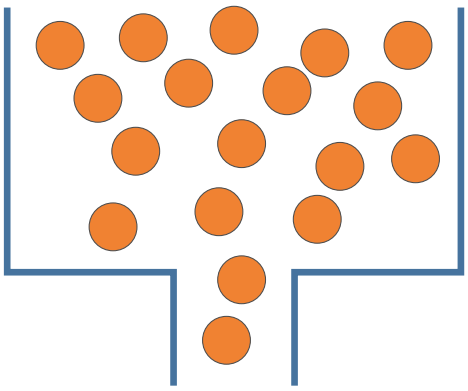


Figure 2.2: A bottleneck situation [12].

The transition from organized movement to chaotic movements is linked to the psychological deterioration of evacuees, specifically the overloading of their cognitive capacity under conditions of extreme time pressure and uncertainty. When an immediate life-threatening hazard is perceived, humans experience an automatic stress response that severely limits logical thinking and cognitive problem-solving abilities [12]. This extreme anxiety cancel out the interpretation phase and subsequent behavioral process of wayfinding. Normally, effective wayfinding relies on an individual's ability to cognitively

perceive environmental signals, such as visually recognizing an emergency exit and logically understanding how to use it [12]. However, panic and excessive stress diminish sensory functioning, reduce Situational Awareness (SA), and induce spatial disorientation [13]. Unable to analyze ambiguous environmental information or evaluate dynamic risks, individuals suffer from cognitive overload and abandon rational decision-making. Consequently, evacuees default to instincts, following familiar routes or engaging in herding behaviors where they completely surrender their individual agency to follow the movement of the immediate majority [12].

The instincts to group and the reliance on familiar paths ultimately expose the fatal limitations of static evacuation routes under pressure. Empirical studies confirm that in high-stress scenarios, evacuees routinely ignore static emergency signage, choosing instead to trust their instincts or blindly follow the crowd [12]. Because static routes and uncoordinated evacuation policies treat evacuees as a homogenous mass and cannot adapt to real-time hazard conditions, they inadvertently guide large groups toward the exact same exits [14]. This group behavior creates artificial bottlenecks, heavily over-utilizing specific pathways while alternative, viable exits remain entirely ignored, or leading to people to dead ends [13]. This fragmented, static approach to EM frequently triggers cascade events where local congestion quickly overwhelms the flow capacity of the structure [14]. Preventing the onset of mass panic and mitigating these crowds necessitates abandoning static directives in favor of dynamic, real-time routing systems capable of distributing the pedestrians load, responding to live environmental variables, and alleviating the cognitive load on evacuees through adaptive, context-aware guidance [14].

2.1.2 Situational Awareness

SA is defined as the capacity to identify, interpret, and formulate responses to dynamic crisis conditions. Although risk mitigation protocols attenuate the most prevalent hazards, they do not neutralize all potential dangers; consequently, emergent crises frequently diverge from anticipated scenarios [15]. SA addresses informational gaps wherein responders lack comprehensive visibility regarding real-time environmental states. To achieve efficacy, a dynamic feedback loop is established wherein affected communities and response personnel continuously monitor environmental conditions, interpret incoming signals, and anticipate subsequent consequences. The primary objective is to minimize the latency between the onset of a disaster and the formulation of a Common Operating Picture (COP) [15].

Traditional risk assessment methodologies focus on well-established hazards and frequently fails to detect rare or compound risk scenarios. Conversely, SA provides the mechanism to detect and respond to unanticipated and unmitigated threat scenarios prior to their escalation. By aggregating heterogeneous data streams, such as those generated by the IoT, SA facilitates the identification of systemic anomalies and interdependencies that conventional approaches typically overlook. This continuous monitoring paradigm enables the detection of weak signals preceding imminent hazards, thereby empowering response organizations to execute decisive interventions [15].

SA adopts a proactive approach by continuously updating the comprehension of evolving hazards, synthesizing disparate data sources into a cohesive analytical model, and flagging early warning indicators of emerging threats. Equipped with this capability, decision-makers obtain a comprehensive operational overview of the emergency, enabling the optimized deployment of resources and the issuance of preemptive warnings while

also accounting for informational blind spots. This framework bypasses the reactive nature of traditional approaches, permitting responders to detect hazards and anticipate cascading consequences, such as structural compromises or population migrations. The operational lifecycle is divided into three phases: during the "blue sky" phase, the environment is monitored to identify systemic vulnerabilities and inform preemptive mitigation strategies. Within the "grey sky" phase, indicates an imminent hazard onset, SA drives agile decision-making and optimal resource allocation, thereby constraining preventable losses. During the post-disaster recovery phase, SA is helpful in delineating restoration priorities and tracking recovery progression [15].

The inherent complexity of SA implementation comes from disaster environments being characterized by fragmented information architectures and extreme organizational pressure, necessitating coordination across personnel, computational processes, and deployed technologies [15].

Hazard Detection and Real-Time Telemetry

Modern environmental and structural safety frameworks rely on distributed sensor networks, IoT devices, and AI models to establish dynamic SA through continuous monitoring [15, 16]. In this context, real-time telemetry acts as the foundational data layer that enables the three hierarchical levels of SA:

1. Perception (Level 1): Telemetry systems capture raw environmental data via IoT sensors to detect the presence of specific hazards.

2. Comprehension (Level 2): The system integrates and analyzes this data to understand the significance of the detected hazards and their immediate impact.

3. Projection (Level 3): Based on current trends, the system forecasts the evolution of the situation to predict cascading effects and potential risks [15].

While this individual framework is fundamental, contemporary research suggests that complex disaster responses often exceed individual cognitive or systemic capacities. This has led to the emergence of Distributed Situational Awareness (DSA). In a DSA model, awareness is not localized within a single node but emerges from the continuous exchange, cross-verification, and alignment of information across multiple stakeholders and technological platforms [15].

Under normal conditions, defined as "blue-sky" SA, the environment is monitored continuously to anticipate potential hazards through redundancy analysis and risk identification. The scope of blue-sky SA encompasses preparedness and mitigation, maintaining baseline environmental awareness to facilitate predictive intelligence and long-term situational understanding during steady-state operations [15]. Conversely, during a hazard event, designated as "grey-sky" SA, the acquisition of high-fidelity data becomes critical for executing a precise operational response. Real-time data streams supply continuous updates regarding the environmental state, equipping decision-makers with actionable intelligence to deploy deterministic interventions rather than relying on ad hoc improvisation. Therefore, grey-sky SA prioritizes the establishment of a COP through continuous data aggregation, dynamic updates, and continuous information triage [15].

2.1.3 Active Evacuation Guidance Systems

Effective wayfinding is critical in hazardous environments, enabling occupants to safely navigate optimal evacuation routes via real-time routing algorithms. Historically, inefficient evacuation routing has been a primary reason for fatalities during emergencies. Environment familiarity and prior safety training significantly enhance an individual's capacity to identify optimal routes, thereby mitigating congestion at primary exits. Furthermore, psychological variables and group behavior frequently result in occupants converging on identical paths, which increases required exit times and occasionally directs individuals towards compromised areas. Traditional signage provides spatial orientation and indicates nearest exits; however, such static information is often insufficient within the rapidly evolving parameters of dynamic emergencies [17–19].

To address these limitations, Evacuation Wayfinding Systems (EWSs) were developed. These systems deliver unambiguous navigational directives to evacuees during an active crisis. EWSs utilize highly visible, strategically placed signage across various routes to guide individuals toward less familiar but safer alternative exit paths. Consequently, this methodology actively changes occupant behavior under pressure, substantially improving overall evacuation efficacy. The optimization of signage parameters including inter-sign distance, typology, spatial location, and trajectory positioning, is calibrated against occupant demographics and visual metrics [17, 19]. The architectural classification of EWSs is distinguished on three primary attributes:

- **Location**: High-level placement (above door height) or low-level placement (lower part of walls); the latter demonstrates superior visibility in smoke-dense environments.
- **Illumination**: Internal illumination (e.g., active light sources) or external illumination; internal illumination is optimal for preventing navigational disruptions during low-visibility scenarios.
- **Distribution density**: Punctual configurations (localized at intersections and terminal exits) or continuous configurations (maintaining a minimum frequency of one directional indicator per five meters), with continuous deployment proving to be superior.

Furthermore, EWSs are categorized into passive and active systems: passive systems display static directions, whereas active systems dynamically recalculate and update routing information in real time based on continuous environmental data ingestion [17].

Advancing beyond standard computational simulations, Balboa et al. [19] conducted empirical evaluations of EWS performance within physical environments. Their experimental trials demonstrated an 89% compliance rate with dynamic emergency signage, correlating with a statistically significant reduction in Total Evacuation Time.

2.2 Digital Twin

"A Digital Twin is a virtual representation of real-world entities or processes, synchronized at a specified frequency and fidelity" [20]. Combined with IoT, DTs allows a flow of data between the virtual representation and the physical one allowing to monitor operations, simulate scenarios and predict future environment state and outcomes, then those insights can be delivered to physical devices that can operate to reach a goal [20].

The evolution of modeling complex systems, from static diagrams, is categorized into three distinct concepts, differentiated primarily by the flow of data between the physical and digital spaces:

- A **Digital Model** is a static digital representation of a physical entity. More specifically, a single model could not be sufficient to fully describe a physical entity, so a single perspective is represented instead of multiple perspectives. An example are 2D/3D graphical models. The characteristic of a Digital Model is that the data flows between the physical space and digital space manually, requiring human intervention to maintain and update the model [20].
- A **Digital Shadow** extends a digital model by integrating data from the physical space, typically via IoT devices. This creates an automatic unidirectional data flow from the physical space to the digital space, ensuring that the digital representation mirrors the current state of the physical counterpart [20].
- A **Digital Twin** not only reflect the real world environment but also actively influences it through a bidirectional automatic data flow. While a Digital Shadow is mainly used for monitoring, a Digital Twin allows to analyze, simulate behaviors and support decision making that will be given to the physical entity [20].

The differing levels of data integration between these three concepts are illustrated in figure 2.3.

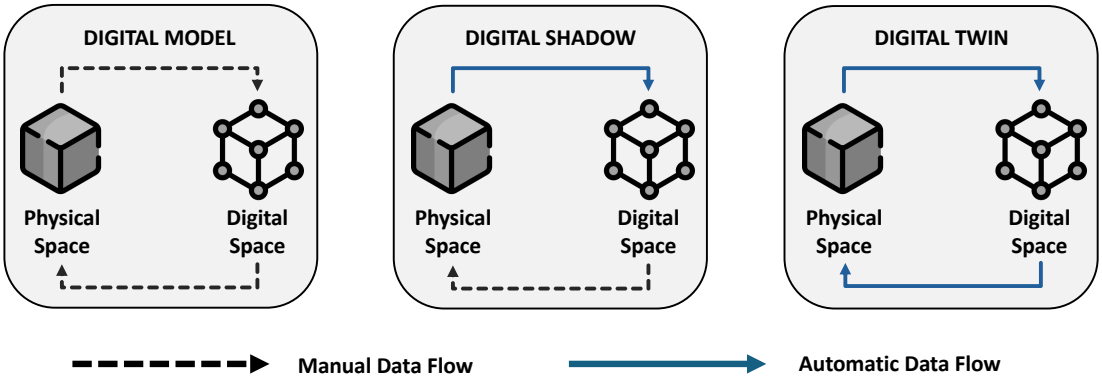


Figure 2.3: Data flow levels between physical and digital spaces [20].

2.2.1 Spatial Digital Twin

A Spatial Digital Twin (SDT) is a specialized type of DT that merges the static geometry of a building. Structurally, the SDT relies on a geometry-based state representation that encodes the exact shape, topology, and physical attributes of the environment [21]. Utilizing standardized interoperability formats such as Industry Foundation Classes (IFC), BIM supplies the comprehensive 3D spatial boundaries and connectivity relations required to map the physical infrastructure into a virtual space [22].

However, a SDT goes over a passive 3D model by establishing a bidirectional connection with the physical world. Through the integration of the IoT, edge sensors, and Real-Time Locating System (RTLS), the virtual entity is continuously updated by live data regarding the physical environment [23]. For instance, during an active emergency, distributed sensors provide immediate updates on hazard propagation such as rising

floodwater levels or smoke accumulation, while RTLS pinpoint the exact locations and density of building occupants who may be trapped [23].

2.2.2 Interplay of Business Processes, Environment and IoT

Business Process Management (BPM) provides a systematic approach to identifying, discovery, modeling, analyzing, and continuously optimizing an organization's operations [24]. Traditionally, business processes have been conceptually separated from the physical environment in which they operate. However, the rise of the IoT has fundamentally bridged this gap. The ability of those devices to collect data through sensors and executing actions through actuators helps to improve processes and enhance efficiency [24]. By digitalizing IoT smart objects can lead to increased efficiency, control, productivity, automation and reduced error-prone human intervention, revolutionizing how businesses reach their goals [24]. *Living Models* concept have emerged as an evolution of the field of IoT systems development by having a virtual representation of physical objects that, with real-time synchronization, replicate characteristics, states and behaviors. Allowing the interplay between IoT and BPM by digitalizing IoT entities from physical world to the digital world of business processes [24].

To design and manage these workflows, BPMN is globally recognized as the standard language for business process modeling [24]. It serves as a visual language for documentation, communication and visualization of processes, easily understood by business analysts, technical developers, and managers [24]. BPMN relies on four core categories of graphical elements to represent system behaviors [24]:

- **Flow Objects**: Events, tasks/activities, and gateways (decision points) that define the path of the process.
- **Connecting Objects**: Sequence flows and message flows that link elements together.
- **Swimlanes**: Pools and lanes that organize activities based on the responsible parties or departments.
- **Artifacts**: Elements like data objects or text annotations that provide supplementary information.

In addition to those elements, they can be enriched with symbols to represent additional information [24]. In IoT-enhanced scenarios, standard BPMN can be extended. For example, specific "sensing tasks" or "actuation tasks" can be modeled to represent the exact moments a system reads data from a sensor or triggers a physical device [24].

Business processes operate within a physical space defined by spatial dimensions (metric properties of a tangible space and navigated by participants) and contextual dimensions (domain knowledge that dictates how the space is used, perceived and utilized, such as safety rules or room purposes) that reveals interactions beyond the spatial dimensions. The ability to leverage those environmental dimensions is referred to as *environmental awareness* [25].

In this way environmental awareness coupled with BPMN enables participants to perform activities based on the environment, by moving or holding a position, and the environment can influence tasks and decisions of processes. This establishes a bidirectional relationship: the status of the environment can constrain activity execution, drive decision gateways, and trigger events, while business process tasks can actively modify the environment or physically relocate participants within it [25].

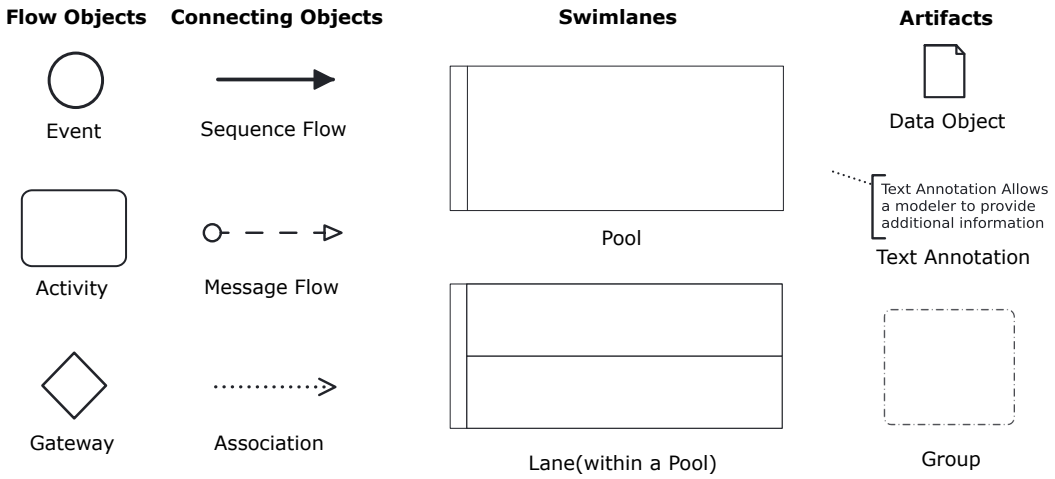


Figure 2.4: Core Set of BPMN Elements [24]

Business Process Simulation

Identifying defects within process models necessitates rigorous analysis prior to real-world deployment and during runtime execution [24]. Business Process Simulation (BPS) is employed to facilitate this analytical requirement for Business Process Models [24]. BPS constitutes a methodological framework designed to model, analyze, and optimize organizational operations [24]. The technique leverages dynamic modeling to comprehend system behavior, identify structural inefficiencies, and evaluate the efficacy of proposed architectural changes [24]. Furthermore, simulations allow to answer "what-if" scenarios and the comparative evaluation of alternative redesigns, thereby establishing a responsive strategy for process optimization [24]. The application of simulation methodologies offers several advantages [24]:

- **Facilitates the comprehension** of existing architectures and the predictive modeling of future states.
- Provides a **cost-efficient** paradigm for evaluating multiple architectural alternatives instead of physical experimentation.
- Ensures a **risk-free environment** for exploring highly variable operational conditions.

Conversely, these techniques exhibit disadvantages [24]:

- Computing a simulation can be time-consuming.
- Interpretation and validation process of simulation outputs can be very complex.
- Experiments may not fully replicate real world scenarios.

Key Performance Indicators are critical for evaluating process performance and efficacy; they provide quantitative metrics to estimate simulated process behavior, with the most prevalent indicators being [24]:

- **Cycle Time Distribution**: The duration of a process, encompassing *Process time* (active resource engagement) and *Delay time* (idle duration of a work unit pending subsequent operations).
- **Waiting Time Distribution**: The temporal interval during which a work item waits for resource allocation.
- **Cost Distribution**: BPS provides the total cost of a scenario, with also the cost distribution of the process instance.
- **Resource Utilization**: The quantitative measure of resource allocation efficiency and functional efficacy.

Ultimately, simulation modeling tools are indispensable for reproducing and analyzing complex workflows, offering the necessary functionalities to design diverse perspectives of business processes. To formalize this architecture, the BPMN standard is used as the foundational modeling language [24].

2.3 AI Agents

Autonomous agents, characterized by their ability to perceive environmental stimuli, execute decision-making processes, and perform actions, have been extensively deployed across diverse industrial and research domains. However, a significant constraint of traditional agentic systems is their inherent lack of adaptability to heterogeneous and dynamic scenarios from their rule-based nature. To address these limitations, LLMs have been integrated to provide a cognitive foundation, thereby augmenting agent capabilities in autonomous decision-making through zero-shot reasoning, Chain-of-Thought (CoT) prompting, tool utilization, and complex task decomposition into manageable subtasks that require distinct reasoning and action planning [26, 27]. Conventionally, these agents interface with LLM via Application Programming Interface (API) endpoints; however, in hazardous or remote environments where internet connectivity is intermittent or unavailable, high-latency, cloud-dependent architectures become untenable. While local hosting of an LLM presents a potential alternative, the deployment of such models is frequently precluded by the requirement for specialized high-performance hardware, prohibitive Video Random Access Memory requirements, memory bandwidth bottlenecks and the proprietary nature of many state-of-the-art architectures. Given that agentic subtasks are often repetitive, domain-specific, and non-conversational, smaller models emerge as a viable solution due to their computational efficiency, predictability, and cost-effectiveness on commodity hardware [27]. By compressing model parameters from 32-bit floating-point (FP32) representations down to 8-bit or 4-bit integer formats, quantization significantly reduces the memory footprint and storage overhead required for edge deployment. This optimization significantly reduces memory footprint and storage overhead while preserving near-lossless performance; specifically, 8-bit settings often demonstrate a negligible performance drop of less than 0.5%, whereas 4-bit configurations can lead to a sharper degradation in accuracy [28]. Consequently, Small Language Models (SLMs) represent a robust alternative, providing reduced latency, lower computational overhead, and enhanced inference efficiency for edge deployment while sustaining competitive performance levels [27]. Furthermore, the decomposition of tasks into specialized sub-components enhances system modularity through the development of sub-agents, facilitating adaptation to evolving requirements and allowing for the

integration of heterogeneous models with varying parameter scales to better align with the diversity of real-world tasks [27].

2.3.1 The Agentic Workflow

While LLMs and SLMs are optimized for single-step tasks, agentic workflows facilitate the autonomous management of complex, multi-step processes and environmental interaction. The fundamental shift from conventional GenAI to agentic workflows involves the transition from single-step responses to iterative computation, as you can see on figure 2.5, utilizing reasoning capabilities to address problems within a structured lifecycle [29]:

1. **Problem Decomposition**: Agents utilize reasoning to partition a primary task into discrete sub-tasks, producing intermediate results that allow the system to manage complex requirements. This process is implemented through several methodologies [29]:
    - *Multi-step Solutions*: Complex objectives are addressed via multi-step reasoning, which may be facilitated through prompting techniques such as CoT or by fine-tuning models to decompose and execute sub-tasks.
    - *Hierarchical Decomposition*: Tasks are partitioned into high-level objectives, which are subsequently refined into granular, specific actions.
2. **Reflection**: This phase involves the assessment and validation of generated outputs to iteratively refine and optimize system performance. However, achieving efficacy through prompting alone remains challenging; consequently, various approaches have been developed to mitigate these limitations [29]:
    - *Verification*: This mechanism permits the termination or re-execution of the reasoning process when intermediate results are erroneous or uncertain; one strategy involves employing multiple alternative reasoning chains to ensure self-consistency.
    - *Refinement*: This prompting technique enhances output quality by iteratively appending previous results to the input context.
3. **Search and Planning**: This methodology involves the generation of diverse reasoning trajectories. Because evaluating exhaustive reasoning chains is computationally intensive, an efficient strategy involves pruning trajectories that yield suboptimal results. Erroneous steps may be refined, whereas viable paths are maintained to explore alternative continuations [29].

2.3.2 Memory Architectures

In contrast to conventional GenAI models that operate statelessly by treating each prompt in isolation, autonomous agents utilize diverse memory architectures to retrieve pertinent information and enhance reasoning for future actions [30]. Internal memory consists of information synthesized during the reasoning phase, which enables the agent to maintain a stateful record of executed operations. Conversely, external memory integrates multi-source data, such as historical conversation logs or structured external databases [30].

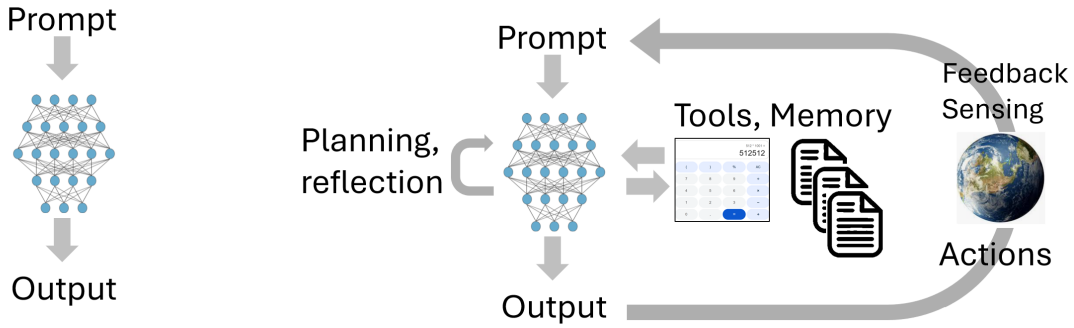


Figure 2.5: Comparative representation of conventional GenAI (left) and Agentic AI (right) [29].

A fundamental utility of memory is the facilitation of self-evolution; through continuous interaction with the environment, memory enables experience accumulation by recording errors, failed attempts, and successful strategies. This archival process allows agents to address analogous scenarios with increased efficacy in future iterations. Furthermore, memory supports environmental exploration by incentivizing novel actions, thereby expanding the agent's knowledge base. To achieve self-evolution, agents can perform knowledge abstraction by distilling raw observations into applicable rules, which promotes adaptability within previously unencountered environments. Memory sources are classified into three distinct categories [30]:

- *Inside-trial Information*: This category includes salient historical steps taken within a single trial, functioning as short-term memory to support immediate decision-making; however, exclusive reliance on this mode may impede the acquisition of generalizable knowledge.

- *Cross-trial Information*: This serves as long-term memory by aggregating historical steps across multiple trials, encompassing successful and unsuccessful actions along with their associated causal insights.

- *External Knowledge*: LLM and SLMs can incorporate external textual knowledge from heterogeneous sources, including structured SQL and NoSQL databases, Knowledge Graph (KG), and IoT data streams.

Agents interact with memory through several fundamental mechanisms [30]:

- *Memory Writing*: The agent persists selected segments of perceived raw or noisy data for future utilization, necessitating filtering mechanisms to determine informational relevance.

- *Memory Management*: This process involves optimizing memory through the generation of abstract representations, the merging of redundant entries, and the purging of obsolete data.

- *Memory Reading*: During the reasoning phase, the agent retrieves relevant information to enhance decision-making accuracy, requiring efficient access to specific entities within massive, heterogeneous datasets.

2.3.3 Tools

Autonomous agents interact with their environments through the utilization of tools. Tools are external functional interfaces invoked by an agent to facilitate environmental interaction via reliable software functions, thereby increasing operational efficiency, mitigating errors, and enhancing system process transparency [29]. When multiple tools are integrated within a single agent, the reasoning phase requires the autonomous selection of the most appropriate tool for the current context. Tool selection capabilities are not inherent to all LLM or SLMs; consequently, various methodologies have been developed to implement this functionality. Proposed strategies include adapting the CoT paradigm for tool selection (used by ReAct framework [31]), fine-tuning foundation models for specific API calls, and integrating tool selection directly into the agentic decision-making framework [29]. Additionally, LLMs facilitate complex task execution by generating code for runtime interpreters, producing SQL queries for relational databases, or utilizing other agents as modular tools to delegate subtasks to specialized systems [29]. Patil et al. [32] propose Gorilla, a finetuned model with the capability of accurately select the right tools from massive amount of API while reducing hallucinations.

2.3.4 Multi-Agent Systems (MAS)

Current LLM-based agent architectures have evolved to incorporate multiple cooperating agents, each one having distinct profiles, interaction protocols, and decision-making mechanisms. This multi-agent paradigm leverages divergent thinking to systematically enhance reasoning capabilities and outcome validation [33, 34]. The efficacy of this architectural approach is based on several core factors:

- LLM-based agents, inherently optimized for Natural Language Processing, can be used in specialized inter-agent and human-agent conversation, thereby elevating output quality through structured conversational roles such as observation, critique, and validation.
- Given the diverse functional capacities inherent to various LLMs, communication among heterogeneously configured models facilitates the modular integration of these specialized capabilities.
- As individual LLMs frequently exhibit limitations in resolving highly complex objectives, the decomposition of such problems into manageable sub-tasks (managed through inter-agent cooperation) significantly improves overall output quality.

Figure 2.6 delineates the generalized architecture of a MAS. This framework highlights four critical dimensions: the agent-environment interface, agent profiling, inter-agent communication, and capability acquisition.

As illustrated in figure 2.6, operational environments are categorized as Physical, Sandbox, or None (human-centric); this research primarily focuses on the Sandbox environment. A sandbox constitutes a simulated or virtual environment where agents interact to iteratively experiment with diverse actions and strategic policies [34].

Agents are formally defined by distinct traits, actions, and specialized skills tailored to achieve specific objectives. Influenced by the target environment, agents dynamically manifest varying capabilities and behavioral patterns. Agent profiling methodologies generally fall into three categories: *Pre-defined* profiles, which are explicitly codified by system architects; *Model-generated* profiles, which are autonomously instantiated

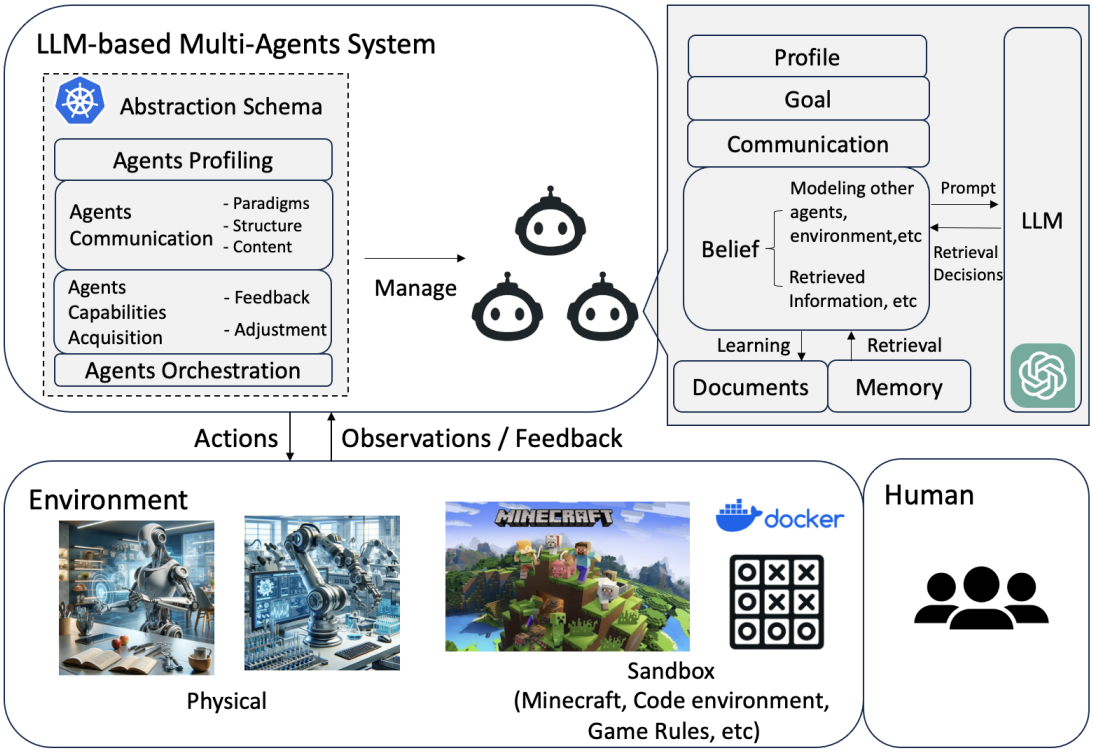


Figure 2.6: The architecture of LLM-based MAS [34]

by foundational models such as LLMs; and *Data-driven* profiles, which are constructed from pre-existing datasets [34].

Another foundational element of MAS architecture involves the protocols governing inter-agent communication. Agent communication is structurally analyzed through two distinct paradigms: interaction modalities and topological configurations. Contemporary MAS frameworks predominantly employ three interaction paradigms. In the *Cooperative* paradigm, agents collaborate to achieve a unified objective. The *Debate* paradigm facilitates argumentative interactions, individual agents propose, defend, and mutually critique solutions, representing an optimal approach for iterative solution refinement. Conversely, the *Competitive* paradigm dictates that agents pursue independent objectives, resulting in conflicting states. Regarding topological structures, communication networks are commonly organized into layered, decentralized, or centralized architectures. *Layered* communication enforces a hierarchical organization where agents operating at distinct levels have specific roles. *Decentralized* communication facilitates peer-to-peer network interactions, a topology frequently implemented in open-world simulation applications. *Centralized* communication necessitates a primary orchestration agent that coordinates subordinate nodes, restricting inter-agent communication to pathways routed exclusively through this central node [34].

The mechanism by which agents dynamically acquire and refine capabilities is very important in MAS. This process is based on two core operational concepts: the integration of feedback mechanisms to enhance agent proficiency, and the deployment of adaptive strategies for complex problem resolution. Feedback mechanisms are classified into several modalities: *Environmental* feedback, derived from physical or simulated environments; *Inter-agent* feedback, sourced from peer evaluations; *Human* feedback, integrated to align agent outputs with human values and preferences; and *None* feedback,

where agents operate autonomously without external correction. Capability enhancement strategies for complex problem solving are generally divided into three approaches. The *Memory-based* approach, detailed in section 2.3.2, enables the system to retrieve and apply historically successful action sequences relevant to analogous scenarios. The *Self-evolution* strategy bypasses static memory retrieval, allowing the agent to iteratively modify initial objectives and strategies during runtime. Finally, *Dynamic Generation* empowers the system to autonomously instantiate new agents specifically designed to address emergent, real-time tasks [34].

Darwish, Rashed, and Khoriba [35] propose a multi agent system that solves the problem of LLM hallucinations and output inconsistency. It proposes an architecture composed by *Consultant-Evaluator* pairs that breaks complex tasks into subtasks to ensure output reliability. It employs two different LLMs, the Consultant generates responses that an Evaluator verifies via rule-based similarity scoring, flagging errors for corrections. This feedback loop achieved an 85.5% improvement in consistency [35].

Despite their potential, autonomous agents encounter significant structural and operational obstacles. These constraints are fundamentally inherited from the underlying LLMs, manifesting as hallucinatory outputs and sub-optimal reasoning capabilities. Furthermore, current frameworks exhibit deficiencies in essential agentic properties, specifically regarding operational autonomy and proactivity. These limitations frequently culminate in the failure of agents to execute long-term strategic planning or perform autonomous error recovery [36].

2.3.5 Knowledge Graphs

LLMs over the years have demonstrated their capabilities to understand natural language, they remain susceptible to hallucinations and exhibit restricted multi-step reasoning capabilities. Furthermore, the finite context window of these models constrains their ability to maintain long-term memory across extended interactions. To mitigate these limitations, KGs are utilized to structure, organize, and interconnect disparate facts, serving as a deterministic foundation for reasoning [37]. KGs represent human knowledge through a structured, graph-based representation that connects entities, real-world objects, and abstract concepts via edges. These edges provide a semantic description of relationships, such as *born in* or *member of*, facilitating a rigorous mapping of interconnected data [38, 39]. The graph-based data model is particularly effective for integrating, managing, and extracting insights from heterogeneous data sources. This architecture enables semantic abstraction across various domains and captures complex dependencies through multi-hop paths and relational edges [39]. Property graphs are frequently used to implement KGs to append attributes or metadata directly to nodes and edges. Within the Resource Description Framework specification, knowledge is instantiated as triples in the format of (*subject*, `predicate`, *object*), or (*head*, `relation`, *tail*). A representative example of this structure is (*Albert Einstein*, `WinnerOf`, *Nobel Prize*) [38]. Primary applications of KGs encompass natural language understanding, dialogue systems, recommendation engines, and specialized fields such as bioinformatics and clinical medicine [38, 39]. KGs are accessed via domain-specific query languages including SPARQL, Cypher, Gremlin, and G-CORE [39]. Autonomous agents leverage KGs to enhance response fidelity by traversing graph structure to derive contextual transitions.

Luo et al. [37] introduced Reasoning on Graphs (RoGs), a method designed to mitigate hallucinations in LLMs through a planning-retrieval-reasoning framework. This

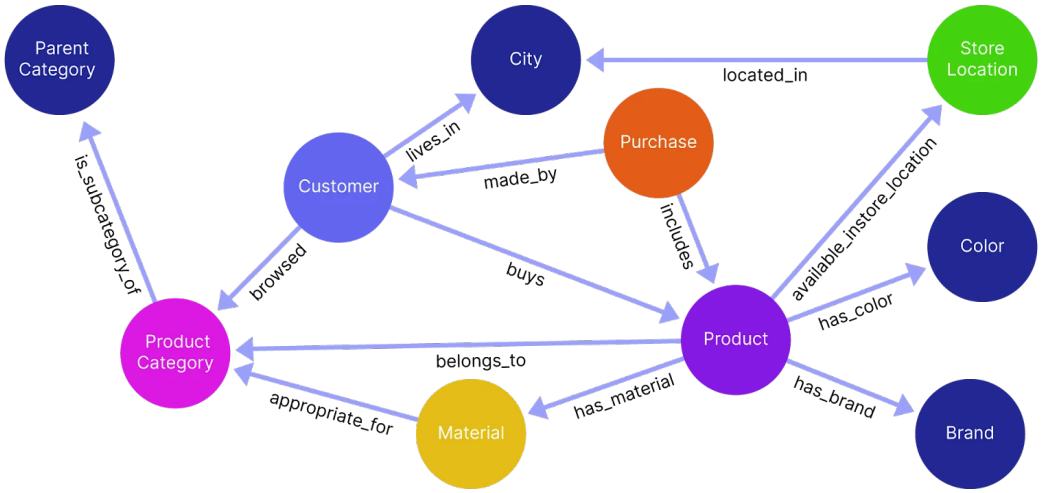


Figure 2.7: An example of Knowledge Graph [40]

framework grounds LLM reasoning processes within the structured, factual evidence provided by KGs. To prevent the generation of erroneous reasoning steps, RoG directs the LLM to formulate relational paths constrained by the KG topology. This constraint ensures that the generated plans are grounded in verifiable data, facilitating faithful and interpretable reasoning. These relational paths serve as navigational queries to retrieve valid, updated reasoning subgraphs from the KG [37].

3. Literature Review

The initial phase of this research involved a comprehensive examination of the existing literature to establish a foundational understanding of MAS applied to emergency management coupled with dynamic routing and Digital Twins. Elsevier, IEEE and ACM served as the primary academic databases, where specific keywords and boolean search strings were executed to aggregate highly relevant literature. Subsequently, manual inclusion and exclusion criteria were applied to filter the corpus, prioritizing articles that demonstrated direct alignment with the research objectives and were accessible via institutional repositories or open-access publishing.

3.1 Methodology

This literature review examines how *Agentic AI*, BIM, and the IoT intersect, building the theoretical and methodological foundation for our proposed multi-hazard framework. To spot existing research gaps, this review targets recent projects addressing dynamic escape routing and real-time hazard adaptation, but also incorporates complementary projects to capture the broader context of the field. The search strategy was refined iteratively to improve the relevance of the results and capture newer methodologies.

3.1.1 Database and Query Search

Searches were conducted across three primary databases—Elsevier, IEEE, and ACM—to cover these interdisciplinary fields. These specific information sources were selected to cover high-impact, peer-reviewed publications spanning computer science, civil engineering, and disaster management.

Search terminology was divided into two primary categories: technological architectures (e.g., Digital Twin, BIM, IoT, AI Agents) and use cases (e.g., Emergency, Evacuation). Synonyms, wildcards and boolean operators were used to capture variations across the literature. The constructed search strings were applied to the titles, abstracts, and keywords of indexed publications. The exact search strings utilized across the databases are shown in table 3.1.

To align the initial query with established eligibility criteria, the search was restricted using database filters, limiting the results to research and review articles published in engineering and computer science disciplines.

[1] Due to database limitations regarding logical connectors, this query is executed as two separate searches and aggregated the results.

Table 3.1: Results for each database with given search string.

Database	**Articles**	**Search String**
Elsevier[1]	25	(("Digital Twin" OR "BIM" OR "Building Information Modeling") AND ("Multi-Agent System" OR "MAS" OR "Agentic AI") AND ("Evacuation" OR "Emergency" OR "Dynamic Routing")) OR (("Digital Twin" OR "BIM" OR "Building Information Modeling") AND ("Multi-Agent System" OR "MAS" OR "Agentic AI") AND ("Hazard" OR "Disaster" OR "Risk"))
IEEE	8	(("All Metadata":Digital Twin*) OR ("All Metadata":BIM) OR ("All Metadata":Building Information Model*)) AND (("All Metadata":Multi*Agent System*) OR ("All Metadata":MAS) OR ("All Metadata":Agentic AI)) AND (("All Metadata":Evacuat*) OR ("All Metadata":Emergency) OR ("All Metadata":Dynamic Rout*) OR ("All Metadata":Hazard*) OR ("All Metadata":Disaster*))
ACM	7	[[All: "digital twin*"] OR [All: "bim"] OR [All: "building information model*"]] AND [[All: "multi*agent system*"] OR [All: "mas"] OR [All: "agentic ai"]] AND [[All: "evacuat*"] OR [All: "emergency"] OR [All: "dynamic rout*"] OR [All: "hazard*"] OR [All: "disaster*"] OR [All: "risk"]]

3.1.2 Inclusion and Exclusion Criteria

Inclusion and exclusion criteria have been defined prior to screening to mitigate selection bias. The criteria were defined based on the research objectives, focusing specifically on the integration of digital tools in emergency management. These criteria included document type, language, and topic, as detailed in table 3.2.

The screening process occurred in two phases. In the first phase, the title, abstract, and keywords of each article were evaluated to ensure they met the review's objectives. In the second phase, a manual full-text review of the remaining articles assessed their relevance and detail to confirm they met all eligibility parameters.

3.2 Findings

Traditional static evacuation plans often fail in complex buildings because they are unable to adapt to real-time events or emerging bottlenecks [41]. Dynamic route planners must constantly monitor and adapt to the state of a building to minimize evacuee travel time and hazard exposure [41]. The foundational spatial awareness for these dynamic systems is frequently established by extracting semantic and geometric data from BIM via IFC to generate 3D indoor navigation networks [42–45]. When converting floor plans into navigation graphs, it is essential to map out physical constraints, like door widths and stairwell limits, so that important escape data is not lost. [44]. Once that the graph-based representation is built, routing algorithms—typically Dijkstra's—are applied using environmental data to weight the routes [41]. To address the computational overhead of continuously recalculating shortest paths across large scale BIM networks, heuristic approaches like A* or genetic algorithms are increasingly implemented to ensure near real-time response times during rapid hazard propagation [41]. For instance, real-time models like EvacuSafe dynamically calculate the lowest risk path by removing incoming

Table 3.2: Eligibility Criteria.

Criterion	Inclusion	Exclusion
Publication Type	Peer-reviewed journal articles and international conference proceedings.	Grey literature, white papers, thesis dissertations, book chapters, and non-peer-reviewed preprints.
Language	Articles written in English.	Articles published in languages other than English, or articles where only the abstract is in English.
Temporal Scope	Publications from 2022 to 2026.	Publications outside of this defined timeframe.
Thematic Focus	Studies directly addressing the use of DTs, BIM, IoT, or AI Agents specifically applied to emergency response, evacuation, or disaster management.	Studies discussing digital technologies in general building management or urban planning without a specific emergency or crisis application.
Technical Depth	Papers proposing novel frameworks, systemic architectures, algorithms, or presenting empirical case studies/simulations.	Conceptual papers lacking methodological rigor, high-level theoretical reviews without implementation, or purely policy-based discussions.

edges to nodes considered dangerous due to fire proximity or blockages [45]. EvacuSafe assesses the danger level of a route by leveraging a Route Risk Index (RRI), which dynamically tracks the proximity of individual occupants to hazards and considers other parameters along the egress path [45]. However, a significant limitation of early dynamic models is their reliance on manually inputted environmental parameters (e.g., temperature, CO_2, crowd density) prior to simulation, or their reliance on Computational Fluid Dynamics (CFD) models to simulate crowds, fires or floods; this prevents true real-time. Recent preliminary studies have integrated BIM with Computer Vision to automatically infer the real-time impact of temporary obstacles and hazards, enabling systems to take into account dynamic contexts and varied evacuee profiles [46].

Human panic and behavior during emergencies are highly unpredictable and complicate effective emergency management. To study and manage those behaviors, MAS coupled with Machine Learning (ML) have been adopted to simulate evacuees. These technologies allow researchers to identify vulnerabilities in static escape plans and develop dynamic, adaptable strategies. Traditional MAS architectures commonly rely on deterministic physical models, such as the Social Force Model, that excel in representing basic congestion, but they struggle to simulate human irrationality and stress-induced behaviors. To address this limitation, recent studies have increasingly adopted ML-driven agents. Modern MAS frameworks are integrated with BIM, Geographic Information System and Reinforcement Learning in order to give agents decision-making capabilities to navigate 3D environments, allowing agents to learn optimal escape strategies through repeated hazard simulations [47]. While ML introduces new capabilities, traditional agent-based models continue to yield insights into evacuation dynamics. For instance, stampede analyses demonstrate that emotional contagion and congestion drastically increase casualty risks during simultaneous evacuations [48]. Similarly, hospital evacuation

assessments highlights the necessity of safeguarding stairwell availability and centralizing emergency elevators to ensure the vertical mobility of vulnerable individuals [49]. To align digital simulations with the physical world, pipelines now extract demographic and emotional parameters directly from real-world images to populate 3D DTs [50]. By assigning agents specific "fear levels", these data-driven spatial models generate more realistic panic states and evacuation dynamics by dynamically degrading an agent's navigation accuracy.

For a more realistic representation of agents interactions, communication and decision making during crises, researchers started to use LLMs as the "brains" of individual agents. This upgrade allows them to perceive the environment, maintain decision history and exchange information among them [51]. These LLM-based agents allow agents to have a better comprehension of the environment context and increase the realism and reliability of emergency evacuations [51]. A possible upgrade can be to integrate Retrieval-Augmented Generation (RAG) with multimodal LLMs in order to have a mechanism to align asynchronous, heterogeneous data streams. The CrisiSense-RAG framework demonstrate that aligning multiple asynchronous data stream—such as social media posts as aerial imagery—in a zero-shot setting to provide a spatial estimate of disaster impacts without specific fine-tuning [52]. Its split-pipeline architecture mitigates asynchronous mismatches, preventing conflicting post-event clear sky imagery from negating important real-time flooding reports [52]. By routing text and images through temporally aware fusion pipelines, these systems reduce hallucinations and prevent post-event data from skewing real-time reports. However, the high token-generation latency of the multi-step LLM reasoning and the hardware requirements remain a bottleneck for real-time emergency routing, necessitating the use of quantized models deployed on edge devices. When LLMs are integrated with multi-agent orchestration mechanisms, they can synthesize fragmented reports into interactive geospatial intelligence [53]. Architectures like Disaster Copilot utilize centralized multi-modality LLMs to delegate tasks to specialized domain agents, facilitating the creation of interoperable DTs for disaster mitigation [54]. Despite the hardware constraints, hierarchical agent architectures allows the mitigation of computational bottlenecks by delegating micro-tasks—such as route simulation or hazard detection—to smaller, specialized models that could be distributed in multiple devices [54]. These agentic workflows allow the system to execute complex task, such as seismic resilience assessments, through long-horizon reasoning and automated task execution [55]. However, scaling these agentic systems for real-world deployment is a challenge due to data scarcity, computational costs and the risk of hallucination cascades.

Disaster management and spatial computing can evolve as DTs and IoT become more integrated. Traditionally, DTs have functioned primarily as virtual replicas utilized for visualization and data integration, that use simulation and optimization tools where experts are needed to fully exploit them [56]. However, the necessity to manage the spatiotemporal complexity of dynamic urban and indoor environments and the limited capability of context-aware analytics and problem solving led to the development of "Agentic DTs". By integrating Agentic AI and the Model Context Protocol (MCP), these frameworks provide DTs with cognitive reasoning, enabling them to autonomously discover relevant sensor data, construct analytical workflows and orchestrate domain-specific simulation tools [56]. Xu et al. [56] propose a conceptual model composed by three modules, each one having their own agents: a cognitive module, data and state module, task execution module. The cognitive module goal is to define analytical procedures, search for relevant data using a scientific and professional knowledge base

based on a stakeholder request. The data and state module is a broker between the physical city and the digital twin, it provides to the cognitive module structural and validated data. The last component is the task execution module that execute the workflows previously defined by the cognitive module by invoking simulation models, optimization solvers and control procedures.

Concurrently, integrating GenAI (including LLMs, Vision-Language Models, and Large Action Model) with Artificial Intelligence of Things has created the Generative IoT (GIoT) paradigm [57]. GIoT elevates traditional IoT architectures by deploying autonomous agents capable of perceiving multimodal environmental inputs, reasoning about complex situational contexts, share context and executing distributed or coordinated actions [57]. This paradigm embeds cognitive intelligence at both the edge (IoT devices), infrastructure levels (e.g. cloud computing) and in Services (to extract insights), reducing decision-to-action latency while enabling context-aware automation and collaborative intelligence across interconnected devices [57]. To ground outputs GIoT can be enhanced by using RAG pipelines, using IoT devices as queries or triggers to query external knowledge sources. GIoT also implement reasoning and cognitive capabilities by leveraging LLMs, allowing enhanced data analysis for Business Intelligence and online learning and adaptation, allowing the IoT systems to dynamically adapt to diverse situations based on new info [57]. Multiple architectures are proposed, Edge-centric, Cloud-centric and Hybrid, each one having their own advantages and limitations, considering factors such as performance, accuracy, latency, privacy, security and cost [57].

Integrating DTs with GIoT directly connects digital emergency simulations to automated physical responses. While multi-agent models can calculate optimal evacuation routes virtually, GIoT facilitates the cyber-physical deployment of these strategies by translating analytical outputs into executable control signals for physical actuators [56, 57]. Through this continuous bidirectional synchronization, an active DT can continuously assimilate live telemetry to dynamically update hazard states, while GIoT orchestration ensures that context-specific survival strategies are immediately realized in the physical environment.

3.2.1 Observations

From the findings, the dynamics of hazard response, emergency management and crowd simulations were considered to be of great importance to improve emergency systems and better understand human behavior during unusual and high-risk situations. Different solutions and approaches have been discovered and analyzed to derive the architectures and identify the major gaps.

A clear paradigm shift from static and pre-planned evacuation routes to adaptive and real-time dynamic guidance systems. Traditional Static Evacuation Signage (SES) and pre-determined routing plans are going to be considered inadequate because of multiple vulnerabilities. Those plans do not consider the events that are happening in the building, ignoring the evolution of hazards over time such as the spreading of smoke and fires [58]. To overcome these vulnerabilities those systems have been developed, models able to continuously monitor building status, detect the exact location of the hazards and automatically generate new evacuation routes to guide occupants to safer places minimizing harm to people as much as possible, considering the conditions change over time [45].

Different methods used to derive evacuation routes, the easier way to find those paths

is using traditional shortest path algorithms such as Dijkstra or A*. The building is represented as a geometric network graph and the edges cost is calculated considering various variables such as the length and the proximity from a danger [41]. Advanced implementations rely on Multi-Agent Reinforcement Learning (MARL), moving from a mathematical pathfinding toward data-driven routing. Zhang et al. [58] uses this strategy to coordinate and balance crowds dynamically by treating dynamic signs as autonomous agents, proving its effectiveness in complex environments such as multi-level metro stations.

In order to give occupants context-aware guidance, to reduce the cognitive load in chaotic environments, these systems rely on Active Dynamic Signage Systems or Dynamic Evacuation Signage [45, 58]. These signs can show evacuees the direction to follow or can display a negated signal to block occupants to pass on dangerous areas. By changing those directions, assuming a full human compliance, those systems showed that those signage systems can prevent crowd congestion and mitigate situations where occupants are trapped waiting at blocked exits [58].

Evacuation paths are not only effective for their short length, but multiple variables are considered to choose one path over another. Mirahadi and McCabe [45] uses RRI to derive a cost of a path based on the duration of an evacuation route and its proximity to hazards.

In order to test those systems, some simulators are used. BIM data is used to build geometric network graphs and is also fed into CFD tools such as Fire Dynamics Simulator (FDS) to simulate the spread of smoke, temperature, and radiation [45, 58]. The data from these fire simulators is then coupled with Agent-Based Modeling to simulate emergent behaviors of panic, competitiveness, queuing, herding, and the kin effect [45].

3.2.2 Results and Gap Analysis

Traditional static evacuation systems have some limitations. SES is widely used but vulnerable because it operates on fixed logic, without considering the spatiotemporal events happening in the building such as smoke propagation and crowd congestion [58]. Also relying on human staff when there are big amount of panicked people and that human decision making under strict time is prone to errors [45]. In the literature proposed systems there are multiple unresolved issues. For example EvacuSafe [45] does not consider the crowd congestion during the path planning, calculating paths assuming that building have unlimited flow rates, ignoring real-world bottlenecks. It cannot quantify the complexity cost of the proposed evacuation strategies, instead of merely rely on mathematical optimal routes, the system could also consider the complexity of those path, might result in a reduced likelihood of panicked evacuees that are more likely to follow the proposed route. In the Zhang et al. [58] work there are different issues. The MARL framework assumes that all occupants will follow the dynamic evacuation signs, but human behavior during a life-threatening situation is highly stochastic, frequently involving herding effects, bounded rationality, and a mistrust of automated systems that lead to non-compliance. To ensure that the model can run efficiently in large scale, the crowd simulator simplify microscopic physical interaction such as pushing at staircases, falling or cohesion of family groups. It assumes that all environmental sensors and displays will work all the time. It does not account for conditions where power outages, extreme heat or dense smoke could destroy or obscure the hardware. A common unresolved issue is the scalability of shortest path algorithms. Since that a

building graph complexity can increase, relying on standard algorithms such as Dijkstra can be a limitation. Systems should evaluate more complex, fast and scalable algorithms to apply on time-critical situations. Systems should also integrate more features for each node end edge to achieve an optimal usage.

To synthesise the literature and highlight the identified gaps, table 3.3 presents a comparative analysis of the primary state of the art approaches discussed in this section. The comparison evaluates the models against essential architectural and operational requirements. The table illustrates the recent advancement in evacuation modeling and critical unsolved issues that remain.

Table 3.3: Comparison of state of the art approaches (✓): Fully match. (✗): Doesn't match.

Feature / Requirement	**Kelepiri, Lentzas, and Vrakas [41]**	**Mirahadi and McCabe [45]**	**Zhang et al. [58]**
Dynamic Routing	✓	✓	✓
Dynamic Route Risk Index	✗	✓	✓[2]
Occupant Guidance	✗	✓	✓
Machine Learning Adoption	✗	✗	✓
Multi-Hazard Compatibility	✗	✗	✗
Management Strategy Flexibility	✗	✗	✗
Real-World IoT Data Injection	✗	✗	✗

As illustrated in table 3.3 while the transition toward dynamic routing and active occupant guidance is well-established in recent research, important vulnerabilities persist across all models. The literature reveals various prototyping approaches for hazard identification: some rely on environmental sensors for fire and crowd detection [58], others employ modules triggered by standard fire alarm systems [45], and some utilize user-defined thresholds on IoT sensors [41].

While these threshold-based methods are generally adequate for identifying single-hazard types and safely rely on tested alarm infrastructures, they present a significant limitation. Rigid system logics and statically defined thresholds struggle to adapt to edge cases, especially when limits are not configured by domain experts, nor can they be easily generalized to handle alternate, emerging hazard types such as gas leaks or structural instability. This rigidity explains why none of the analyzed frameworks currently support true multi-hazard compatibility or management strategy flexibility. To satisfy this need for management strategy flexibility, the proposed architecture introduces a DT that utilizes BPMN to define and orchestrate strategy processes. By leveraging BPMN, the system can explicitly define the diverse behaviors and responsibilities of different actors during a hazard. This BPMN-driven workflow is designed to interact continuously with another architectural component that represents the live physical environment, ensuring that strategic execution remains tightly coupled with real-time contextual changes.

[2]While Zhang et al. [58] do not define a traditional RRI, they employ a similar risk metric evaluated at the area level rather than on the edges.

A substantial gap exists in the transition from simulated environments to physical deployment. The analyzed systems have largely been restricted to Proof of Concept environments, relying on manually inputted data or computational CFD simulators like FDS. Because they have not been deployed in real-world settings, these approaches universally lack the robust capacity to inject and process live, real-world IoT telemetry to drive their operational logic.

To address these specific shortcomings, the proposed architecture must move away from rigid, single hazard simulators and use a more adaptable, data-driven paradigm. By leveraging the cognitive reasoning of Agentic AI combined with the cyber-physical integration of GIoT and BPMN-orchestrated DTs, the resulting system aims to satisfy these unmet requirements, providing a scalable, multi-hazard, and fully dynamic emergency management solution.

4. Architecture Design

Before going into the prototype, taking a step back and look at the problem from a higher level is helpful. Section 3.2 pointed out a few gaps in current approaches to indoor emergency management. Most of them rely on a fixed plan, or only work inside a simulation, and none of them combine live sensor data with a reasoning step that decides what is happening and what to do about it. This chapter describes the architecture designed to close that gap, explaining what each part of the system is responsible for and how they work together, before the chapter 5 shows how each part was actually built.

4.1 Design Goals

The architecture has to satisfy a few requirements, coming directly from the objectives introduced in chapter 1.

- Speed is one requirement, since an emergency changes fast. The time between a hazard appearing and the system reacting has to be short. This rules out any design where data passes through several slow or blocking steps before something happens.

- Another requirement is handling data that comes from many different sources, in many different shapes. A temperature sensor, a motion sensor and a weather feed do not send the same kind of data, at the same rate, or in the same format. The architecture needs a place where this data is collected and turned into something the rest of the system can use. This way, no other component has to know how to read every possible source.

- The third requirement is keeping the interpretation of the data separate from the decision of what to do about it. Seeing that a hazard is developing and deciding how people and systems should respond are two different problems. Keeping them separate means each one can change on its own. A new hazard detection method does not require touching the response logic, and a new response strategy does not require a new detection method.

- The last requirement is extensibility. The system should support new hazard types and new response strategies without rewriting its core. This should be possible just by adding new interpretation logic or new process definitions. A building manager should be able to add a new sensor, and a safety team should be able to define a new response for a new hazard type, without a developer having to change the rest of the system.

4.2 Conceptual Architecture

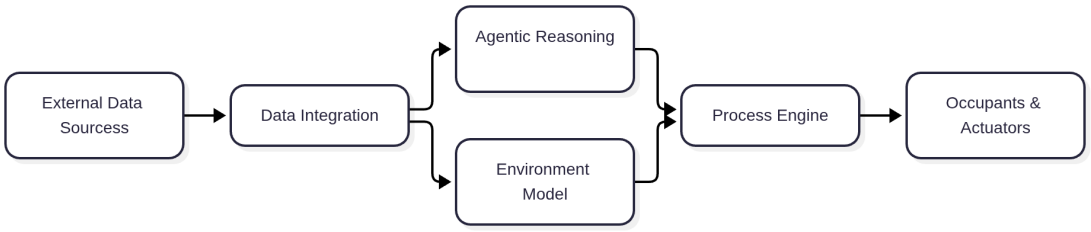


Figure 4.1: High level view of the proposed architecture.

Figure 4.1 shows the architecture at a conceptual level. It is made of four components, connected by a flow of data, plus the physical world at both ends of that flow.

At the edge of the system are the external data sources. These are anything that can tell the system something about the building or its occupants: IoT sensors, occupant reports, or external services such as a weather feed. They are treated as unstructured, since the system has no control over how or when they send data, or how reliable that data is.

These sources feed into the data integration. Its only job is to take data in whatever shape it arrives and turn it into common formats that the rest of the system can rely on. It also works as a bridge between the outside world and the rest of the architecture, so adding a new sensor type only means adding a new adapter here, not changing every other component. From this component, information moves in two directions.

One direction goes to the agentic reasoning component, where structured data is interpreted into something meaningful, for example recognizing that a temperature spike with smoke readings means a fire is starting in a specific area. This component is kept separate from the rest of the system on purpose, since its only job is to turn data into a hazard classification, not to decide what happens next. It can be extended with new interpretation logic for a new hazard type without any other component needing to change.

The other direction goes to the environment model, that keeps an updated representation of the building, both as raw physical data, such as rooms and how they connect, and higher level concepts built on top of it, such as a safe zone or a blocked exit. Both the hazard classification coming from the reasoning component and the raw sensor data are used to keep this model current. Without this component, every part of the system that needs to know about the building would have to keep its own copy of that information, and those copies would not be consistent over time.

The hazard classification and the environment model are then used by the process engine. This component coordinates the response: it defines, separately for each type of actor involved, such as occupants, emergency wardens or building systems, what should happen once a hazard is confirmed, and it checks the environment model to ground its decisions on the current state of the building rather than on a pre-computed plan. What comes out of this component is what eventually reaches the physical world, as instructions to occupants or as commands to actuators such as lights or doors.

This separation is what makes the architecture easy to change. The data sources can be extended without touching the reasoning logic, the reasoning logic can be extended to new hazard types without touching the process definitions and the process definitions can be updated to reflect a different response strategy without touching how hazards

are detected in the first place.

An Example Walkthrough

Consider a smoke sensor in a corridor that starts reporting values above its normal range. The reading reaches the data integration layer, which turns it into a structured message: a sensor id, a value, a timestamp and the area it belongs to. That message is sent to the agentic reasoning component, which looks at it together with the temperature readings from the same area and classifies the situation as a fire starting in that corridor.

At the same time, the reading is also used to update the environment model, so the corridor now carries this new information alongside its usual data, such as its size and its connections to nearby rooms. Once the fire classification arrives, the environment model marks the corridor as unsafe.

The process engine picks up this change. For every occupant it is tracking, it asks the environment model for a path to the nearest safe exit that avoids the unsafe areas. If an occupant's usual route would go through the corridor, a new route is computed instead, and the directional lighting along that new route is turned on to guide them. The emergency warden assigned to that part of the building is notified separately, following a different process definition meant for wardens rather than occupants.

If the fire spreads and a second corridor or room becomes unsafe, the same steps repeat: the environment model is updated, and the process engine recomputes routes for anyone affected. No part of this requires a person to manually monitor data and redraw an evacuation map, which is the static approach this thesis is trying to move away from.

Why This Architecture

A simpler alternative would be to write a program that reads the sensors directly and reacts to fixed rules, such as turning on an alarm once a temperature sensor crosses a threshold. This would work for a single hazard type and a small building, but it does not meet the goals from section 4.1.

Adding a second hazard type, such as a gas leak, would mean going back into that same program and adding another set of rules next to the existing ones. Over time the rules for different hazards would end up mixed together, and changing how the system reacts to a fire could risk breaking how it reacts to a gas leak. Splitting the reasoning from the response keeps these responsibilities apart: a new hazard type means adding a new interpretation step, not editing existing ones.

A second alternative would be to compute evacuation routes once and simply follow them during an emergency. This is what most of the systems reviewed in section 3.2 do, a fixed route cannot account for a hazard that spreads or moves after the plan was made. Keeping an environment model that is updated in real time, and having the process engine query it every time instead of following a plan computed once, is what allows the routes to stay valid as the situation changes.

A third option would be to have the whole system work with a polling mechanism, almost like a batch process, checking every few seconds whether new data is available instead of reacting as soon as it arrives. This still satisfies the separation-of-concerns goal, but it goes against the speed requirement from section 4.1. The delay between a hazard appearing and the system noticing it would depend on the polling interval, not on how fast the components can actually process the data. An event-driven design

instead lets each component react as soon as the data it depends on changes, so the reaction time stays close to the processing time itself.

The cost of this design is added complexity: instead of one program, there are four components that need to agree on a common data format and communicate with each other. This cost is accepted because it is what makes the system able to handle more than one hazard type, and able to keep working as the building's conditions change during an emergency.

5. Prototype

This chapter presents the prototype built for this thesis[1]. The goal of the prototype is to show that BPMN, IoT and multi-agent systems can work together in an emergency response framework for indoor environments.

Chapter 4 described the architecture behind this prototype at a conceptual level, without naming any specific tool. Table 5.1 shows how each of those conceptual components was implemented, and with which technology. This chapter first covers the system architecture, then the technology stack motivating the choice of each tool, and finally moves into implementation details.

Table 5.1: From the conceptual architecture (chapter 4) to the modules and technologies implementing it.

Conceptual Component	**Module**	**Technology**
Data Integration	Data Integration Middleware	Node-RED
Agentic Reasoning	Emergency Agent	LangGraph
Environment Model	Environment Engine	Memgraph
Process Engine	Environment-aware BPMN Engine	Camunda

Two more tools are connected to the process engine but do not map to a separate conceptual component on their own. The Environment-aware BPMN Modeler is used to design the process models and deploy them into the components above. The Environment-aware BPMN Cockpit gives a dashboard to monitor the state of those processes while they run. Node-RED is also used in two other parts of the system, besides the Data Integration Middleware. It reads the JSON file that defines the place graph and loads it into the Environment Engine and also acts as the mediator between the BPMN Engine and the Environment Engine. Camunda runs the process, and Node-RED executes the tasks. Node-RED workers reads the task variables, queries or updates the place graph, and then reports the task as completed to Camunda.

The Data Integration Middleware is responsible for collecting and processing IoT telemetry, and the multi-agent system used to detect hazards is included. The Environment Engine maintains the digital representation of the building, both as raw

[1]The complete source code of the framework, including the Node-RED flows, the LangGraph multi-agent architecture, and the Camunda BPMN model, is available at: https://github.com/nicol-buratti/multi-hazard-emergency-framework

spatial data and as higher level concepts that can be used by the process logic. The Environment-aware BPMN Engine uses this representation to coordinate the actors involved in an emergency, such as occupants and emergency wardens, and to trigger physical actuators in the building. These components compose a working prototype that can detect a hazard, reason about it, and turn that reasoning into concrete actions inside the building.

5.1 Architecture

Due to the highly dynamic nature of indoor emergencies, the system architecture's priority is to reduce the latency between the identification of the hazard and the actual response, handle large amounts of data from different sources and scale components to meet the latency requirement. The system adopts a hybrid, component-based Event-Driven Architecture. At a higher level, the system is divided into highly decoupled components to ensure independent scalability and separation of concerns. At a behavioral level, the communication and execution are event-driven, allowing the system to react with low latency to dynamic external stimuli, such as sensor data or occupants inputs, during indoor emergencies.

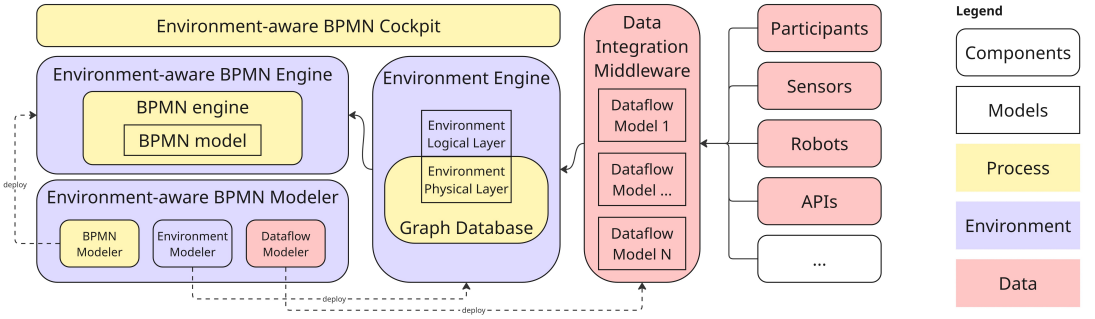


Figure 5.1: System architecture

A high-level diagram of the architecture can be seen in figure 5.1. From right to left, we have an example of the multiple external data sources feeding into the system. These external sources are very heterogeneous, ranging from data given by the occupants or sensors to external APIs such as weather data.

These external data are injected into the Data Integration Middleware, where multiple data flows are defined in order to handle and format the data to be injected into the Environment Engine. For instance, when a temperature sensor pushes a value, the data is instantly processed by the Middleware, mapped into the Graph Database and then the temperature change can trigger a specific task in the BPMN engine.

The Environment Engine is composed of two layers: the Physical Layer and the Logical Layer. The Physical layer is implemented by using a Graph Database. This layer represents the building as a graph, where each node represents an area such as rooms, corridors, or emergency exits. Each node contains data; this data is divided into static data and dynamic data. The static data comprises attributes that represent the physical space, such as its size, coordinates or BIM data. Dynamic data, on the other hand, consists of information that comes from the external data brought by the Data Integration Middleware. The Logical Layer instead is a component that performs

computations on the graph, aggregating physical nodes into semantic concepts (logical places) that can be interpreted by the BPMN engine, requesting places such as a 'safe zone' or a 'gathering point'.

Then there is the Environment-aware BPMN engine. It is a component that executes a modified BPMN model able to interact, via a mediator, with the Environment Engine taking advantage of the Environment Logical Layer. This component is used to activate the actuators, such as lights, to direct the occupants toward the safest exit.

The last components are the Environment-aware BPMN Modeler and the Environment-aware BPMN Cockpit. As illustrated by the dashed arrows in figure 5.1, the Modeler is responsible for designing and deploying the generated models directly into their respective runtime environments (Middleware, Environment Engine, and BPMN Engine). The latter provides users a dashboard to monitor the current state of the processes.

In the next sections, those components will be deeply analyzed, defining their scope, tech stack and actual implementation.

5.2 Technology Stack

After a careful analysis, multiple tools were selected to achieve the project's objectives. To implement the agentic interpretation workflow, the LangGraph framework was utilized. Instead of relying on predefined hierarchical structures or swarm sublibraries, the system leverages LangGraph's native state graph and core multi-agent orchestration capabilities to define autonomous, specialized agents capable of interpreting IoT data effectively. Additionally, Node-RED was employed to define multiple data flows from various sources to the system's components. With its low-code paradigm, even individuals without expert knowledge can comprehend the underlying data transformations and their applications. Finally, Camunda, a BPMN engine, was integrated to allow the system to define multiple hazard management strategies and simulate occupant behavior during emergency situations.

5.2.1 Evaluation of AI Agent Frameworks

Indoor emergency evaluation requires a system capable of processing IoT telemetry in real time to assess both the type and the severity of a hazard. Because different emergencies (e.g., fires, gas leaks) present distinct data patterns, the architecture requires multi-agent support to deploy multiple AI agents, each specialized in evaluating a specific hazard type. Several technologies for implementing this multi-agent framework were evaluated:

- **CrewAI**: This framework is designed for static, sequential, or hierarchical task execution. Its architecture relies heavily on persona-driven configurations (roles, goals, and backstories), that exponentially increase token usage during execution loops. For an architecture demanding low-latency hazard detection, this increases inference time without adding analytical value.
- **AutoGen**: It is based on autonomous conversation among agents and is effective for problem-solving through group discussions. However, it has high latency and could have termination issues that can lead to infinite chat loops. For a system that must quickly classify the type and level of danger during critical incidents, this non-determinism is an unacceptable risk.

- **LlamaIndex Workflows & Haystack**: These frameworks utilize an asynchronous, event-driven architecture that is highly effective for data-heavy RAG and decoupled pipeline processing. However, event-driven systems can introduce non-deterministic execution paths. For an architecture tasked with critical, low-latency hazard classification, this lack of rigid, explicit state-loop control makes it less suitable than an explicit Finite State Machine (FSM) approach.
- **LangGraph**: LangGraph is an open-source orchestration framework that helps developers create stateful multi-agent systems. Unlike other frameworks, it does not use strict Directed Acyclic Graph (DAG) structures, allowing for the explicit creation of cycles or continuous loops. This ability to manage multiple specialized agents as an FSM, with a shared persistent state, integrates seamlessly with the continuous evaluation of dynamic IoT telemetry.

Table 5.2: Comparison of AI Agent Frameworks for IoT Hazard Detection

Comparison Criterion	**LangGraph**	**CrewAI**	**AutoGen**	**LlamaIndex**
Flow Control	Deterministic	Sequential / Hierarchical	Free Conversational	Event-Driven
State Persistence	High	Medium	Low	Medium
Inference Overhead & Latency	Low	High	High	Medium
Suitability for IoT Systems	Excellent	Limited	Limited	Moderate

Based on the project requirements and the comparative analysis summarized in table 5.2, the final selection was between CrewAI and LangGraph. LangGraph was selected due to its native support for dynamic conditional routing via state graphs. CrewAI enforces static, sequential task execution; implementing conditional early termination or selective routing requires custom overrides or hierarchical manager agents, which introduces latency. Because the system requires distinct agents for specific hazards, and inference time must be minimized during evaluations, LangGraph's deterministic control flow is required to route IoT data strictly to the relevant hazard agent and eliminate redundant model calls.

LangGraph is an open-source, low-level orchestration framework that assists developers in building advanced, stateful and long-running agents supported by LLMs [59]. It implements advanced agent architectures that learn from interactions, continuously evolve and autonomously retrieve context from external sources [60]. A key characteristic of LangGraph that differentiates it from other frameworks is that it does not use DAG structures. An agent is composed of nodes connected by directed edges; LangGraph enables the creation of cycles or loops within the structure. This property removes the limitations of DAGs, allowing the creation of truly dynamic and adaptive AI agents [60]. These loops allow developers to create iterative processes, feedback loops and recursive

behaviors.



(a) LangChain logo (b) LangGraph logo

Figure 5.2: LangChain and LangGraph logos

LangGraph is tightly integrated with LangChain; this allows the use of an extensive suite of tools for LLMs from LangChain with the LangGraph stateful framework. Developers who have familiarity with LangChain can rapidly integrate LangGraph to create and deploy complex agents faster [60]. The LangChain utilities combined with LangGraph's cyclic workflows allow the implementation of feedback loops and iterative behaviors, also providing tools for testing and debugging AI agent behaviors. This integration also provides tools to easily connect agents with external systems and data sources [60]. LangGraph agents use centralized state management. The agents operate on these shared state objects, typically defined using strongly typed schemas, and as the graph executes, this state is passed to the working nodes. The nodes can return state updates instead of overwriting it; those updates are then merged into a global state. The state mechanism allows the framework to maintain a structured context across complex iterations. The combination of the frameworks is also effective for implementing multi-agent systems. LangChain provides built-in agent templates, and LangGraph can orchestrate them. In this thesis, the core multi-agent orchestration capabilities of LangGraph are utilized. Instead of relying on predefined hierarchical structures or swarm sublibraries, multi-agent systems can be implemented directly using LangGraph's native state graph. The architecture is composed of a network of highly specialized agents, represented as individual nodes within the graph. The core mechanism for transferring control between these agents is conditional routing based on the shared global state. Each agent is defined with a specific system prompt, toolset, and structured output. When a task requires a different domain of expertise, the graph's conditional edges evaluate the updated state and dynamically route the execution to the agent best suited to handle the task. This graph-based approach ensures that the system remains highly modular and scalable, allowing developers to introduce new specialized agents by simply defining new nodes and routing conditions without disrupting the existing network.

5.2.2 Node-RED

Node-RED is "a programming tool for wiring together hardware devices, APIs and online services", which provides a way of "low-code programming for event-driven applications" [61].

Originally created by the IBM Emerging Technology Services group in 2013, it was open-sourced in the same year and later, in 2016, became one of the founding projects of the OpenJS Foundation. The primary objective of the tool is to simplify the development of data flows, wiring them together using customized nodes, and deploying those flows seamlessly.

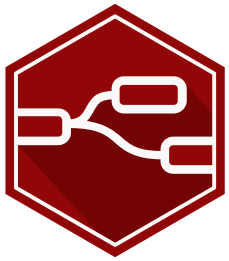


Figure 5.3: Node-RED logo

Node-RED introduces a paradigm for developing data flows known as Flow-Based Programming. Applications are modeled by defining black-box processes (nodes) that exchange data through connections (flows) between them. This approach abstracts the high-level logic from the underlying implementation of individual nodes, making the system highly comprehensible while enhancing modularity and reusability. Typically, the exchanged messages are formatted in JSON.

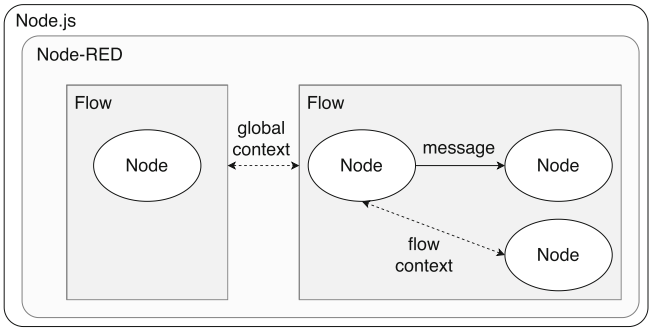


Figure 5.4: Node-RED architecture [61]

As illustrated in figure 5.4, the Node-RED architecture is composed of multiple applications called *flows* that link components known as *nodes*. It allows for the simultaneous execution of multiple flows and supports inter-node and inter-flow communication via messages passed through the wiring. Nodes that are in different flows can also communicate via the *global* context [61]. Each node acts as a Node.js application that is triggered upon receiving a message on an input port, subsequently sending the computed results to its output ports.

A flow is composed of multiple interconnected nodes; developers can construct flows utilizing both built-in and third-party nodes. These flows are represented as JSON files, mapping node outputs to inputs within a graph structure. Messages are passed these connections, and a single message can be broadcast to multiple nodes. Node-RED provides a web-based graphical interface to model and deploy these flows efficiently [61].

Direct connections between nodes are not the only method of information exchange. *Contexts* allow multiple nodes or flows to share state and pass information without being explicitly wired together. There are three context scope levels: *node-level* (visible only to the node that sets the value), *flow-level* (visible to all nodes within the same flow), and *global-level* (visible to all nodes across any flow).

The underlying runtime is built on Node.js. This technology was selected for its event-driven architecture and non-blocking I/O model, making it lightweight and ideal for execution on resource-constrained edge devices such as Raspberry Pi. Despite these

advantages, one limitation of the visual programming approach is that flows can become difficult to manage and debug as the application's complexity and the number of logical loops increase.

Node-RED has been widely utilized in various IoT analytics tasks, ranging from monitoring indoor radon gas and photovoltaic system performance to orchestrating complex workflows in remote oil facilities and water treatment Supervisory Control and Data Acquisition systems [62].

In this thesis project, Node-RED is employed as an integration middleware between IoT devices, alongside multiple data sources, and the other components of the architecture. IoT data is captured, transformed, and manipulated before being injected into downstream components, such as workers and databases.

5.2.3 Evaluation of Graph Databases

The selection of the graph database is necessary to fulfill the operational requirements of the system. The architecture needs a graph storage component capable of handling high-frequency write operations to update graph attributes with real-time telemetry from IoT sensors. The structural topology of the graph remains mostly static, minimizing the need for rapid topological updates such as inserting or deleting nodes. The system also requires high-speed query capabilities to continuously identify logical places and execute complex graph algorithms, such as breadth-first search and weighted shortest paths for evacuation routing. To handle intensive workloads, a potential decoupling of the storage layer from the computation layer is considered. Several graph databases were evaluated based on these criteria.

- **JanusGraph** offers a pluggable and decoupled architecture. Its computation layer is logically and physically independent from the storage layer. While this provides flexibility, the physical separation can introduce network latency bottlenecks during the continuous execution of real-time OLTP (Online Transactional Processing) operations.

- **NebulaGraph** implements a distributed architecture that divides query execution and graph storage into two separate services. This separation permits the query layer and storage layer to scale independently, allowing horizontal scaling. But its heavily distributed nature might overcomplicate the deployment for a system managing a relatively limited number of nodes, where low latency is prioritized over massive horizontal scalability.

- **Memgraph** is an open-source, in-memory graph database engineered for real-time performance, high-throughput data ingestion, and low-latency analytics. By leveraging an in-memory storage engine, it guarantees minimal query response times, which is necessary for event-driven systems that need to process continuous, dynamic data streams. It features full ACID-compliant transactions and natively implements the standardized openCypher query language. Various graph algorithms are shipped directly with the database, natively supporting breadth-first search and weighted shortest path executions without requiring external computation layers. Write scalability is achieved vertically, which suits the system's need to continuously update sensor attributes without encountering distributed sharding overheads.

Table 5.3: Comparison of evaluated Graph Databases

Criterion	**JanusGraph**	**NebulaGraph**	**Memgraph**
Architecture	Decoupled	Decoupled	In-memory / Integrated
Query Language	Gremlin	nGQL / openCypher	openCypher
Algorithm Execution	Engine-dependent	Separated Query Service	Built-in / Native
Read/Write Latency	Network-dependent	Network-dependent	Ultra-low (RAM)

Based on the project requirements and the comparative analysis summarized in table 5.3, Memgraph was selected to implement the foundational physical layer of the system. Although decoupled architectures offer superior horizontal scaling and flexibility, the in-memory engine of Memgraph provides the absolute lowest latency for continuous algorithm execution and high-frequency attribute updates.

5.2.4 Memgraph

Founded in 2016 by Dominik Tomicevic and Marko Budiselic, Memgraph is an open-source, in-memory graph database. Instead of traditional relational tables, it models data through nodes and edges. Since it stores data primarily in-memory, it handles high-throughput ingestion and complex traversals with very low latency, a key requirement for event-driven systems where continuous data streams needs to be analyzed. The database engine is implemented in C and C++ to minimize execution overhead and ensure sub-millisecond query performance. Even if primarily operates in-memory, it supports ACID compliance and data durability through Write-Ahead Logging and periodic database snapshots, using Multi-Version Concurrency Control to process simultaneous read and write operations without putting structural locks. To handle growing workloads, Memgraph supports the horizontal scaling of read queries through database replication, intentionally avoiding data sharding for write queries to maintain high performance and avoid complex infrastructure management. Instead of partitioning the dataset, it leverages MemGQL —a federated graph query language— to provide horizontally scalable distributed compute; partitioning the execution of complex queries across a cluster of stateless workers and aggregates the results at a coordinator node. This query-centric distribution permits the system to add compute nodes for intensive graph workloads without the overhead of cross-node data chatter.



Figure 5.5: The Memgraph logo

From a technical point of view, Memgraph natively supports both the openCypher query language for querying complex topological patterns, and the Bolt network protocol, ensuring easy integration with existing middleware and drivers built for ecosystems like Neo4j. The architecture includes native integrations for event streaming platforms like Apache Kafka and Apache Pulsar, enabling the database to continuously ingest dynamic data streams and update structural topologies without service interruption. The open-source Memgraph Advanced Graph Extensions (MAGE) library extends Memgraph's analytical capabilities by offering over fifty built-in algorithms such as PageRank and Breadth-First Search, that execute directly within Cypher queries. MAGE also supports dynamic algorithms that incrementally update graph metrics as new data arrives, bypassing the need for full recalculations

For AI applications, Memgraph provides functions in a MCP server. This enables LLMs and autonomous agents to directly query the graph, effectively using the database as an active knowledge base during reasoning tasks. Recent releases incorporate native vector indexes to facilitate semantic similarity searches allowing the combination of vector search and structural graph traversals in a single query execution.

Within the context of this thesis, Memgraph forms the system's physical data layer. It acts as the central spatial repository for both the building's static topology and incoming dynamic IoT telemetry. Through its MCP integration, it feeds this structural and real-time context directly to specialized AI agents for hazard evaluation.

5.2.5 Camunda Process Engine

Camunda is an open-source platform for the automation of business processes and orchestration of workflows. In complex systems, tracking the state of a process that spans multiple systems is difficult. Camunda acts as a director, coordinating calls between services and managing timeouts and compensations.

CAMUNDA

Figure 5.6: Camunda Logo

Camunda BPM is an open-source, lightweight and composite platform for Business Process Automation and orchestration of workflows, defining an approach consisting in designing, executing and monitoring end-to-end processes. It allows to facilitate complex automation with multi-stakeholder environments and bridging business needs and IT implementation [63].

Camunda's architecture is based on the international standards BPMN 2.0 and Decision Model and Notation, languages that ensure independence from individual vendors. It uses a Model-Driven Execution, the BPMN diagram is not only a graphical representation of processes, but an XML file that the Engine interprets and executes as code [63].

Camunda introduces a methodological framework called "Camunda House", that separates the Strategic Process Model (abstract and results-oriented) and the Operational Process Model, that includes both human process flow and technical flow [63]. It is composed by multiple components, main ones are:

- **Modeler**: The modeling tool to design BPMN models
- **Engine**: The execution engine that can execute these models
- **Tasklist**: A web application to process user tasks
- **Cockpit**: A web application to monitor and administer running processes
- **Cycle**: A web application used to synchronize the development environment with various business-oriented BPMN modeling tools
- **Admin**: A web application to administer users, groups and authorizations

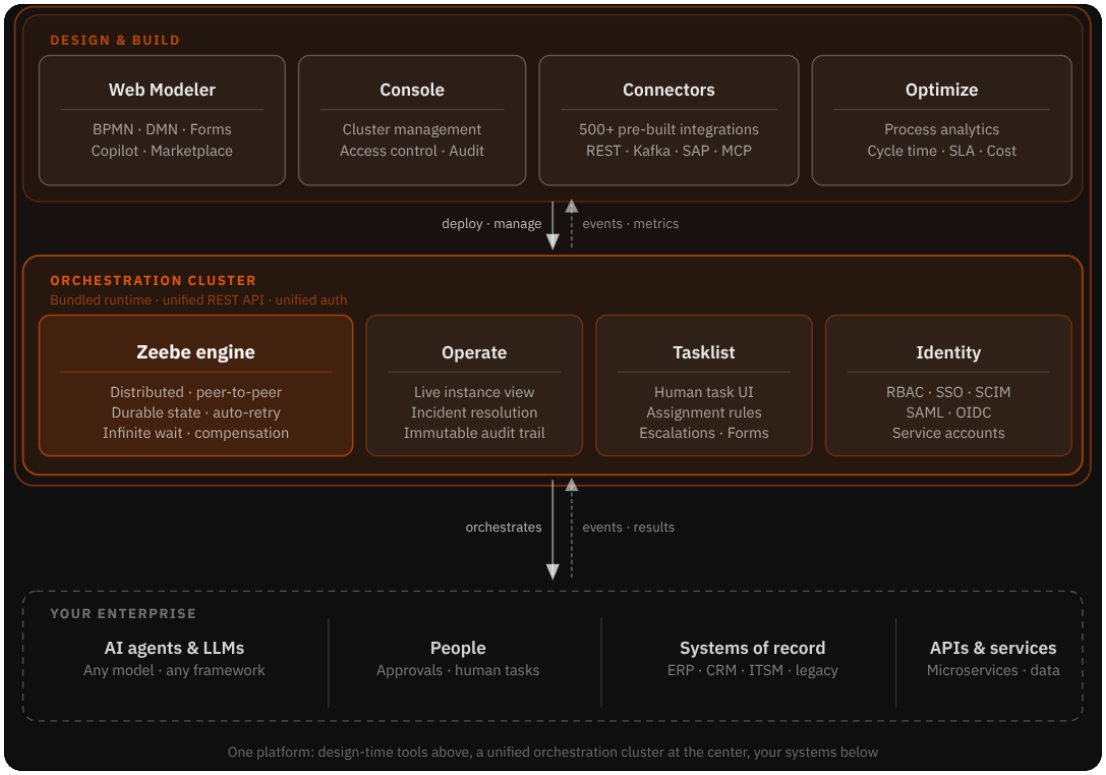


Figure 5.7: Camunda architecture [64]

In figure 5.7 more components are defined. The Engine uses the concept of tokens to track the progress of instances. Tokens flows through the model, indicating the system where each process is at any moment. This mechanism allows to handle timeouts, errors and failovers, ensuring operational continuity even in long-running processes. The use of the orchestration engine can be used to coordinate multiple services, having a centralized control logic instead of point-to-point choreography, facilitating monitoring through tools such as the Cockpit.

5.3 Data Integration Middleware

The Data Integration Middleware acts as the main integration layer between heterogeneous external data streams and the internal processing components of the framework. In dynamic indoor environments where conditions can get worse rapidly, the system

relies on this layer to continuously ingest IoT telemetry alongside other real-time inputs. Rather than simply passing raw data downstream, the middleware ensures that all incoming information is processed, transformed, and routed directly to the Environment Engine for spatial contextualization.

To orchestrate this continuous, high-volume flow of sensor metrics, the architecture deploys multiple specialized Node-RED flows. These pipelines handle data formatting, temporal aggregation, and stateful caching, ensuring that the downstream systems are neither overwhelmed by noise nor starved of relevant updates.

Furthermore, this integration layer also performs higher-level analysis of the incoming data. It includes a multi-agent architecture implemented using the LangGraph framework to interpret the incoming sensor telemetry. The agents analyze the refined data in real time to detect and classify active indoor hazards as they emerge. Upon predicting a threat, the middleware autonomously updates the spatial graph representation within the system, thereby initiating the cascading logic required to trigger context-aware emergency responses across the building's physical environment.

5.3.1 Node-RED Flows

In this section, the implemented flows that inject into the system external data are presented, describing their utility and how data is transformed along them.

Place Graph Initialization Flow

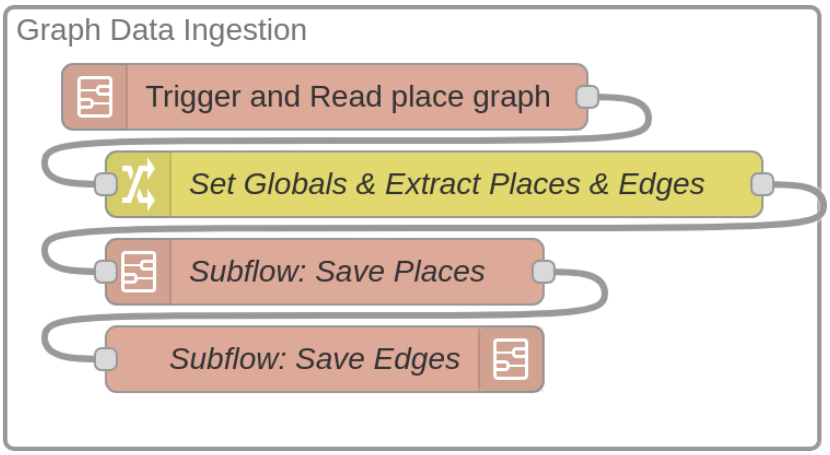


Figure 5.8: The place graph initialization flow

Figure 5.8 shows the flow that injects data into the graph database and set as global variables the logical layer of the Environment Engine. The "Trigger and Read place graph" subflow starts the flow. When the flow is deployed the first time, this subflow triggers the flow and reads the JSON file containing the place graph configuration. It also have a mechanism that automatically updates the stored place graph when its JSON file is modified, by triggering the flow. When the flow is triggered it performs two operations: it inserts the logical places and views as global variables, in order to be used by the other flows, and extracts nodes and edges of the graph. After the message have been formatted, it sequentially save the nodes and the edges on the graph database To communicate and send queries to the Memgraph database, the external library `node-red-contrib-neo4j-bolt` has been used. Since a dedicated Node-RED node for

Memgraph is not available, this connector was adopted because Memgraph natively implements the Bolt network protocol and supports the openCypher query language, as detailed in section 5.2.4. The same workflow has been applied to the Node-RED config file to insert and update the global variables.

IoT Caching Flow

The second flow is responsible for handling the IoT data injection into the agent and the Place Graph in the Environment Engine. Since the IoT data can arrive at a high frequency, a caching flow has been developed to avoid submerging these components with simultaneous requests. The flow is shown in figure 5.9.

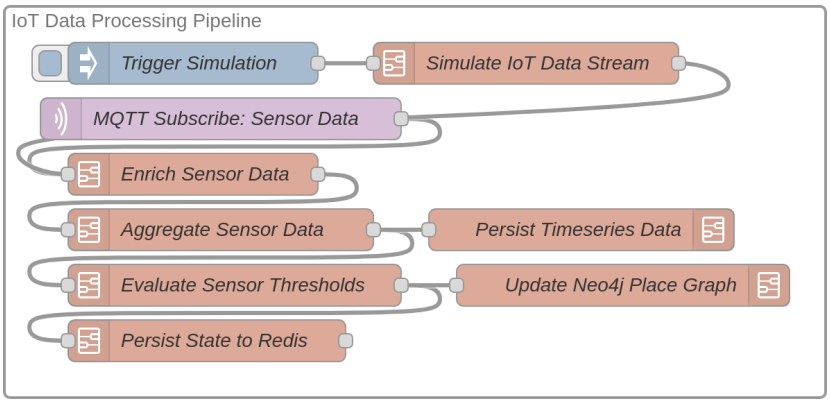


Figure 5.9: IoT caching flow

The ingestion of the raw IoT telemetry comes from the MQTT node, which is subscribed to the `sensors/+/data` topic. For testing purposes, the "Simulate IoT Data Stream" have been developed in order to simulate a data stream coming from a file that collects real data stream if a MQTT broker is not available. In the "Enrich Sensor Data" subflow, raw data is filtered by dynamically deleting parameters and mapping the `device_id` to the respective room and department based on the global settings defined on the initial flow. After the data enrichment, the flow is splitted by sensor type. If a device captures multiple parameters (e.g., temperature and humidity), those parameters are splitted by creating multiple flows, where each one contains a single parameter and a topic, where the naming have been defined as *department:room:parameter*, is assigned to be used later on the aggregation.

Regarding data aggregation, the architecture was revised to optimize both performance and usability. Initially, delegating high-frequency time-window aggregations to a Redis database was considered. However, this approach can become inefficient under heavy loads; since Redis operates on a single-threaded architecture, continuous aggregations risk blocking the event loop and degrading overall performance.

For this reason, the aggregation logic has been entirely moved into Node-RED. Internally the incoming telemetry is now dynamically routed based on the sensor type (e.g., temperature, humidity, accelerometer) to dedicated aggregator nodes. This specific architectural choice was made to fully exploit the low-code paradigm. It allows non-technical users to manage and customize the data pipeline without writing any code. By simply wiring nodes together, a user can effortlessly set up new aggregations or modify the time windows directly through the node's graphical interface. After the time-window elapses, the aggregated metrics are mapped and batched by room. This

aggregation flow can be seen on figure 5.10.

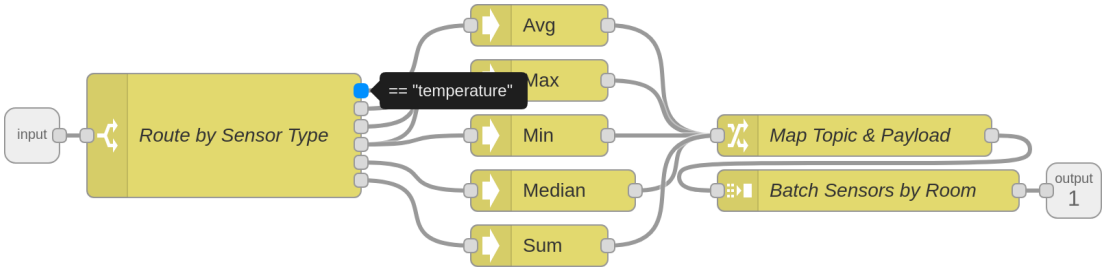


Figure 5.10: Visual IoT aggregation in Node-RED. Non-technical users can easily trace data flows by hovering over the output nodes to reveal filtering parameters like `==` `‘‘temperature’’`.

While executing computations in internal memory significantly improves processing speed and reduces network overhead, it makes the time-window aggregations stateful. In a high-throughput scenario requiring multiple load-balanced Node-RED instances, standard round-robin routing would lead to fragmented and inconsistent results, as telemetry from the same room might be distributed across different instances.

To overcome this limitation and enable horizontal scalability in future iterations, the ingestion layer will require a topic-based partitioning strategy (e.g., sticky routing). This strategy will guarantee that all messages belonging to the same topic are deterministically routed to a single, specific Node-RED instance, ensuring perfectly consistent aggregations without losing the flexibility of the in-memory, low-code approach.

At every time window, the node pushes the aggregated data to two separate nodes. In the “Persist Timeseries Data” subflow, the data is formatted and inserted into a priority queue. The queue is designed to insert new aggregated telemetry and remove old measurements. Leveraging the timestamp as the priority of a measurement, it is removed if it exceeds the retention time, defined in the configuration variables. This queue is used by the agent to fetch the latest telemetry.

The second node connected, the “Evaluate Sensor Thresholds” node, retrieves the cached “snapshot” of the room from Redis, which provides a reliable representation of the room’s telemetry. This snapshot is used to check if the current telemetry aligns with the cache. Since each metric has its own characteristics and variability, the comparison can be personalized. For each metric, the `threshold_type` and `threshold_value` can be defined. Two strategies have been implemented: percentage (e.g., ±5% ppm) and absolute (e.g., ±2°C). Additionally, the “always” type has been introduced for debugging purposes. When the strategy type is defined, its corresponding value must also be defined. If these configurations are not defined, a standard fallback strategy is used, in this case, ±5%. If the current telemetry snapshot differs from the cached one, the flow continues, otherwise is stopped. When the data differs means that the internal systems data should be updated. The nodes “Update Neo4j Place Graph” and “Persist State to Redis” starts at the same time. The first one updates the room telemetry on the corresponding node in the place graph, the second one instead updates the room snapshot in the cache and applies a Time to Live (TTL) to the updated values.

The TTL is applied to remove the sensor metrics from the snapshot if the values have not been updated for a long time. This helps handle prolonged sensor disconnections, substitutions, or removals. The TTL acts as an automatic garbage collection mechanism: if a sensor is physically removed, replaced, or decommissioned, its latest values will

naturally expire from the room snapshot without requiring manual database cleanup, thus optimizing Redis memory usage. When a sensor reconnects after the TTL has expired, the absence of cached data forces the system to bypass the threshold checks, ensuring an immediate resynchronization of the Place Graph.

After applying the TTL, the room snapshot is inserted into a priority queue on Redis. This ensures that the agent can retrieve and analyze the room with the highest priority. There are two priorities: the first is the level of danger, and the second is the timestamp of the latest update. The danger level is derived from a previous computation by the agent in the same room; if it is not defined, it is set to the lowest level. The priority is defined as a float structured as {danger_value}.{timestamp} (e.g., 2.1786976824). This is designed to allow the agent to always extract the most dangerous rooms, ensuring a more precise prediction of the danger. Furthermore, the timestamp ensures that, given the same threat level, the oldest pending data is processed first, preventing continuous updates from a single sensor from blocking the analysis of the rest of the building. This makes it possible to handle peak loads; during an extended emergency, the queue protects the agent from overloads, guaranteeing that critical alerts are managed first. It also optimizes resources by avoiding state computations for rooms that are safe or unchanged.

Agent Output Flow

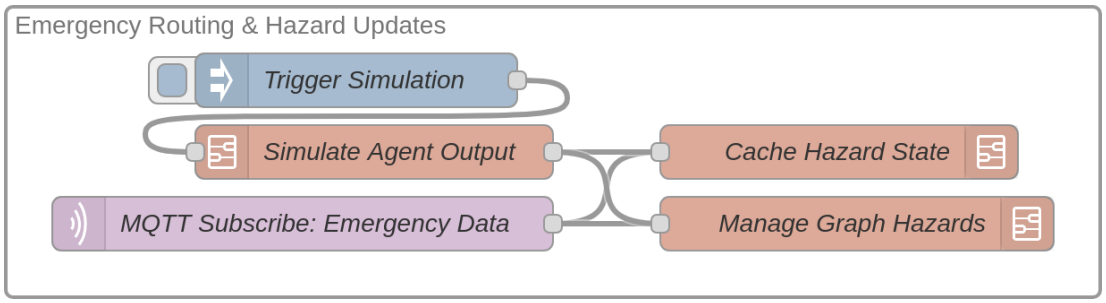


Figure 5.11: The agent output flow

Figure 5.11 illustrates how the agent's output is processed. The agent publishes its response to the MQTT topic `emergency/agent/data`, which is monitored by Node-RED. When a danger is detected, the message proceeds through the pipeline and is routed to two concurrent branches: one for updating the Redis cache and another for updating the place graph. In both branches, the message is formatted for ingestion by the respective systems.

Originally the place graph update branch was intended to incorporate a dynamic weight calculation mechanism to identify the safest evacuation routes based on real-time hazards. Due to time constraints, this dynamic weight calculation feature was not implemented in the current version of the system but have been staged as a future work. On future work will be evaluated the most optimal architectural approach: whether to continuously maintain and update a subgraph representing all shortest paths from every room to the exits as conditions evolve, or to compute these routes entirely on the fly only when an emergency is actively triggered.

5.3.2 Emergency Agent

The primary objective of the agent is to detect emerging or active hazards within a physical space based on the IoT data collected from that area. The agent relies on the core LangGraph framework. The architecture employs a multi-agent approach, establishing multiple specialized agents with expertise on individual hazards. Leveraging the framework's separation of concerns and modularity, LangGraph's conditional routing delegates tasks to specialized agents. This routing mechanism delegates complex tasks to the most appropriate agent based on the graph's state.

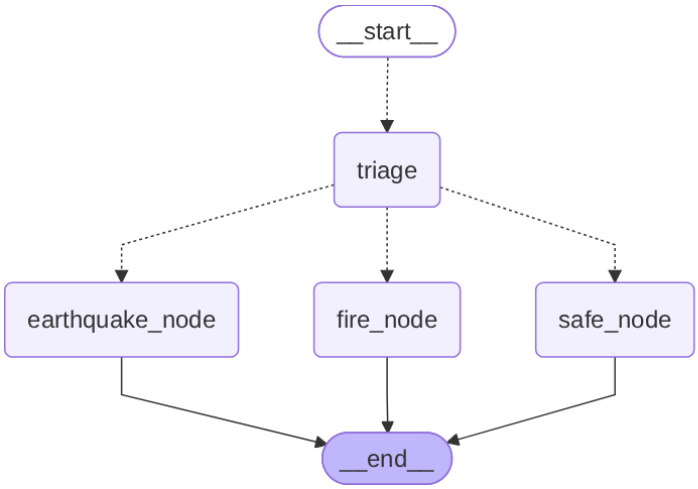


Figure 5.12: Agent architecture

Figure 5.12 illustrates the agent architecture. Dotted arrows represent conditional connections between the triage and expert nodes, whereas solid arrows denote deterministic transitions. The system implements a "one-shot" execution model to ensure low latency and to prevent the processing of outdated IoT data during faults.

Listing 1 defines the two designed graph states. The graph receives the `GraphState` class as input. This class comprises the messages, the IoT data, the required experts, and the list of generated threat assessments.

```
class GraphState(TypedDict):
    messages: Annotated[list[AnyMessage], operator.add]
    data: dict[str, Any]
    required_experts: list[str]
    assessments: Annotated[list[dict[str, Any]], operator.add]


class ExpertState(TypedDict):
    messages: list[AnyMessage]
    data: dict[str, Any]
```

Listing 1: The graph states

Listing 2 details the agents' structured output. Initially, the output specifies the analyzed room. The warning field indicates ambiguity in hazard prediction, signaling the necessity for manual inspection. The danger field provides an abstraction of the

hazard's threat level. The danger type identifies the specific hazard. The danger score quantifies the hazard's severity. The justification field serves strictly for debugging purposes.

```python
class ThreatAssessment(BaseModel):
    """Class that represent the threat assessment of a room"""

    room: str = Field(
        description="The room identifier for which the assessment is made."
    )
    warning: Literal["none", "pre-alert"] = Field(
        description="Early warning indicator for suspicious data that may
        ↪ precede a danger"
    )
    danger: Literal["high", "medium", "low", "none"] = Field(
        description="Final assessed danger level based on all gathered data."
    )
    danger_type: Literal["fire", "smoke", "heat", "earthquake", "other",
    ↪ "none"] = (
        Field(description="Type of danger identified, if any.")
    )
    danger_score: float = Field(
        description="Severity score on a scale from 0.0 to 1.0."
    )
    justification: str = Field(
        description="Brief justification for the assigned danger level."
    )
```

Listing 2: The Agent's structured output

The following defines the agents' specific responsibilities. The **triage** agent processes the received IoT data. It identifies the active hazard at a high level. Based on this initial assessment, conditional edges route the shared state to the appropriate specialized agents—**fire_node** or **earthquake_node**—for precise danger evaluation. The architecture allows concurrent invocation of multiple specialized agents to perform parallel threat assessments. If telemetry data and building node parameters indicate safe conditions, the routing mechanism directs the flow to the **safe_node**, finalizing the assessment.

The agent is created by using the LangChain's prebuilt `create_agent` function. This function allows to easily build agents by simply defining the LLM model, the tools used, response format, system prompt and a symbolic name. Details about the tools used by the agent will be provided later in this section. The agent generates the structured output defined in listing 3.

This structured separation enables the agent to return distinct data types based on the context, significantly reducing execution latency. When requiring task delegation, the Triage agent assigns the string "escalate" to the action parameter of the `EscalateAction` class and appends the target agent names to the `required_experts` parameter. During escalation, the agent omits `ThreatAssessment` generation, preventing computational resource consumption on non-specialized responses.

Conversely, if specialized agents remain unnecessary, the Triage agent assigns "assess"

```python
ExpertType = Literal["fire", "earthquake"]

class EscalateAction(BaseModel):
    action: Literal["escalate"] = Field(
        description="Select this action if a specialized analysis is
        ↪ required."
    )
    required_experts: list[ExpertType] = Field(
        description="Experts required. Do not generate assessment."
    )

class AssessAction(BaseModel):
    action: Literal["assess"] = Field(
        description="Select this action if the conditions are safe and no
        ↪ experts are needed."
    )
    assessments: list[ThreatAssessment] = Field(
        description="List of safety assessments."
    )

TriageOutput = Union[EscalateAction, AssessAction]
```

Listing 3: The Triage structured output

string to the action parameter and generates a single assessment. The system formats this assessment as a list to maintain coherence with the specialized agents' outputs.

System Prompt: Triage Agent

```
You are the Hazard Assessment Triage, the entry point of an indoor telemetry
↪ monitoring system.
You receive multiple IoT telemetry snapshots for one room.
Your sole objective is to evaluate the input data and select the appropriate
↪ action based strictly on the provided structured output schema.
1. If the data exhibits anomalies, hazards, or complex patterns requiring
↪ specialized domain analysis, select the escalate action and populate the
↪ required experts using only the valid values defined in the schema. Do
↪ not generate a threat assessment yourself.
2. If the conditions are safe, normal, or only require a basic warning
↪ without specialized analysis, select the assess action and provide the
↪ threat assessment directly.
Make your classification based entirely on the provided schema fields. Do not
↪ assume the behavior of any downstream agents, and do not invent expert
↪ types or assessment values outside of the allowed definitions.
```

The `create_agent` function also instantiates the specialized **Fire** and **Earthquake** agents, which produce a structured output restricted to a list of `ThreatAssessments` and utilize personalized system prompts.

System Prompt: Fire Agent

```
You are the Fire Safety Expert, invoked only after the Triage agent has flagged a possible fire-related anomaly in one room.

1. Analyze the telemetry for fire hazards: heat/temperature spikes, smoke density, and CO2 or other combustion gas readings.
2. Call the get_room_data tool for the primary room to ground your assessment in its known spatial properties, then call get_adjacent_rooms to identify neighboring rooms, ventilation paths, or connected structural nodes that fire or smoke could spread through. Query only as far (depth) as needed to answer whether propagation is likely | do not perform broad, speculative traversals.
3. Assess the primary room first. Only produce an additional assessment for a neighboring room when the topology and telemetry together indicate a credible propagation path (e.g. a connected room reachable via ventilation with no isolating barrier); do not assess rooms with no plausible link.
4. Base danger_score strictly on the evidence gathered: reserve 0.7-1.0 for confirmed high-heat/smoke combinations, 0.3-0.7 for a single ambiguous signal, and below 0.3 for weak or borderline readings. Keep justification short and tied to the specific values and graph facts you used | never state a room, value, or connection you did not actually observe or retrieve.
5. Tool failure handling (fallback): if a tool returns an error, times out, does not respond, or returns empty/unusable data, do NOT retry it. You are strictly forbidden from invoking the same tool more than once with the same parameters, whether the first call succeeded or failed. Never loop on a tool. Instead, proceed with the information you already have: base your assessment on the telemetry alone, do not assess any neighboring room whose connection you could not verify, and state briefly in the justification that topology/room data was unavailable. Do not invent any room, value, or connection to fill the gap, and treat a telemetry-only assessment with appropriate caution when choosing danger_score.
6. Once you have produced the ThreatAssessment(s) for every room you evaluated, stop: do not call further tools or continue reasoning.
```

System Prompt: Earthquake Agent

```
You are the Earthquake Safety Expert, invoked only after the Triage agent has flagged a possible seismic anomaly in one room.
1. Analyze the telemetry for seismic activity: vibration levels, acceleration, and any indicators of structural shift.
2. Call the get_room_data tool for the primary room to ground your assessment in its known structural/material parameters, then call get_adjacent_rooms to map load-bearing dependencies and identify connected structural elements that damage could propagate through. Query only as far (depth) as needed to answer the propagation question | avoid broad, speculative traversals.
3. Assess the primary room first. Only produce an additional assessment for a neighboring or structurally connected room when the retrieved topology and material data indicate a credible damage-propagation path; do not assess rooms with no plausible structural link.
```

```
4. Base danger_score strictly on the evidence gathered: reserve 0.7-1.0 for confirmed high-magnitude vibration/acceleration combined with a vulnerable structural link, 0.3-0.7 for a single ambiguous signal, and below 0.3 for weak or borderline readings. Keep justification short and tied to the specific values and graph facts you used | never state a room, value, or structural dependency you did not actually observe or retrieve.
5. Tool failure handling (fallback): if a tool returns an error, times out, does not respond, or returns empty/unusable data, do NOT retry it. You are strictly forbidden from invoking the same tool more than once with the same parameters, whether the first call succeeded or failed. Never loop on a tool. Instead, proceed with the information you already have: base your assessment on the telemetry alone, do not assess any neighboring room whose structural link you could not verify, and state briefly in the justification that structural/topology data was unavailable. Do not invent any room, value, or structural dependency to fill the gap, and treat a telemetry-only assessment with appropriate caution when choosing danger_score.
6. Once you have produced the ThreatAssessment(s) for every room you evaluated, stop: do not call further tools or continue reasoning.
```

Because each agent possesses the capability to formulate predictions and terminate the execution cycle, all agents share identical tools. The MCP server provides the foundational infrastructure for these tools. Memgraph's MCP server supplies the initial toolset. The architecture does not directly attach these default tools to the agents, as exposing the complete suite introduced execution latency. Unrestricted access caused agents to execute redundant database searches and generate hallucinatory queries targeting non-existent nodes and edges. Consequently, the integration restricts the available MCP toolkit to the core `run_query` function, utilizing it exclusively as the execution engine for custom, prebuilt tools.

The custom `get_room_data` tool retrieves spatial properties and metadata, supplying foundational context to the agent. Data retrieval requires the agent to input the department and room name. Listing 4 defines the corresponding query.

```
MATCH (n:Place)
WHERE toLower(n.department) = toLower('{department}')
AND toLower(n.name) = toLower('{room}')
RETURN n
LIMIT 1
```

Listing 4: `get_room_data` query

The `get_adjacent_rooms` tool executes a variable-length graph traversal to map topological flow and proximity. The agent provides the department, room, relationship depth, and a record limit as input parameters. The query in listing 5 expands outward via CONNECTED_TO relationships, flattens the resulting paths, and extracts discrete connected pairs (n1, n2) alongside their connecting edge. Strict ordering of results by traversal depth enables the sequential assessment of spatial hazard vectors from the origin point.

The Triage agent utilizes conditional edges for conversation routing. Rather than

```
MATCH (n:Place)
WHERE toLower(n.department) = toLower('{department}')
AND toLower(n.name) = toLower('{room}')

MATCH p = (n)-[:CONNECTED_TO*1..{depth}]-(target)

UNWIND range(0, length(p) - 1) AS i
WITH
relationships(p)[i] AS rel,
nodes(p)[i] AS n_from,
nodes(p)[i+1] AS n_to,
i AS level

ORDER BY level ASC

WITH rel, collect({{n1: n_from, n2: n_to, depth: level}})[0] AS data

RETURN
data.n1.name AS n1,
type(rel) AS edge,
data.n2.name AS n2
ORDER BY data.depth ASC, n1 ASC, n2 ASC
LIMIT {limit}
```

Listing 5: `get_adjacent_rooms` query

relying on handoff tools, LangGraph enables the Triage agent to analyze the initial data and update the shared state. Guided by this state, a conditional routing mechanism directs the flow to the appropriate specialized agent, transferring both control and context. This methodology facilitates the development of distinct hazard-specific agents while maintaining strict separation of concerns. The resulting architecture ensures scalability and modularity, streamlining the integration of additional agents without necessitating modifications to the central logic.

The graph architecture comprises four distinct nodes and a conditional routing edge. During compilation, typically a checkpointer is introduced to allow interrupted workflows to pause and resume. The "one-shot" paradigm of this agent necessitates the intentional omission of this feature. Resuming an interrupted workflow in this context is counterproductive; upon resumption, the underlying IoT sensor values may experience changes, rendering the previous state obsolete. Bypassing the checkpointer also eliminates disk I/O operations, thereby reducing latency. The system maintains the graph's state exclusively in volatile memory during processing and discards it upon reaching the terminal node.

Regarding the execution flow, the system incorporates a triage node, conditional routing, and expert nodes. Acting as the entry point, the triage node invokes the triage agent to analyze raw data and extract a structured output. Instead of following a static execution path, the conditional edge receives data from the triage node and dispatches the IoT telemetry exclusively to the required expert nodes. The specialized nodes invoke their respective agents, process the context, and generate a threat assessment. Listing 6 presents the graph compilation logic.

```
builder = StateGraph(GraphState)
builder.add_node("triage", run_triage)
builder.add_node("fire_node", run_fire)
builder.add_node("earthquake_node", run_earthquake)
builder.add_node("safe_node", run_safe_node)

builder.add_edge(START, "triage")

# Dynamically construct the allowed nodes list
allowed_nodes = [f"{expert}_node" for expert in VALID_EXPERTS] +
 ↪  ["safe_node"]
builder.add_conditional_edges("triage", route_experts, allowed_nodes)

builder.add_edge("fire_node", END)
builder.add_edge("earthquake_node", END)
builder.add_edge("safe_node", END)

self.app = builder.compile(debug=debug)
```

Listing 6: The graph compilation code

A simulated use case demonstrates the complete workflow. Execution starts upon the system's receipt of an IoT telemetry batch from a specific room (e.g., room AA1 in department A), registering a rapid temperature spike and elevated carbon monoxide levels.

The system initializes the `GraphState` with the raw telemetry and passes it to the Triage node. Through data analysis, the Triage agent identifies the anomaly, classifying the data as indicative of a fire risk rather than a seismic event. To minimize latency and prevent hallucinations, the agent refrains from generating a generic assessment; instead, it outputs an `EscalateAction`, configures the action parameter to "escalate", and appends the `fire_node` to the `required_experts` list.

The conditional routing mechanism evaluates the updated state. Following the request for the `fire_node`, the conditional edge dynamically directs the execution flow and shared context to the Fire agent. Upon activation, the Fire expert evaluates the telemetry, determining the necessity for spatial context to accurately predict hazard propagation. It invokes the MCP tools connected to Memgraph, initially executing `get_room_data` to retrieve the structural metadata of the affected room, and subsequently executing `get_adjacent_rooms`. This graph traversal maps the connected corridors representing potential smoke movements.

Following the aggregation of the IoT anomaly data and topological context, the Fire agent synthesizes the information. It generates a final `ThreatAssessment`, assigning a high `danger_score` (e.g., 0.85), specifying "Fire" as the `danger_type`, and writing the `justification` field. After appending this structured output to the state, the graph transitions to the terminal node. The "one-shot" execution concludes, dispatching the assessment to an MQTT topic and discarding the volatile state.

The architecture processes normal conditions with minimal computational overhead. In a scenario where incoming IoT telemetry registers standard temperatures, normal CO2 levels, and zero seismic vibrations, the Triage agent analyzes the `GraphState` and confirms the absence of anomalies.

The agent subsequently bypasses the escalation process. Avoiding delegation, it configures the action parameter to "assess" and generates a standard `ThreatAssessment` list, classifying the environment as secure. The conditional routing mechanism processes this updated state and directs the execution flow straight to the **safe_node**, skipping the specialized expert nodes. This direct path minimizes latency and database interactions, finalizing the safety assessment and terminating graph execution immediately.

The architecture inherently supports parallel execution for complex emergencies. If the IoT sensors detect a multifaceted event, such as an explosion triggering both extreme heat and structural vibrations, the Triage agent identifies the overlapping hazards. Under these conditions, the agent configures the action to "escalate" and appends both **fire_node** and **earthquake_node** to the `required_experts` list. The conditional routing mechanism dispatches the shared state to both specialized agents simultaneously. The Fire and Earthquake experts execute their respective inferences and tool invocations in parallel. The system yields two distinct `ThreatAssessment` objects: one quantifying the fire risk and the other evaluating seismic or structural damage. Then the system appends both assessments to the terminal state prior to concluding the workflow.

5.4 Environment Engine

The Environment Engine serves as the spatial and contextual component within the system architecture, transforming raw telemetry into high-level concepts and providing the context required for the orchestration of emergency responses. It provides a real-time representation of the building by mapping both static architectural elements and dynamic environmental states. This component ensures that external IoT data is contextualized within the physical geometry of the facility. To effectively manage both raw data and semantic abstractions, the Environment Engine is divided into two interdependent sub-components: the Physical Layer and the Logical Layer. The Physical Layer functions as the structural foundation, utilizing a graph database to store the building's topological data alongside real-time sensor metrics. The Logical Layer translates physical nodes into abstract concepts such as safe zones and gathering points. This dual-layer architecture allows the system to track hazard locations and understand their operational impact, preparing the environmental context for advanced evacuation routing and decision-making.

5.4.1 The Physical Layer

The Physical Layer of the place graph is defined in a JSON file and then stored in the Memgraph database, as detailed in section 5.2.4. The building is represented as a spatial graph where nodes correspond to distinct rooms or areas, and relationships denote the various physical or structural connections between them, such as walkable passages or ventilation shafts.

The proposed graph architecture is structured using a decoupled schema with distinct node labels: `Place`, `EnvironmentState`, and `Hazard`. This separation prevents the `Place` node from accumulating complex or deeply nested properties. The `Place` node stores static data such as spatial coordinates or BIM data alongside process execution variables useful to the BPMN process (e.g. turn on lights on the area, trigger alarms). The separation of process data into a node was evaluated to further enforce domain isolation, but it was ultimately retained within the `Place` node. Spatial navigation is

established through one-to-many `CONNECTED_TO` relationships between `Place` entities. The `EnvironmentState` node contains the aggregated IoT snapshot data generated by the caching flow described in section 5.3.1, mapped to `Place` via one-to-one `HAS_STATE` relationships. Similarly, the `Hazard` node captures dynamic threat evaluations produced by the agent output flow (section 5.3.1), which utilize one-to-many `AFFECTS` relationships to represent multiple threats on a space.

The JSON file representing the physical layer, processed by the Data Integration Middleware, defines the topological and contextual properties of each node. Each node entry comprises the following key attributes:

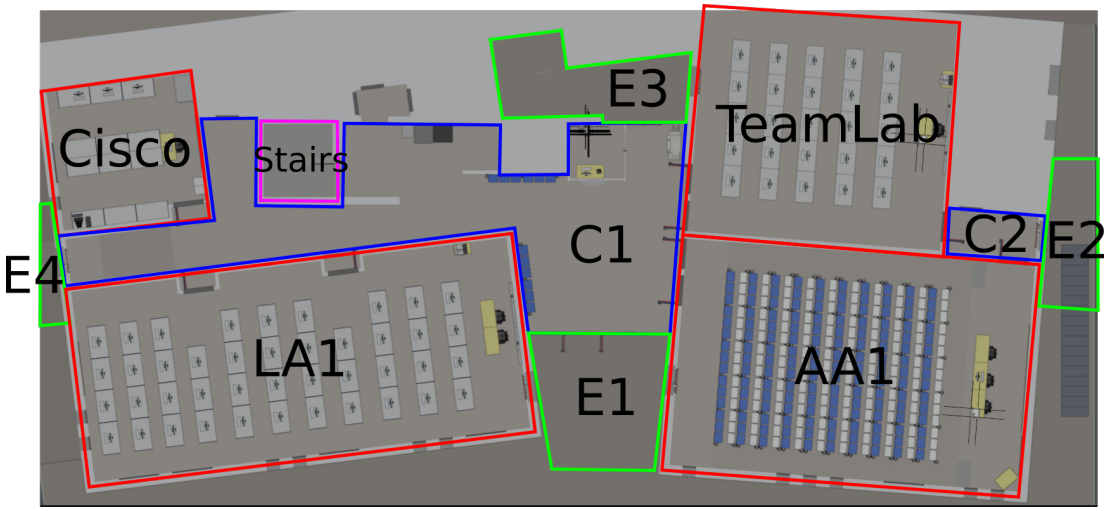
- **Node Identifier** and Name: The identifier corresponding to the actual name of the room (e.g., "AA1" or "TeamLab"). To maintain a standardized nomenclature, corridors are designated with a "C" prefix followed by a numerical value (e.g., "C1"), while emergency exits use an "E" prefix (e.g., "E1").
- **Department Name**: Specifies the building or wing, allowing the system to monitor multiple facilities independently.
- **Spatial Weight**: A numerical metric representing the degree of danger or the traversal cost associated with the room. It will be combined with the dynamic threat evaluations from the `Hazard` nodes during future pathfinding computations.
- **Attributes**: A list of properties that will be added as node parameters.

The JSON schema defines the spatial topology through specific relationship mappings:

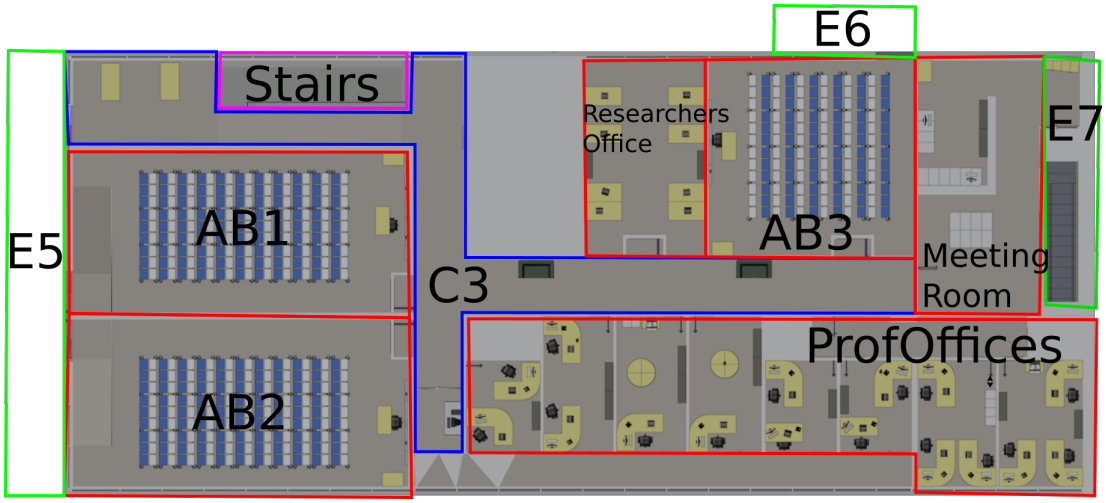
- **Relationship Identifier**: A unique string identifying the specific connection (e.g., "edge1"), optionally accompanied by a human-readable name summarizing the route (e.g., "cisco to c1").
- **Source and Target Identifiers**: The specific node ids ('source' and 'target') representing the origin and destination of the link, establishing the physical topology of the environment.
- **Relationship Type**: A descriptor defining the nature of the connection or relationship between the two nodes (e.g., "CONNECTED_TO").
- **Attributes**: An object used to store any additional metadata, routing metrics, or custom properties relevant to the relationship.

Figure 5.13 illustrates the areas of the department that have been mapped to a node in the graph. Nodes are linked via `CONNECTED_TO` edges wherever a navigable pathway exists between the physical spaces. The two floors are connected by the "Stairs" node. Elevators were intentionally excluded from the physical model in this iteration because they are typically restricted or unsafe during emergency hazard scenarios. The resulting graph has 21 nodes and 104 `CONNECTED_TO` edges, because navigable connections are modeled as pairs of directed edges to enable bidirectional traversal. Currently, the physical layer is built manually by writing the JSON; in future, an Environment Modeler, as seen in section 5.1, will be developed.

Figure 5.14 shows the spatial graph used as the primary baseline for testing and validation during the development phase. After the JSON import, only the foundational `Place` nodes (represented in red) are instantiated. The `EnvironmentState` (yellow)



(a) Ground-floor layout



(b) First-floor layout

Figure 5.13: Floor plans for Department A of the Computer Science division at the University of Camerino

and `Hazard` (orange) nodes are not present at startup; instead, they are dynamically appended by the Data Integration Middleware once the system begins processing incoming IoT telemetry and agent outputs.

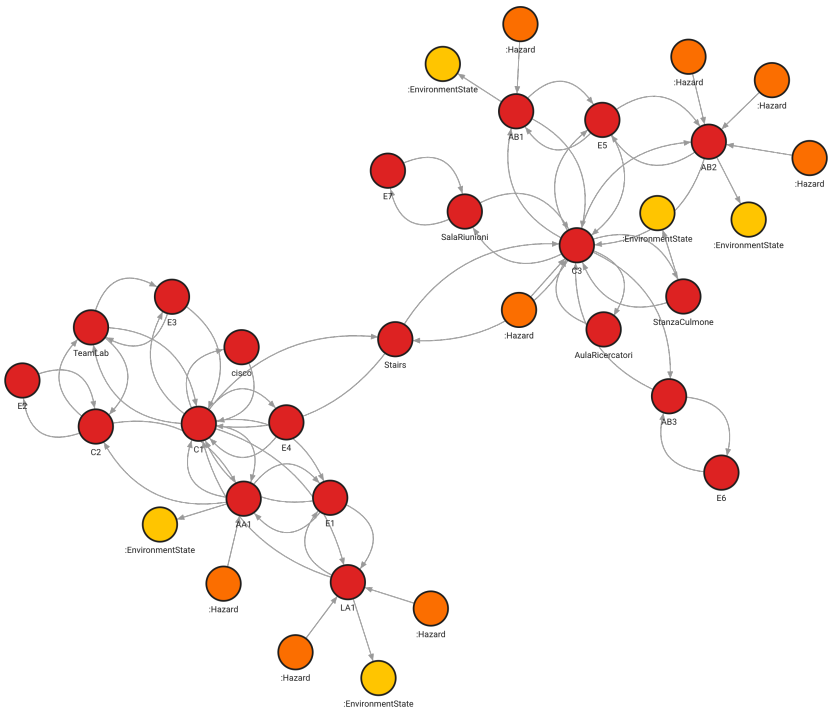


Figure 5.14: Visualization of the populated physical layer graph in Memgraph. Red nodes represent static `Place` entities, while yellow and orange nodes represent dynamic `EnvironmentState` and `Hazard` nodes, respectively.

5.4.2 The Logical Layer

Inside the JSON file that stores the place graph, logical places and views are defined in addition to the nodes and edges; these components make up the logical layer. A logical place represents a set of physical places that satisfy specific attribute conditions. A view, instead, merges one or more logical places and logical attributes. An example of a logical place is "Safezones", which extracts the places that occupants can safely traverse to exit the building.

The Logical Layer is a semantic aggregator that allows the Environment-aware BPMN engine to interpret the building represented as a graph. It allows the BPMN engine to request abstract semantic concepts, such as safe zones, to activate actuators and guide occupants during an emergency.

Within the JSON structure, each logical place is represented as an object comprising the following fields:

- **Identifier**: A unique string identifying the logical place, currently formatted as the prefix "set" followed by a sequential counter.
- **Name**: A brief description of the logical place.
- **Query**: The query utilized to extract the corresponding places.

View objects follow a similar structure and consist of:

- **Identifier**: A unique string formatted with the "view" prefix.
- **Name**: A designated term utilized by the system.
- **Logical Places**: A set of logical places that compose the view.
- **Logical Attributes**: A set of attributes, each paired with a string specifying its corresponding aggregation function. Currently, only the "COUNT" and "SUM" functions are supported.

Views can be composed of a single logical place or multiple logical places by merging the result sets of two or more distinct queries. For instance, the "FullBuilding" view combines the "Building" and "GatheringPoints" logical places. They can be divided into static views, such as "Building" or "GatheringPoints", which rely on fixed node attributes, and dynamic views, which evaluate real-time conditions by checking the relationships between `Place` nodes and linked nodes. The currently implemented views are:

- **SafeZones**: Places asserted as safe.

```
MATCH (p:Place)
WHERE NOT EXISTS {
  MATCH (p)-[:AFFECTS]-(h:Hazard)
  WHERE NOT h.level IN ['none', 'low']
}
RETURN DISTINCT p;
```

- **DangerZones**: Places recognized as dangerous.

```
MATCH (p:Place)
WHERE EXISTS {
  MATCH (p)-[:AFFECTS]-(h:Hazard)
  WHERE h.level IN ['medium', 'high']
}
RETURN DISTINCT p;
```

- **SignalledAreas**: Areas signalled by people who are unable to evacuate.

```
MATCH (p:Place)
WHERE p.signalled = true
RETURN p;
```

- **Building**: Places inside the university.

```
MATCH (p:Place)
WHERE p.amenity = 'university'
RETURN p;
```

- **GatheringPoints**: Emergency exits.

```
MATCH (p:Place)
WHERE p.amenity = 'gatheringPoint'
RETURN p;
```

- **FullBuilding**: A comprehensive view encompassing the entire structure by merging the "Building" and "GatheringPoints" logical places.

By abstracting the raw spatial and telemetry data into these semantic views, the Logical Layer acts as a bridge between the physical environment and the system's operational logic. This architectural decoupling is essential for the Environment-aware BPMN Engine, which relies on these high-level concepts to orchestrate emergency responses and dynamically actuate building guidance systems.

5.5 Environment-aware BPMN Engine

Standard process engines are built to orchestrate administrative workflows, they do not natively account for the physical environment where tasks actually take place. Traditional BPMN tasks are represented as logical steps, they do not consider the spatial dimensions and physical context of the real world. When managing indoor emergencies, the operational logic cannot be separated from the physical space. The environment directly influences how an evacuation process is executed, while the process may also need to modify the environment in response, for example through an actor extinguishing a fire or the system changing physical signage.

In the proposed architecture, the Environment-aware BPMN Engine is responsible for deciding how the process should react to the current state of the environment. It uses the semantic concepts provided by the Logical Layer, such as identified safe zones and danger areas, and uses them to coordinate real-time physical responses to guide occupants. To enable this interaction, standard BPMN tasks had to be reconsidered and adapted to interact dynamically with the physical space.

5.5.1 BPMN Extension and Custom Tasks

Standard BPMN shows limitations when applied where business processes and collaborations are developed in a physical place characterized by dimensional spaces and context [25]. The main limitations include a lack of native integration with the environment; BPMN does not provide tools to model the environment or describe the bidirectional relationship between the environment and process execution. Examples include cases where the environment influences process execution (e.g., an occupant cannot find a safe evacuation path) and when the process modifies the environment (e.g., a person extinguishes a fire) [25]. The BPMN standard does not allow users to specify constraints on coordinated or synchronized movements between multiple participants (e.g., an emergency warden leading an occupant to the exit), where one participant passively follows the movements of another [25].

To allow interaction with the environment, the BPMN metamodel is extended with three new types of tasks that interact with the Physical Layer and Logical Layer: the Movement Task, the Binding Task, and the Unbinding Task. However, in this thesis

project, the extension has not been strictly applied using custom task types as defined in Corradini et al. [25].

The **Movement Task** is an activity that involves the physical movement of a participant, and potentially other bound participants, across the place graph. It handles the relocation of a participant toward a destination. Instead of using rigid physical locations, it targets logical places that are dynamically mapped to physical nodes in real-time. During execution, the active participant moves along the path and automatically drags along any other participants currently linked to them via the Binding Task. The process can also modify the environment upon arrival by updating place attributes [25].

The **Binding Task** is a synchronization task that permits two participants to "link" with each other in order to temporarily perform coordinated movements in the environment. Both participants must be located in the same physical place to perform this task. Once executed, the passive participant follows the physical movements of the active one [25].

The **Unbinding Task** breaks the previously established link with another participant. After execution, both participants regain their independence and can resume separate movements and activities within the physical environment [25].

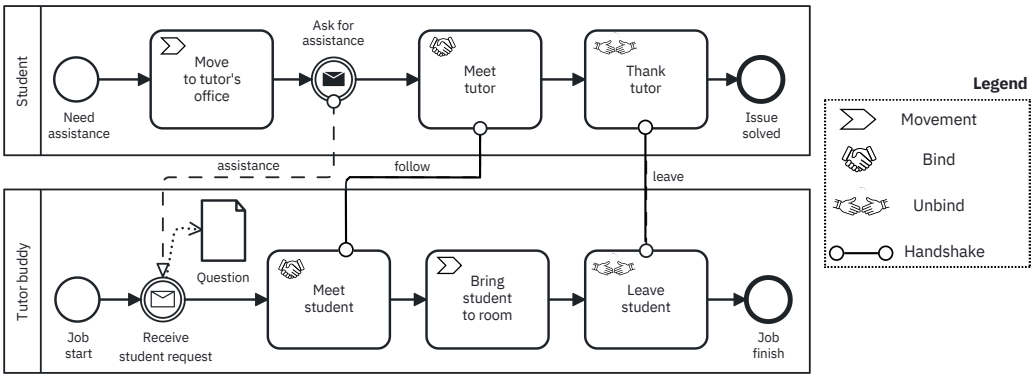


Figure 5.15: A process example involving new tasks [25].

Figure 5.15 shows a minimal example that uses the extended tasks. The student starts the process by moving to the tutor's office to request course assistance. After registering the request, the tutor locates an available study room, and the two participants perform a binding task. This synchronizes their movements, enabling the student to follow the tutor through the shared environment to the destination. Once they reach the room and the assistance is complete, they execute unbinding tasks to separate and conclude their coordinated interaction [25].

Even though this thesis builds upon the theoretical foundation of environment-aware BPMN collaborations, the extension was not implemented using the custom task types conceptualized in the reference literature. Instead of using the introduced elements, standard BPMN elements were adapted to bridge the theoretical metamodel with practical execution. The Movement Task was implemented as a Service Task, while the Binding and Unbinding Tasks were implemented as Send and Receive Tasks.

5.5.2 Emergency Process Modeling & Implementation

This section describes the design of operational workflows for emergency management and details regarding their subsequent implementation in the orchestration engine.

Modeling

The process has been developed by defining three pools: the **Security System**, the **Emergency Warden** and the **Occupant**.

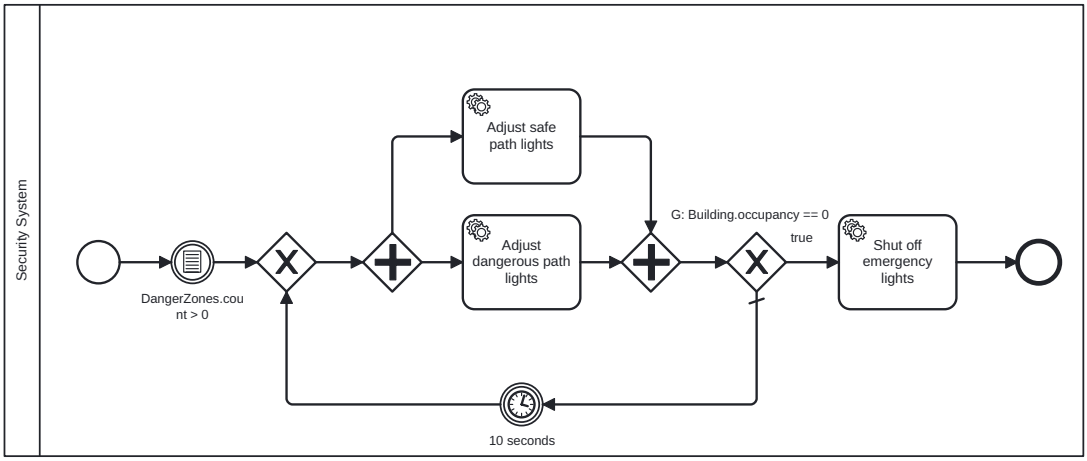


Figure 5.16: The Security System Process

Figure 5.16 details the operational flow of the Security System pool. To comply with the Camunda orchestration engine's requirement for a None Start Event, the process initializes and immediately halts at an Intermediate Conditional Catch Event. Active execution is triggered when the environment engine detects a hazard, waiting for the logical attribute `DangerZones.count` to exceed 0.

When active execution begins, the process enters an execution loop managed by Exclusive Gateways. A Parallel Gateway splits the flow into two tasks that run at the same time. The first turns off the lights in hazardous zones (`A: DangerZones.lights := ‘‘off’’`). The second turns on the lights of the safe evacuation corridors (`A: SafeZones.lights := ‘‘on’’`) to prevent occupant panic. A second Parallel Gateway waits for both tasks before the flow continues.

The system then evaluates the `Building.occupancy` logical attribute. An execution listener on the gateway retrieves its current value before the condition is checked. If the building is not yet empty, a Timer Catch Event pauses the execution for 10 seconds before looping back to readjust the lights based on the latest hazard predictions. When the building is fully evacuated, the loop breaks. The process then concludes with the "Shut off emergency lights" service task, which turns off the lights of the safe zones (`A: SafeZones.lights := ‘‘off’’`) before reaching the End Event.

Figure 5.17 shows the collaborative processes of the Emergency Warden and the Occupant during an emergency event. Similar to the Security System, the Occupant process is triggered by a Conditional Start Event when a hazard is detected in the environment (`DangerZones.count > 0`). After the initiation, the occupant attempts to evacuate the building by executing the "Get out" service task; it represents a movement action where the occupant attempts to navigate toward the emergency exits

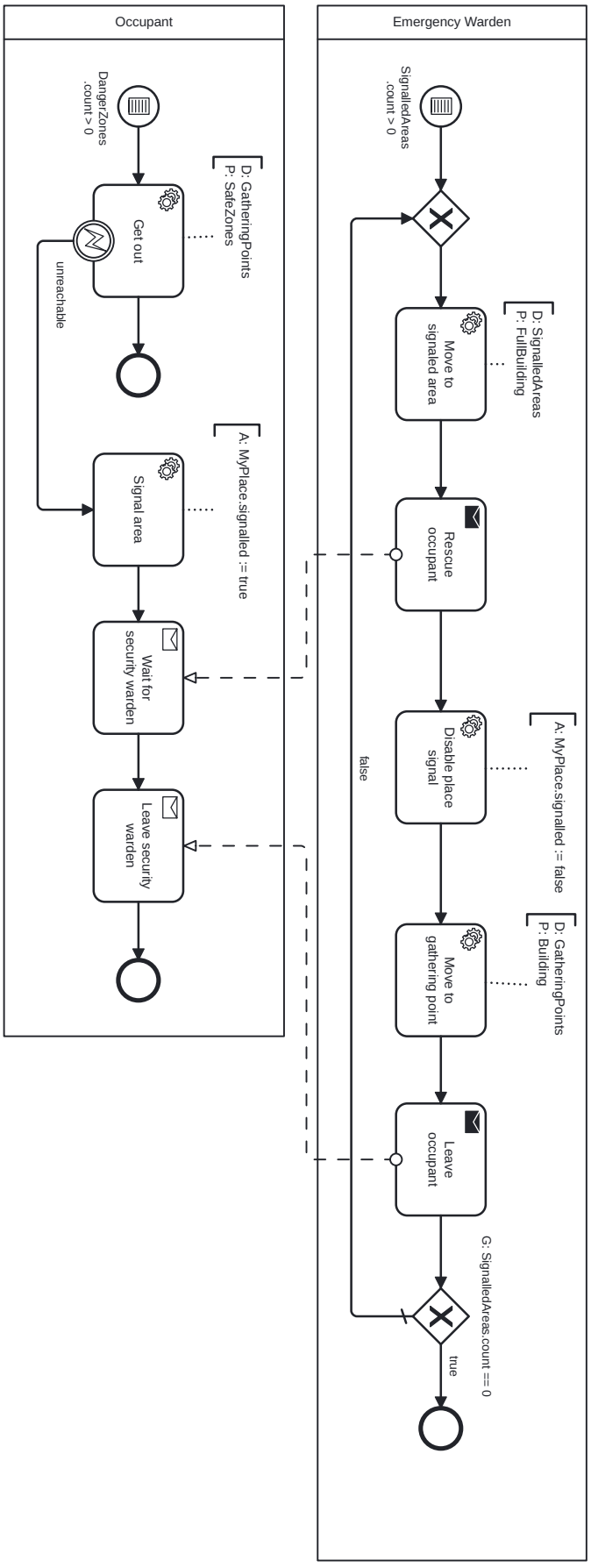


Figure 5.17: The Emergency Warden and Occupant Processes

(`D: GatheringPoints`) exclusively through paths designated as safe (`P: SafeZones`), following the dynamic lighting managed by the Security System.

If a safe path to a gathering point exists, the occupant successfully evacuates and the process terminates normally. Otherwise, if the occupant is trapped by surrounding hazards and the exit becomes unreachable, a Boundary Error Event deflects the flow to the recovery process. In this flow the occupant executes the "Signal area" service task, which updates the environment by setting the local place attribute `A: MyPlace.signalled := true`. Then the occupant waits at their current location for assistance via a Receive Task ("Wait for security warden").

The Emergency Warden process is instantiated by a Conditional Start Event that monitors the environment for distress signals, activating as soon as `SignalledAreas.count > 0`. Once activated, the warden iteratively rescues occupants. The first step involves a service task directing the warden to move toward a distressed occupant's location (`D: SignalledAreas`). Unlike the occupant, the warden is not limited to safe paths, since it moves along the whole building (`P: FullBuilding`), and can cross hazardous zones to reach the target.

Upon reaching the signaled area, the warden initiates a coordinated interaction with the occupant. While theoretically modeled using Binding and Unbinding tasks to represent synchronized movement, this coordination is practically implemented using Send and Receive Message Tasks. The warden executes the "Rescue occupant" Send Task, which is caught by the occupant's "Wait for security warden" Receive Task, binding the two participants. The message is correlated through the name of the signalled place (`MyPlace.name`) and carries the identifier of the warden (`WardenID`), which the occupant saves as `WardenRescuerID`. The warden then executes the "Disable place signal" service task, which removes the distress signal (`A: MyPlace.signalled := false`). Finally, it guides the occupant to a gathering point via the "Move to gathering point" task (`D: GatheringPoints`, `P: Building`).

At the gathering point, the warden executes the "Leave occupant" Send Task, corresponding to the occupant's "Leave security warden" Receive Task. This message is correlated through `WardenID`, which matches the `WardenRescuerID` saved by the occupant. This interaction unbinds the participants, allowing the occupant's process to conclude. The warden then evaluates the environment state through an Exclusive Gateway checking `G: SignalledAreas.count == 0`. An execution listener on the gateway updates this count before the condition is checked. If other occupants still require assistance, the execution loops back to locate the next signaled area. If all signals have been resolved, the Emergency Warden process terminates.

Implementation

The communication between the Camunda Engine and Node-RED is based on the External Task pattern (Job Worker pattern). In this architecture, Camunda orchestrates process instances while the physical and analytical execution of activities is delegated to Node-RED, acting as an external worker. To allow this communication, the external library `node-red-camunda8` is employed, allowing Node-RED flows to interact with the engine by using appropriate nodes, such as workers, message, errors, deploy and so on.

The interaction relies on an input mapping mechanism defined in the Camunda modeler. When Camunda activates a service task, the generated JSON job payload is polled by the worker node provided by `node-red-camunda8`. Node-RED parses this data and extracts the essential process context variables needed for its job, such as path

computation or place graph updates via the logical layer.

Execution terminates once the task logic is completed, to do so Node-RED triggers the library's completion node, which sends a request back to the engine to mark the external task as finished. This updates the instance state within Camunda and advances the process token to subsequent gateways and tasks in the workflow.

Multiple subflows manage interactions with the Environment Engine. Grouping these operations into macro-tasks keeps the system modular and easier to maintain The primary subflows include:

- **View Aggregation**: Queries the Logical Layer to retrieve the nodes of a logical view. It then computes an aggregate (sum or count) of one of their attributes.
- **Movement**: Implements the Movement Task by computing the shortest path to the destination and updating the participant's position. It uses the *Path* and *Destination* strings from the Camunda input mapping to locate the correct logical views.
- **Assignment**: Updates node attributes within a defined logical view mapping the *Value* to the target *Attribute* using the logical view *Place* provided by the input mapping. If *Place* is `MyPlace`, the node of the participant's current location is used instead.
- **Binding/Unbinding**: It sends the binding message by providing a message name on the subflow ui.
- **Utility**: Accessory subflows that handle routine background operations, such as data cleaning, database queries, and fetching place graph data.

A total of 13 flows were developed to implement the BPMN model. The majority of these consist of a worker wrapper and a macro-task, such as Movement or Assignment, as illustrated in figure 5.18. The wrapper manages the worker listener, handles Camunda context data, and ensures proper task completion. For brevity, only the flows containing more than these two basic nodes are detailed in this section. Multiple tasks rely on a single macro-task node. The "Adjust safe path lights", "Adjust dangerous path lights", "Shut off emergency lights", and "Disable place signal" tasks exclusively use the Assign node. The "Move to signaled area" and "Move to gathering point" tasks depend solely on the Movement node. The `get-building-occupancy` and `get-signalled-areas` execution listeners, attached to the two exclusive gateways, use only the View Aggregation node. Lastly, the binding tasks for the Emergency Warden are implemented using only the Binding subflow.

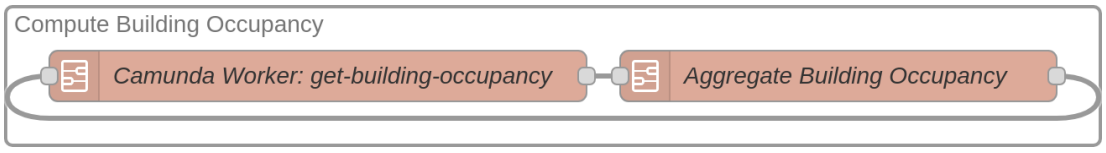


Figure 5.18: Example of a simple flow implementation

During an occupant's evacuation attempt, there is a possibility that no safe path is available to follow. When this happens, the BPMN engine interprets this situation as an error, and an alternative task sequence is triggered. Figure 5.19 shows how this

operational behavior is implemented in Node-RED. The execution flow begins with a Camunda Task Wrapper that continuously polls for `occupant-get-out` job requests. Once a job is successfully received and parsed, the `MOVEMENT` subflow is activated. This subflow extracts the logical views for the path and destination, and then queries the graph database (Environment Engine). The database dynamically calculates the optimal route using a weighted shortest-path algorithm, based on the occupant's current location, the permitted paths, and the final destination. After the database query, a switch node evaluates the response payload to determine if at least a path exists. If a path is found (indicated by a non-empty payload), the system updates the occupant's location variables and routes the token back to the wrapper to successfully complete the task. Otherwise, if no path is found (resulting in an empty payload), a dedicated error-handling subflow is activated. This subflow transmits a specific `no-way-found` error code and message back to Camunda. This error propagation allows the BPMN process to catch the boundary event and execute the alternative evacuation recovery logic.

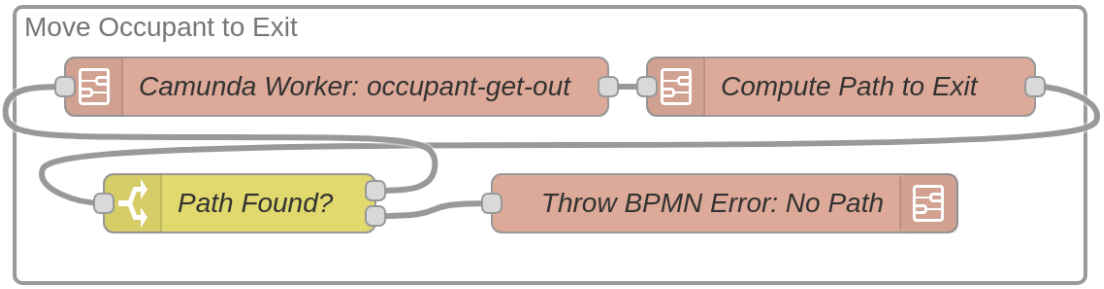


Figure 5.19: The occupant evacuation flow

Figure 5.20 shows the implementation of the "Signal area" task. The execution begins with a Camunda Task Wrapper that receives the job request and forwards the payload to an assignment subflow. This subflow updates the occupant's current location in the Environment Engine by setting the `signalled` attribute to true. Then a View Aggregation subflow calculates the total number of active signaled areas by evaluating the `count` logical attribute. This aggregation is done to trigger the Emergency Warden's Conditional Start Event to activate it as soon as distress signals are detected within the environment. To prepare for the warden's deployment, a function node retrieves the necessary process definition keys and dynamically assigns the warden's initial position to a randomly selected emergency exit. The flow finishes when an HTTP POST request is dispatched to the orchestration engine to evaluate the conditional start event, triggering the Emergency Warden process before the task wrapper completes execution. To trigger the conditional start event, a custom HTTP request have been sent because the library `node-red-camunda8` does not implement a node to do so yet.

The flow in figure 5.21 distinguishes itself from the previously described workflows because it does not use a Camunda external task worker. Instead, the flow is triggered by an MQTT input node subscribed to the `emergency/agent/data` topic, that is used by the agent to publish their results. During tests, this input can be replaced by an inject node that replays agent data stored in a JSON file. After that the message is received, execution is intentionally delayed by five seconds to ensure that the agent's output flow, detailed in section 5.3.1, has fully concluded its environmental updates before the flow proceeds. This delay ensures that if a danger is spotted, then the following node will be able to detect it from the Environment Engine. The View Aggregation subflow computes

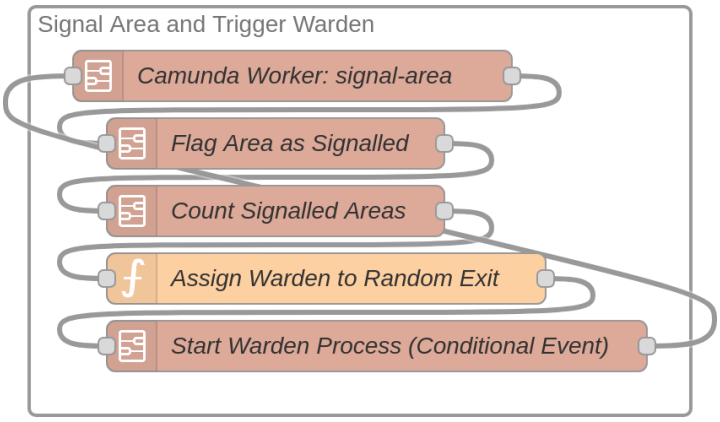


Figure 5.20: The occupant signal area flow

the total number of active hazard zones, then the calculated `DangerZones.count` is sent to a Camunda node to update the process variables in order to be read later by the conditional event on the orchestration engine. Given that the current implementation serves as a pure simulation rather than a physical deployment, a function node is utilized to initialize the occupant's starting context by assigning a hardcoded location identifier (e.g., "AB2") to the `MyPlace.name` attribute of the occupant's process variables, in place of a value that would otherwise be provided by an indoor positioning sensor (e.g., GPS, Wi-Fi RTT). Since the flow logic uses `MyPlace.name` regardless of its origin, this substitution does not affect the validity of the subsequent processing steps and allows the system's behavior to be evaluated independently of a physical sensing infrastructure. In a real-world deployment, this node would be replaced by an interface to the actual positioning system. Finally, this structured data is routed to the subflow that executes an HTTP POST request to the engine's endpoint, that evaluates the updated process variables and subsequently triggers the Conditional Start Event, thereby instantiating the Occupant evacuation process if the condition is satisfied.

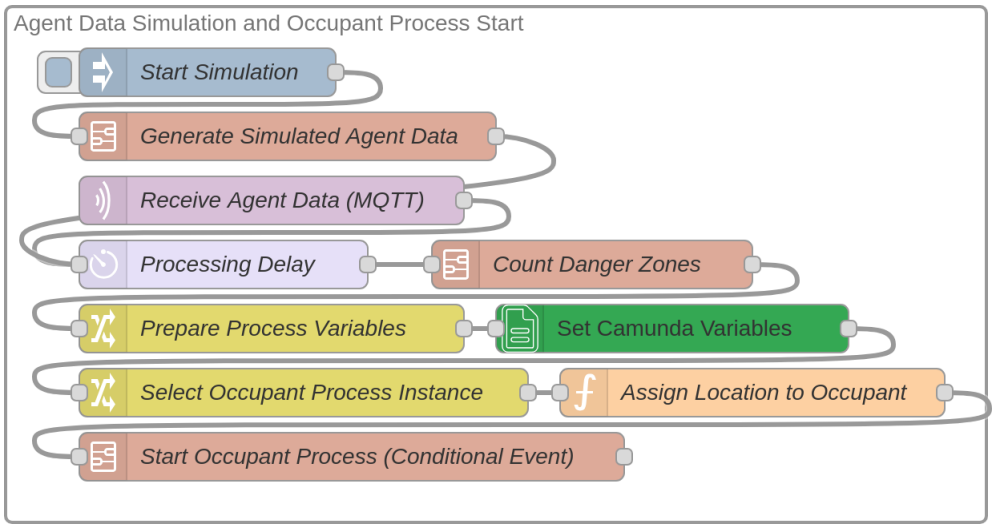


Figure 5.21: The Emergency Conditional Flow

The final flow is responsible for deploying and starting the emergency process on the Camunda engine. Figure 5.22 illustrates the sequence of operations required to

start the process instance. The execution is triggered automatically when the Node-RED environment starts or restarts. When triggered, the system reads the BPMN file containing the emergency evacuation diagram from the local directory. The *Deploy and Start BPMN Process* subflow structures the diagram data into the required payload format and deploys it to the Camunda engine. Following a successful deployment, the ID of the Security System process (`Process_SecSys`) is extracted from the deployment response, and a new instance of that process is created. The process variables are then initialized as an empty set and passed to the *Process State Initialization* subflow, which sets them on the process instance. The same subflow saves the deployment data in the flow context, so that the other flows can later start the Occupant and Warden processes.

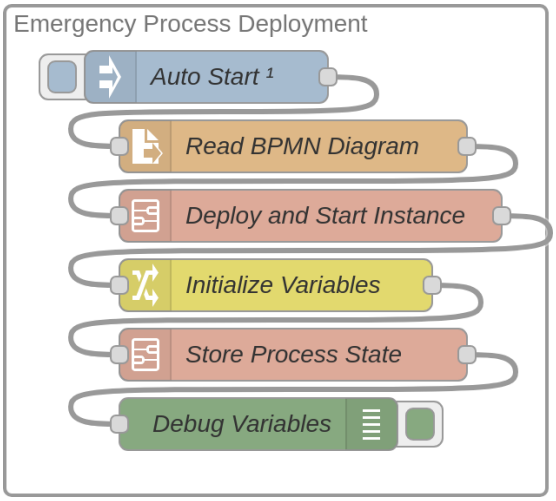


Figure 5.22: The Process Deployment Flow

This section shows how a conceptual emergency management strategy was implemented as a running, event-driven process. The Security System, Emergency Warden, and Occupant pools operate concurrently in the same environment. They react to changes through the Environment Engine by identifying safe and dangerous zones, adjust building systems and guiding evacuations in real time.

Standard BPMN elements were used instead of custom tasks to handle movement and coordination while maintaining compatibility with standard engines like Camunda. Using messages rather than native synchronization introduces some rigidity, but it allowed the prototype to rely on functionality supported by the BPMN engine. The calculations and interactions with the Environment Engine are handled by Node-RED, allowing the BPMN engine to focus primarily on the process logic.

In conclusion, the implemented system demonstrates that an environment-aware evacuation process can be deployed and executed end to end, from hazard detection to occupant guidance and warden intervention. Some aspects, such as dynamically weighting paths based on live hazard data, remain open for future refinement.

6. Evaluation

This chapter evaluates two parts of the prototype: the routing strategy and the multi agent system that feeds it with hazard data.

The first part evaluates whether the system generates safe evacuation routes and adapts them as building conditions change, using the graph shown in figure 5.14 and comparing the proposed routing strategy against a traditional static approach unaware of current hazards. Three scenarios are considered: S0 represents the building under normal conditions and serves as a baseline, S1 introduces a single static hazard, while S2 simulates a sequence of hazards appearing over time. For each scenario, results are discussed with particular attention to cases where no safe route exists, distinguishing structural limitations of the graph from limitations of the routing strategy itself. This part of the evaluation is conducted at the topological level: physical occupant movement, egress speed, and crowd behavior are not simulated and are left to future work.

The second part evaluates the multi agent system that turns raw sensor readings into hazards recorded in the place graph, the component that the routing strategy relies on to know where the danger is. Here the focus is on the agents themselves: whether they produce output the rest of the system can use, whether the triage agent sends each situation to the right specialized agent, and how much time, tokens, and money a single analysis takes. This part does not measure how accurate the hazard classification is, that question is left for future work.

6.1 Evaluation Objectives

The goal of this evaluation is to assess whether the proposed framework provides safe and adaptive evacuation routes when multiple, dynamic hazards are present. The evaluation focuses on the interaction between the building state and the routing process, considering safety compliance, path cost, feasibility and adaptability.

A first objective is to verify that the routing strategy avoids dangerous zones and prefers the safest ones. Hazard avoidance is treated as a hard constraint: hazardous areas must not be traversed if a safer alternative exists. The routes produced by the framework should completely avoid hazardous zones while still selecting the minimum-cost path among the safe alternatives.

A second objective is to measure how much the cost of safe paths differs from that of standard routes. The proposed strategy is compared against a static pathfinding approach, run on the same building graph (shown in figure 5.14) and evacuation origins, which selects the minimum-weight path without considering hazards. Comparing the path weights of the two strategies quantifies the additional cost of enforcing safety.

Another objective is to evaluate whether the routing can find safe paths when multiple hazards are happening, and correctly determine when no feasible safe route exists. In

that case, the system should report that no safe evacuation route is available rather than returning an unsafe one.

The evaluation also aims to assess how the framework reacts to changing conditions. When the building state changes and a previously safe route becomes dangerous, the framework should update the routing and compute a new safe alternative if one exists. This addresses scenarios where hazards propagate over time and safe paths change accordingly.

A final objective concerns the computational performance of the system, including the latency of graph updates, route computation, and the response of the multi-agent component where applicable. These measurements assess the responsiveness of the software rather than the physical dynamics of an evacuation.

The routing evaluation (section 6.2) provides the evidence for RQ3, and the evaluation of the multi-agent system (section 6.3) provides the evidence for RQ1.

The evaluation is conducted at the graph level: each accessible room is treated as an evacuation origin, and routing strategies are evaluated by computing paths from these origins to the exits. Physical movement of occupants is not simulated, so evacuation time, pedestrians speed, congestion, bottlenecks, crowd dynamics and individual behavioural responses are not considered, though they could be addressed in future work.

The second part of the evaluation concerns the MAS, which writes the hazards on the building graph in order to spot the dangerous zones for the routing. The objectives of this part are three. The first is to check that the agents return outputs in the predefined structured format, since the following components read them directly and an invalid output would break the pipeline. The second is to check that the triage agent handles normal situations on its own and sends the others to the specialized agent that matches the hazard, and that the danger score is coherent with the type and severity of each scenario. The third is to measure the latency, the number of LLM and tool calls, the tokens and the cost of a single analysis, and how much of the delay comes from the queries to the graph rather than from the model. The classification accuracy of the agents is not measured here and is left for future work.

6.2 Evacuation Routing Evaluation

This section evaluates the routing strategy, the first part of the evaluation described in section 6.1. The hazards will be manually inserted in the place graph, so the MAS is not involved here and is evaluated separately in section 6.3. The results are read against the routing objectives: hazard avoidance, cost of safe paths, feasibility and adaptation to changing conditions. Three scenarios are analyzed: S0, where the building is in its normal state and no hazard is present, S1, where a single static hazard is introduced, and S2, where hazards appear one after another as the emergency evolves. Each scenario is discussed separately, reporting the relevant metrics and commenting on the cases where no safe route could be found. S0 is presented first, since it is the baseline the other two scenarios are compared against.

6.2.1 Metrics

The following metrics are used to evaluate the proposed framework, covering five aspects: route cost, safety compliance, route feasibility, adaptation capability, and computational performance.

Path Weight. Path Weight (PW) represents the topological cost of an evacuation route. For a path P composed of a sequence of nodes, it is defined as:

$$PW(P) = \sum_{v \in P} w(v), \tag{6.1}$$

where $w(v)$ denotes the weight associated with node v.

This metric is used to compare the cost of routes generated by the static and adaptive routing strategies.

Hazard Exposure. The Hazard Exposure (HE) measures the number of hazards associated with the nodes traversed by a route:

$$HE(P) = \sum_{v \in P} h(v), \tag{6.2}$$

where $h(v)$ represents the hazard value associated with node v.

A valid safe route generated by the framework should satisfy:

$$HE(P_{\text{adaptive}}) = 0. \tag{6.3}$$

Hazard-Free Route Rate. The Hazard-Free Route Rate (HFR) measures the proportion of valid adaptive routes that do not traverse hazardous nodes:

$$HFR = \frac{N_{\text{hazard-free adaptive routes}}}{N_{\text{valid adaptive routes}}} \times 100. \tag{6.4}$$

For scenarios where a safe route exists, the expected value is 100%. If a scenario has no safe path, the system should return "no feasible route" instead of a dangerous one.

Safe Path Cost Overhead. The SPCO measures the additional cost introduced by hazard avoidance with respect to the static baseline:

$$SPCO = \frac{PW_{\text{adaptive}} - PW_{\text{static}}}{PW_{\text{static}}} \times 100. \tag{6.5}$$

A positive value indicates that the adaptive strategy chose a safer but more expensive route.

Route Success Rate. The RSR measures the proportion of evacuation origins for which a feasible safe route to an exit can be found:

$$RSR = \frac{N_{\text{origins with feasible safe route}}}{N_{\text{origins}}} \times 100. \tag{6.6}$$

When no safe route exists, the framework should report that no feasible evacuation route is available rather than return a dangerous one.

Route Adaptation Rate. The Route Adaptation Rate (RAR) measures the framework's ability to update safe paths when new hazards are discovered in the environment:

$$RAR = \frac{N_{\text{correctly adapted routes}}}{N_{\text{routes requiring adaptation}}} \times 100. \tag{6.7}$$

This metric is evaluated in dynamic hazard scenarios, where a previously valid route may become unsafe as the emergency evolves.

Computational Latency. Computational latency measures the software-level responsiveness of the framework, including graph update latency, route computation time, and agent response time. These measurements describe the computational performance of the architecture and are not meant as a measure of physical evacuation performance.

6.2.2 No-Hazard Scenario (S0)

This scenario represents the everyday state of the building, where the graph is composed only of `Place` nodes and their connections.

Table 6.1: Summary S0

Metric	Value
Number of Origins	14
Valid Adaptive Routes	14
HFR%	100.0
RSR%	100.0
Origins with Static PW = Adaptive PW	14
Origins with Static PW = Adaptive PW (%)	100.0

This scenario verifies that the proposed framework behaves like the static routing strategy when no hazard is present. Table 6.1 confirms this: both strategies find an evacuation path for every origin, and the last row shows that the framework does not modify routing unnecessarily in the absence of a hazard. These results establish S0 as the baseline against which the deviations introduced by hazards in S1 and S2 are measured.

6.2.3 Static Route Hazard (S1)

The first hazard scenario simulates a high-severity hazard confined to C1. The hazard remains static and does not propagate to neighboring places.

Table 6.2 shows how the framework reacts to the hazard. LA1 is unaffected because its route to the emergency exit never passes through C1, and the same holds for every other unaffected origin that are not listed, only the relevant cases are listed here. Stairs is the clearest example: the static route crosses C1 directly, exposing occupants to the hazard (HE=1), while the adaptive route reroute through the first floor and reaches a different exit, removing the exposure entirely (HE=0) at the cost of a 33.33% increase

Table 6.2: Illustrative example

Origin	**Static PW**	**Adaptive PW**	**SPCO (%)**	**Static HE**	**Adaptive HE**
Stairs	12.0	16.0	33.33	1	0
LA1	14.0	14.0	0.0	0	0
C1	9.0	N/A	N/A	1	N/A
cisco	13.0	N/A	N/A	1	N/A

in PW. C1 and cisco instead have no safe alternative: C1 is the hazard source itself, and cisco is connected only to C1, so both origins correctly return no feasible route rather than a dangerous one. This result is a consequence of the structural vulnerability of the graph, since no alternative edge exists to route around the hazard from either node.

Table 6.3: SPCO distribution across the origins affected by the hazard

Metric	**Value**
N° Origins with Static Route on Hazard (all)	3
N° Origins (reroute case)	2
N° Origins (hazard = origin itself)	1
RSR% [reroute cases]	50.0
RSR% [all affected]	33.33

Table 6.3 summarizes the experiment. Only Stairs undergoes a reroute, so its SPCO of 33.33% is also the mean, median, and maximum by construction. HFR is 100%, confirming that every valid adaptive route avoids the hazard entirely. RSR value depends on the number of origins considered: among the two reroute cases it is 50%, since Stairs succeeds and C1 fails on itself, and among all three affected origins, including the case of cisco, it drops to 33.33%. Computed over the full set of 14 origins, RSR for S1 is 85.71%, since only C1 ends up without a route; this figure best reflects the framework's overall robustness in this scenario.

6.2.4 Dynamic Hazard (S2)

This scenario evaluates how the framework reacts when hazards appear one after another, measuring its ability to keep routing occupants safely to an emergency exit as conditions change. Four steps are evaluated: t0 is used to establish a hazard-free baseline, and hazards are then introduced step-by-step in C1, E7, and E5.

Table 6.4 shows the framework's effectiveness based on the static alternative. At t1 the static strategy would leave RSR at 85.71%, meaning some occupants would be routed through a hazardous zone or left without guidance, while the adaptive framework keeps HFR at 100% throughout all four steps. The mean adaptive PW grows from

Table 6.4: Step-by-step evolution

Step	Active Hazard	RSR%	HFR%	Mean Adaptive PW	N Routes Recalculated
t0	none	100.0	100.0	12.21	0
t1	C1	85.71	100.0	12.75	3
t2	C1 + E7	85.71	100.0	13.67	1
t3	C1 + E7 + E5	85.71	100.0	18.08	7

12.21 to 18.08 across the run, and the growth is not linear: the increase from t2 to t3 (+4.41) is far larger than the two previous steps combined (+0.54 and +0.92), showing how fewer safe alternatives remain as the hazard spreads. Across all 11 transitions that required a reroute, the framework produced a safe route, or correctly reported none available, in every case, giving a RAR of 100%.

Table 6.5: Concrete examples of adaptation

Origin	Step	PW Before	PW After	Outcome
C1	t1	9	N/A	no_feasible_route_reported
cisco	t1	13	N/A	no_feasible_route_reported
Stairs	t1	12	16	adapted_safe_route
SalaRiunioni	t2	7	18	adapted_safe_route
SalaRiunioni	t3	18	23	adapted_safe_route
Stairs	t3	16	21	adapted_safe_route

Table 6.5 lists a subset of the recalculated paths; the t1 step reproduces the same outcome as S1, including the no-route cases for C1 and cisco. Stairs and SalaRiunioni are each rerouted twice over the course of the run, at t1/t3 and t2/t3 respectively, and their PW increases every time a new hazard invalidates their previous path.

Table 6.6: Comparison of RSR Between S0, S1, and S2

Scenario	RSR% (all origins)	RSR% (excl. self-hazard)	Self-Hazard Origins	SPOF Origins Without Route
S0	100.0	100.0	0	0
S1	85.71	92.31	1	1
S2	85.71	92.31	1	1

Table 6.6 confirms RSR across S0, S1, and S2 to isolate the effect of the structural

vulnerability from the hazard-handling behavior itself. S0 remains at 100.0% on both metrics, confirming the absence of any single point of failure under normal conditions. S1 and S2 report an 85.71% RSR when all origins are counted, this result comes from the inability to find a route on C1 (source of the hazard) and the cisco room connected. Once the self-hazard case is excluded, RSR rises to 92.31% for both S1 and S2, indicating that the framework resolves every origin for which an alternative path structurally exists. This test revealed how the number of Single Point Of Failure (SPOF) rooms (rooms connected to a single room, here the cisco room) change the RSR results in S1 and S2, the remaining failures comes from graph connectivity rather than from limitation of the adaptive routing strategy.

Table 6.7: Cross-Scenario Summary Table

Scenario	**RSR% (all origins)**	**RSR% (excl. self-hazard)**	**HFR%**	**Mean SPCO (%)**
S0	100.0	100.0	100.0	0.0
S1	85.71	92.31	100.0	2.78
S2	85.71	92.31	100.0	52.16

Table 6.7 summarize the evaluation comparing each scenario with RSR, HFR, and SPCO for comparison. RSR and HFR reproduce the values already discussed for S0, S1, and S2, so the table's added value lies mainly in the SPCO column. Mean SPCO jumps from 2.78% in S1 to 52.16% in S2, reflecting that a single static hazard forces only one costly reroute (Stairs), while the cumulative propagation across t1 to t3 pushes several origins onto progressively longer detours as safe corridors are impossible to traverse. Overall, the framework holds HFR at 100% under every condition tested, trading a bounded and hazard-dependent increase in path weight for routes that are never unsafe, its only unresolved failures remain structural SPOF.

6.3 MAS Evaluation

The routing evaluation of the previous sections assumes that the hazards are already stored in the building graph. This section evaluates the component that puts them there, the multi-agent system described in section 5.3.2. The evaluation checks three things: if the agents return outputs that the rest of the system can use, if the triage agent sends each situation to the right specialized agent, the last is how much time, how many tokens and how much money a single analysis needs. The section gives a partial answer to RQ1, showing that the agents produce usable and consistent output, but it does not measure the accuracy of the hazard classification.

6.3.1 Experimental Setup

Because of hardware limitations, the agents were not run locally. All calls went through OpenRouter, using the model `google/gemma-4-26b-a4b-it-20260403` provided by NextBit. This model has been used because since the system is intended to have a local LLM, it is free to use and deploy on local machines, the size of the model have been selected to balance the resource usage, latency and accuracy.

A script generates the input data for four scenarios: in the normal scenario all sensor values are in the expected range, in the fire scenario the telemetry shows a temperature spike and high co2 levels, the earthquake scenario shows vibration and acceleration values and then the fire and earthquake scenario both kinds of signals are present, which is similar to what an explosion would produce.

Each scenario is defined by how the signal moves over the length of a time window. In the normal scenario every value stays close to a baseline, for example 24°C for temperature and 420 ppm for CO_2, with only small variations around it. In the fire scenario the temperature and CO_2 values increase gradually, from a baseline of about 30°C and 600 ppm up to about 90°C and 2400 ppm, to represent a fire that gets worse minute by minute rather than appearing all at once. In the earthquake scenario this same gradual increase is applied to the vibration and acceleration, while the other environmental values are kept at their normal baseline. In the fire and earthquake scenario the environmental values spike to the high value halfway through the time window, and the seismic values show one spike that decays over the following minutes, which is closer to what an explosion would look like than a slow escalation. On top of these, every value in every scenario has random noise added to it, drawn from a Gaussian distribution with a standard deviation equal to about 5% of the range covered by that signal. This noise keeps two runs of the same scenario different, and it is meant to simulate real sensor fluctuations. All values are also clamped to a plausible range, so the added noise cannot push a value over what a real sensor could realistically report. The environmental readings (CO_2, temperature, humidity, TVOC, eCO_2) are generated once per simulated minute, while the vibration and acceleration readings are generated four times per minute, because seismic sensors typically sample faster than environmental ones. The two resulting series are merged and sorted by timestamp before being sent to the agent, so the input is one mixed list of environmental and seismic readings rather than two separate streams.

Each run executes the graph of figure 5.12 once. The triage agent either assesses the room directly or escalates to one or more expert agents, that can call two tools, `get_room_data` and `get_adjacent_rooms`, which query the building graph in Memgraph. In this evaluation the additional tool have been removed because occasionally the earthquake agent call those tools repeatedly without returning a response. The 200 runs produced 771 LLM generations in total, each scenario have been ran 50 times.

6.3.2 Metrics

The metrics are split in three groups: effectiveness, efficiency and tool use.

The effectiveness metrics measure the quality of the output. They are needed to check whether the agents can actually be trusted to feed the place graph, since a fast or cheap answer is useless if it is not correctly structured, sent to the wrong expert, or inconsistent from one run to the next. Schema adherence is the percentage of runs that ended without a validation error against the structured formats of listing 2 and listing 3. It only counts errors, so the fields of a valid run are not inspected. It matters because the next components read these outputs, and a malformed output breaks the pipeline. Routing correctness is the percentage of runs where the `danger_type` of the final assessment matches the type expected for the scenario. When the triage agent escalates it does not write an assessment, so this metric checks the type produced by the expert it chose. The expected type is fire, smoke or heat for the fire scenario, earthquake for the earthquake scenario, both of these for the fire and earthquake scenario, and

`none` for the normal scenario. The danger score is the value between 0 and 1 in the `ThreatAssessment`, reported as mean, standard deviation and median for each scenario. It should be close to 0 in the normal scenario, high in the hazard scenarios, and similar across runs of the same scenario.

Efficiency metrics measure the cost of a run. They matter because the agents run continuously in the background, so a small delay or cost is repeated at every analysis. They are the number of LLM calls, the number of tool calls, the latency, the total number of tokens and the cost in USD. The latency is the time from the moment the run receives the telemetry to the moment it returns the assessment, and it is reported as a mean. The total tokens are reported as mean and standard deviation. The cost is the amount billed for the run, after the discount of the prompt cache; a local deployment would not pay this cost, but it shows how much the system would cost if it had to use a hosted model, for example when the local one is not available or not accurate enough. On the whole experiment is also reported the number of API failures, the total cost and the share of prompt tokens read from the cache.

Tool use metrics look only at the tools. The tools are the only part of a run that queries the place graph, so they are the most likely source of unpredictable latency and cost. The LLM generation time is the sum of the generation times of all the calls of a run. The tool overhead is the time spent executing the tools, and its share is this time divided by the latency. The share shows how much of the delay comes from the graph and how much from the model. The number of tool calls is reported as a mean, together with the percentage of runs that made the most frequent number of calls for that scenario. This shows whether the agents make the same number of queries at every run or change from run to run. Also the distinct tools used in a run are counted.

6.3.3 Effectiveness

Table 6.8 shows the effectiveness results. All the 200 outputs respected the output schema, no run ended with an API failure. The escalation had some problems that will be seen in table 6.9.

Table 6.8: Effectiveness per scenario.

Scenario	Danger score mean	Danger score SD	Danger score median
Normal	0.000	0.000	0.00
Fire	0.950	0.000	0.95
Earthquake	0.859	0.019	0.85
Fire + Earthquake	0.950	0.000	0.95
Overall	0.690	0.401	0.93

Looking at the normal scenario, the danger score stayed at 0 in every single run. This is because the triage agent handled these cases without ever escalating to an expert. In the fire scenario, and in the fire and earthquake scenario, the score was exactly 0.950 every time, with zero variation. This is expected behaviour for an LLM: given a clearly recognizable fire signal, the model tends to converge on a stable value rather than

producing a different number at every run. Another reason could be that the synthetic data used for these scenarios is not representative of real fire progressions, and instead describes an already large fire rather than one that is just emerging. The earthquake scenario shows a bit more movement, averaging 0.859 with a standard deviation of 0.019, but the variation stays small. Overall, the lack of large deviation across the hazard scenarios is a satisfying result, since it means the model responds consistently to similar conditions instead of behaving erratically.

The overall standard deviation of 0.401 might look like a sign of instability at first glance, but that is not really the case. It is simply a consequence of averaging together scores of 0 from the normal runs with scores of 0.85 and above from the hazard runs. The median of 0.93 reflects the same effect, since most of the runs in the dataset belong to a hazard scenario.

These results are preliminary and require further verification. The scenarios were designed with strong, unambiguous signals, which makes the four situations easy to separate from one another. The fact that routing worked correctly 100% of the time shows that the agents behave as expected under these conditions, but it says nothing about how they would perform with sensor data that is noisy, incomplete, or ambiguous.

6.3.4 Efficiency

Table 6.9 shows the efficiency results.

Table 6.9: Efficiency per scenario

Scenario	**LLM calls / run**	**Tool calls / run**	**Latency mean (s)**	**Total tokens (mean ± SD)**	**Cost / run (USD)**
Normal	1.00	0.00	3.73	2718 ± 4	0.00025
Fire	4.00	2.00	6.88	11774 ± 56	0.00083
Earthquake	6.00	4.00	12.14	18253 ± 31	0.00122
Fire + Earthquake	4.42	2.32	9.06	13032 ± 4050	0.00092
Overall	3.85	2.08	7.95	11444 ± 5957	0.00080

The normal scenario needs only one LLM call, the one done by the triage agent. It takes 3.73 seconds, uses 2718 tokens and costs 0.00025 USD. This is the cheapest case, and it is also the most frequent one in a real building, where most of the analyzed rooms have no hazard. If we consider a run every minute, annually it can cost 131.40 USD, that can be considered cheap to maintain, otherwise a local deploy can cost even less.

In the fire scenario a run has a mean of four LLM calls and two tool calls. The first call is the triage, and the other three belong to the fire agent, which calls the two tools

and then writes the assessment. The latency mean is 6.88 seconds, about 1.8 times the normal one, and the token count is 11774, about 4.3 times higher. The token count grows faster than the latency because every call sends again the system prompt and the previous messages of the run.

The earthquake scenario is the most expensive. A run has six LLM calls and four tool calls, so the earthquake agent calls the tools twice as often as the fire agent. The exact reason why the earthquake scenario triggers this double tool calling so consistently should be further investigated. The latency is 12.14 seconds, the tokens are 18253 and the cost is 0.00122 USD, about 4.9 times the normal one. The standard deviations of the tokens are very small in the fire and earthquake rows, which means that the agents follow the same sequence of calls in almost every run.

The fire and earthquake scenario is the least regular. The mean values are close to the ones of the fire scenario and far from what two experts would need. The standard deviation of the tokens is 4050, much higher than in the other scenarios. Table 6.10 explains why: in 45 of the 50 runs (90%) the agents made exactly two tool calls, as in the fire scenario. In the other runs they made more six calls which is the two calls of the fire agent plus the four of the earthquake agent. So in most runs the triage agent called one expert, and in the others it called both. When both experts are called they work in parallel, this is proven by the sum of the generation times that is higher than the wall-clock latency.

Over all the scenarios a run takes 7.95 seconds on average, with 3.85 LLM calls and 0.00080 USD. These values refer to one room, so if several rooms need an analysis and they are processed one after the other, the times add up.

Table 6.10 shows the latency difference between LLM generation and tool execution.

Table 6.10: Tool-use behaviour and latency composition

Scenario	**Tool calls mean**	**Runs at modal tool count (%)**	**Distinct tools**	**LLM Gen. Time (s)**	**Tool overhead (s)**	**Tool overhead (%)**
Normal	0.00	100	0.00	3.50	0.00	0.0
Fire	2.00	100	2.00	5.82	0.12	1.8
Earthquake	4.00	100	2.00	10.47	0.28	2.3
Fire + Earthquake	2.32	90	2.00	8.72	0.15	1.6
Overall	2.08	-	1.50	7.13	0.14	1.7

The tool overhead is small in every scenario, it goes from 0.12 seconds in the fire scenario to 0.28 seconds in the earthquake scenario, and it is 1.7% of the total latency on average. The largest share is 2.3%, in the earthquake scenario and almost all the latency is from the LLM generation. In the normal scenario 3.50 seconds out of 3.73 are from the LLM, and in the fire scenario 5.82 out of 6.88. The remaining time is not broken down in the tables and it is small compared to the generation time.

In the normal, fire and earthquake scenarios the minimum and the maximum number of tool calls are equal, so every run makes the same number of calls (0, 2 and 4). The fire and earthquake scenario is the only one with a range, from 2 to 6 calls, and 90% of its runs sit at the lower value, which is the modal one. The number of distinct tools is at most 2, the two tools of section 5.3.2.

This result changes how the cost of the tools should be seen: running a query on the graph, nowadays, is cheap. The cost of a tool call is in the LLM call that follows it, because the model has to read the result and generate again. If the latency needs to be reduced, the number of LLM calls is the value to act on, making the graph queries faster would change very little.

6.3.5 Reliability and Cost

The experiment consisted of 200 runs, for a total of 771 LLM generations, without API failures. The structured output was valid in all the runs, so the messages sent to the next component never needed to be repaired or discarded, neither failed API requests.

The 200 runs cost 0.1610 USD in total, which is 0.80 USD for 1,000 runs. These numbers come from running the system on OpenRouter, so a local deployment of the models would lower them further. The prompt cache reused 61.1% of the prompt tokens, token-weighted, lowering the cost by 25.3%. Tool calls accounted for 1.72% of the total latency. The cache works well in this system because the calls of one run share a long prefix, made of the system prompt and the messages already exchanged. Runs with more calls gain more from it, while a normal run has only one call and has little to reuse inside the run. Since most of the runs the system will face are no-hazard scenarios, which only need one call, the system stays efficient for most of the time it is running.

7. Discussion

The evaluation in the previous chapter shows that the framework keeps HFR at 100% in every scenario, at the cost of a longer path that grows with the number of hazards. This chapter steps back from those numbers and looks at what they actually mean for the framework as a whole. The main point is that reacting to hazards at the level of the Place graph seems enough to keep occupants away from danger, at least for the topology tested here. But the system is not ready for a real deployment, since several assumptions were made to keep the scope of this thesis manageable; such as how hazards are detected and how occupants are represented, while others concern how the framework was tested. This chapter goes through these points one by one, first as limitations of the current work, then as directions for future work.

7.1 Research Questions Revisited

Section 1.1 introduced three research questions to guide the work, table 7.1 maps each one to the evidence collected in chapter 6 and gives it a status. Pass means the question is answered by the results in hand. Limited means the results give a partial answer, or the answer only holds under the conditions tested. KO means the question was not answered, or the result goes against what was expected. The notes column points to the limitations that justify the status, which are discussed in section 7.2.

Table 7.1: Summary of the research questions and how they were addressed. The full text of each research question is given in section 1.1.

RQ	Status	Notes
RQ1	Limited	Agents returned a schema-valid assessment in all 200 runs, and the danger score was stable across runs. Triage correctly routed single-hazard cases to the matching expert, but in the fire and earthquake scenario it called only one expert in 45 of 50 runs. Accuracy was not measured: the telemetry is synthetic, the four scenarios are clear-cut, and the danger score was not validated against a reference.
RQ2	Limited	The BPMN processes for the occupant, the warden and the security system are not tied to a specific hazard; this supports the design, but only one emergency management strategy was developed. Other strategies, such as sheltering in place or locking down the rooms in the case of a terrorist attack, or strategies tailored to each type of hazard (a different response for a fire than for an earthquake), were never implemented.
RQ3	Pass	HFR stayed at 100% in every scenario, as expected for a hard constraint. Compared to the static baseline, the extra path weight was zero for unaffected origins, 33.33% for the only rerouted origin in S1, and 52.16% on average in S2 once three hazards had accumulated. No threshold for "reasonable" was fixed beforehand. Origins left without a route are due to SPOF rooms in the building graph, not to the routing strategy.

7.2 Limitations

Physical and hazard model. The hazard detection, actually, only reads temperature and smoke values coming from generic IoT sensors, so anything related to the building's HVAC system is left out. This is a gap, because an HVAC system can carry smoke or gas between rooms that share the same ventilation ducts, even when those rooms are not adjacent in the Place graph. When the agents want to see if a hazard can be propagated in multiple places, the implemented tool only follows the `CONNECTED_TO` relationships, which represent doors and corridors, so a path marked as safe in the graph could still be unsafe in practice if it shares an air duct with the hazardous area. More generally, the physical layer graph would need to be enriched with this kind of information to produce more accurate predictions.

The physical layer does not include any BIM data yet, even though this kind of information could help the agents make better predictions by having more spatial context and details on the building materials.

Dynamic weight calculation for the routing algorithm was not implemented. Without it, hazards are considered as a hard constraint on the possible paths, and the evaluation results are less precise than they could be, since the generated paths would probably show more variation once hazard adjacency and severity is considered into the node weights.

Occupant behavior model. Compared to the systems reviewed in chapter 3, this work does not include any simulation of people. The evaluation is carried out at a topological level, treating each room as a source and computing only the cost of the resulting paths. Other works in the literature go further and combine CFD tools such as FDS with Agent-Based Modeling, which lets them simulate panic, herding, congestion, and dynamic fear levels among occupants; none of this behaviors is captured here.

In the `Occupant` BPMN process, an occupant is represented as a token that deterministically follows the safe path indicated by the dynamic lighting. There is no individual variability, no modeling of groups moving together, and no probability of an occupant not complying with the instructions. This is worth pointing out because it mirrors one of the gaps identified in the literature review: Zhang et al. [58] also assume that every occupant will follow the dynamic evacuation signs, ignoring herding effects, bounded rationality, and the mistrust people can have toward automated systems. Furthermore, the number of occupants and wardens was fixed to one instance per type. In a real deployment, every occupant, or every group of occupants in a room, would likely be represented by its own process instance, and the same would apply to wardens. This kind of parallel behavior still needs to be modeled.

Emergency management strategy. Only one emergency management strategy was developed in this work, the evacuation of the occupants along a safe path. This is applied for every hazard, so the framework does not yet distinguish between situations where evacuating is the right response and situations where it is not. For example, in a terrorist attack the appropriate strategy would be to lock down and shelter in the rooms, and in an earthquake the immediate response differs from the one for a fire.

Validation and evaluation scope. The BPMN process was not validated by domain experts or real users. The validation performed in this work is quantitative, so there

is no confirmation that the modeled strategy is actually plausible from an operational point of view.

The evaluation was carried out on a single topology, the Department A of the Computer Science division at the University of Camerino. Generalizing the results would require testing the system on more topologies, including larger buildings and irregular layouts.

Because of a lack of data, the accuracy of the hazard detection itself was not evaluated. This is also tied to the fact that each agent is currently specialized only through its system prompt, so there is still development work needed before a proper accuracy evaluation would make sense. This is left as future work.

The MAS results are valid only for the conditions described in the evaluation. It used a single LLM from a single provider, one random seed, one topology and the telemetry it was tested on is synthetic, and the four scenarios were built as clear cases rather than ambiguous ones. The accuracy of the hazard classification on real telemetry was not measured, and the danger score returned by the agents was not validated against any reference value. Because of this, the evaluation answers the part of RQ1 about the agents producing consistent and usable output, and it leaves their accuracy on real data as an open question. The agents returned a valid assessment in every run, the routing is always successful for the fire agent but the earthquake agent needs a rework. A run took between 3.7 and 12.1 seconds on average, most of that time is spent generating text, and the graph tools call added less than 3% to it.

Infrastructure. Lastly, the current implementation of the agent has no fallback mechanism for when the LLM is unavailable. This matters a lot for a system that is supposed to keep working when the underlying infrastructure might be compromised. A possible direction is deploying the LLM locally and scaling it based on demand, for example by monitoring the redis priority queue containing the rooms to analyze. More broadly, the fault tolerance of the infrastructure was not analyzed. Availability, latency, rate limiting, timeouts, and connectivity interruptions were all left out of scope, together with any fallback mechanism to handle them. Those aspects should be considered in future works with much attention, since it is a safety-critical system, if it does not work during the emergency it becomes useless.

7.3 Future Work

Physical and hazard model. The physical layer graph used in this thesis was built by manually writing the JSON file, only defining the connections between places and the fundamental attributes of each node. Building data coming from BIM sources was never integrated, even though it was part of the original idea. This gap from the idea and what was actually implemented gives a clear improvement to pursue: adding BIM data would give the agents more spatial context, such as details on building materials, and would likely lead to better predictions.

Dynamic re-routing is also a feature that could be interesting to develop, it was not implemented because of time constraints. The idea is to recalculate the weights of the graph as a hazard appears, updates or spreads, and to plug this recalculation into the live pipeline, from the agent to Node-RED and then to Memgraph, instead of only using the static weight baseline used in the evaluation of this thesis.

Occupant behavior and accessibility. The physical movement of occupants is not simulated in this work. This means that evacuation time, pedestrians speed, congestion, bottlenecks, crowd dynamics, and individual behavioral responses are all left out. Addressing this would connect the topological evaluation carried out here with the more realistic simulations found in other works, for example the ones combining CFD tools with Agent-Based Modeling.

Building accessibility is a related gap, elevators were intentionally excluded from the physical model in this iteration, and there is currently no modeling of occupants with reduced mobility, nor any dedicated path or refuge area for them. This is in contrast with the literature review, where safeguarding stairwell availability and centralizing emergency elevators is described as important for the vertical mobility of vulnerable people. Modeling elevators and reduced-mobility occupants explicitly, and giving them dedicated paths in the routing logic by enriching the physical layer and updating the BPMN model, is left as future work.

Hazard agents. The Fire and Earthquake agents can still be improved, since each agent is currently specialized only through its system prompt, they can be improved by adding more tools to query the place graph, giving knowledge about the respective hazard by finetuning the LLM or implementing a fast RAG system. Beyond improving the two existing agents, the architecture was designed to make this kind of extension easy. The triage step and the conditional routing built around it allow new hazard agents, such as a Gas Leak agent, to be added without changing the central logic, so expanding the system to new hazard types is a natural next step.

Emergency management strategies. The framework currently implements a single strategy, the evacuation from the building. An extension is to add other strategies as separate BPMN processes, such as shelter in place for a terrorist attack, where occupants are told to stay in or move into secure rooms instead of heading to the exits. The strategy could be selected per hazard type: by extending the logical views, we can get from the environment engine more specific logical places that can be used by the BPMN environment engine to automatically trigger a specific emergency response process.

Modeling tooling. The architecture described in section 5.1 includes a modeler for the physical space, together with the BPMN and data flow modelers. This physical space modeler was never built, and at the moment the BPMN process can be edited with the BPMN modeler provided by Camunda and the data flow with Node-RED, but there is no equivalent tool for editing the physical layer graph.

The BPMN extension is also incomplete, this thesis builds on the theoretical foundation of environment-aware BPMN collaborations defined in Corradini et al. [25], but the extension was not implemented using the custom task types described. Standard BPMN elements and an external mediator, Node-RED, were used instead as a workaround. Building a meta-model, or a native extension of the Modeler, that supports these custom task types directly would remove the need for this workaround.

Infrastructure and scalability. The agent worker at the moment does not scale automatically, a DevOps process could be added to scale the number of workers up or down based on demand. Something similar applies to Node-RED: the ingestion layer

will need a topic-based partitioning strategy, for example sticky routing, to keep the in-memory stateful aggregation consistent once more instances are added.

The current implementation only relies on OpenRouter to access LLMs. In the future, a different stack, such as vLLM, could be used instead, which would make it possible to scale the LLM inference layer on its own.

Fault tolerance and reliability were only briefly discussed in this thesis. No redundancy or distribution was discussed for Memgraph, Camunda, Node-RED, or MQTT, and none of these components have a fallback mechanism for when they become unavailable. Introducing redundant components and proper fallback strategies for each of these parts is left as future work.

Security. The literature review lists privacy, security, and cost as critical factors for these kinds of architectures, but in this thesis insufficient security mechanisms have been used. Node-RED, MQTT, the Camunda REST interface, and the MCP toolkit are not covered by any authentication, authorization, or tamper protection mechanism. This is not a small detail, since the system controls physical actuators, such as emergency lights, during an actual emergency, and a malicious actor could, in theory, disable or spoof the escape route signals. Addressing this should be a priority in any future development of the system.

Human oversight and regulatory compliance. The whole pipeline, from hazard classification by the LLM agents to the activation of physical actuators through the BPMN engine, currently runs without any human involvement. There is not any moment where a human operator can validate or block an action before it reaches an actuator. Given that hallucination is already a known risk for LLM-based agents, the lack of a human-in-the-loop mechanism is a significant gap for a life-safety system.

Regulatory compliance was also left out, NFPA 101 is never used to check whether the strategies produced by the BPMN engine, such as paths, gathering points, and evacuation times, actually respect real fire code requirements, like minimum escape route widths or maximum travel distances. The validation carried out in this work stays purely technical, and adding a normative check against real building and fire codes is an important direction for future work.

8. Conclusion

This thesis moved the paradigm of indoor emergency response from static evacuation plans to a system that can react while a hazard is happening. The idea comes from an observation: fixed escape routes do not account for the actual state of the building and where the hazard actually is, and in the worst case those paths can send people directly into dangerous zones. To address this, the thesis proposed a framework that combines a graph representation of the environment, real-time IoT data, a multi-agent system and BPMN process logic to detect hazards and compute evacuation routes that adapt as environment conditions change.

A prototype was built to test this idea on a real building, the Department A of the Computer Science division at the University of Camerino. Node-RED was used to collect and aggregate heterogeneous data such as IoT telemetry, Memgraph to store and query the environment as a graph, and Camunda to run the BPMN processes that define how the system, occupants and security warden react to hazardous scenarios. LLM-based agents were introduced to interpret the incoming aggregated data and classify hazards, enriching the building graph and triggering the appropriate response via the BPMN processes.

The evaluation focused on the routing behavior of the system rather than the physical dynamics of an evacuation. When no hazard is occurring, the routes generated by the framework match the traditional ones. This confirms that it does not introduce unnecessary detours in a normal environment. When hazards appear, whether a single one or a sequence over time, the framework always found a valid safe route that avoids hazardous zones, at the cost of a longer path. When no safe route is found for a given location, this is not a failure of the routing itself, but a consequence of the building graph, where rooms are connected through a single corridor, or simply of the fact that no alternative safe path is actually available.

The results support the idea that combining a graph representation of a building with hazard-aware routing can help people evacuate safely along an efficient path. Still, the prototype is a simplified version of what a deployed system would need to be. It does not track people's movement or behavior during an emergency, it does not yet use BIM data to enrich the building graph, and it assumes every occupant strictly follows the guidance. There is no way for a human to stop the system when a false positive happens, and fault tolerance, security and compliance with hazard regulations were left outside the scope of this work. These are not minor details for a system meant to work in a life-safety context, so they will be considered as next steps.

The most impacted users are the occupants, who will implicitly use the system to exit the building safely. The second user is the security warden: by interacting with the system, they can spot the people who need help, without wasting time checking empty rooms. The third user is the developers, since this framework can be reused and modified to handle different types of hazards and hazard-management BPMN processes,

simply by using the pre-built nodes in Node-RED and wiring some configuration, such as the aggregation type of the IoT parameters. The fourth user is the external first responders (firefighters, paramedics, civil protection), who upon arrival can consult the system to know where the detected hazards are located, which areas have already been evacuated, and where occupants may still be trapped, instead of having to discover this information on site.

The work presented here shows that an adaptive, agent-driven approach to indoor evacuation is technically feasible and can outperform static routing once hazards appear. What remains is to close the gap between the prototype and a system that could be trusted during a real emergency, which means validating the occupant and hazard models against real behavior, testing the architecture on different buildings, and addressing the reliability and safety concerns previously discussed. In conclusion, this work is a first step, but it shows that the direction is worth pursuing. With more work on the open points, a system like this could genuinely make evacuations safer.

Bibliography

[1] *NFPA 101 Code Development*. URL: https://www.nfpa.org/codes-and-standards/nfpa-101-standard-development/101 (visited on 04/08/2026).

[2] Timothy Onyenobi, J. Hudson, and M. Ormerod. "Impact of Building Geometry on Fire Spread Rate". In: 2006. URL: https://www.arcom.ac.uk/-docs/proceedings/ar2006-0739-0748_Onyenobi_Hudson_and_Ormerod.pdf (visited on 04/08/2026).

[3] *Simulation of Static and Dynamic Evacuation Algorithms in Intelligent Buildings Using Unity3D — IEEE Conference Publication — IEEE Xplore*. URL: https://ieeexplore.ieee.org/document/9486536/ (visited on 04/08/2026).

[4] Mohammad Morsali and Siavash H. Khajavi. *Digital Twin and Agentic AI for Wild Fire Disaster Management: Intelligent Virtual Situation Room*. Feb. 9, 2026. DOI: 10.48550/arXiv.2602.08949. arXiv: 2602.08949 [cs]. URL: http://arxiv.org/abs/2602.08949 (visited on 04/08/2026). Pre-published.

[5] George D. Haddow, Jane A. Bullock, and Damon P. Coppola. *Introduction to Emergency Management*. 2. Butterworth-Heinemann Homeland Security Series. Amsterdam Heidelberg: Elsevier Butterworth-Heinemann, 2006. 408 pp. ISBN: 978-0-7506-7961-9.

[6] Shahriar Akter and Samuel Fosso Wamba. "Big Data and Disaster Management: A Systematic Review and Agenda for Future Research". In: *Annals of Operations Research* 283.1 (Dec. 1, 2019), pp. 939–959. ISSN: 1572-9338. DOI: 10.1007/s10479-017-2584-2. URL: https://doi.org/10.1007/s10479-017-2584-2 (visited on 04/21/2026).

[7] Zhenyu Lei et al. *Harnessing Large Language Models for Disaster Management: A Survey*. Aug. 22, 2025. DOI: 10.48550/arXiv.2501.06932. arXiv: 2501.06932 [cs]. URL: http://arxiv.org/abs/2501.06932 (visited on 04/22/2026). Pre-published.

[8] Fengyi Xu et al. "Large Language Model Applications in Disaster Management: An Interdisciplinary Review". In: *International Journal of Disaster Risk Reduction* 127 (Sept. 1, 2025), p. 105642. ISSN: 2212-4209. DOI: 10.1016/j.ijdrr.2025.105642. URL: https://www.sciencedirect.com/science/article/pii/S2212420925004662 (visited on 04/22/2026).

[9] Murat Ayhan et al. "Requirements of Indoor Navigation During Emergency Response and Disaster Management". In: Apr. 16, 2012.

[10] Jing Lin et al. "How Occupants Respond to Building Emergencies: A Systematic Review of Behavioral Characteristics and Behavioral Theories". In: *Safety Science* 122 (Feb. 1, 2020), p. 104540. ISSN: 0925-7535. DOI: 10.1016/j.ssci.2019.104540. URL: https://www.sciencedirect.com/science/article/pii/S0925753519321514 (visited on 04/22/2026).

[11] Erica D Kuligowski. *The Process of Human Behavior in Fires*. NBS TN 1632. Gaithersburg, MD: National Bureau of Standards, 2009, NBS TN 1632. DOI: 10.6028/NIST.TN.1632. URL: https://nvlpubs.nist.gov/nistpubs/Legacy/TN/nbstechnicalnote1632.pdf (visited on 04/23/2026).

[12] Markus Friberg and Michael Hjelm. "Mass Evacuation-Human Behavior and Crowd Dynamics-What Do We Know?" In: *Lutvdg/tvbb* (2015), pp. 1–71. URL: https://lup.lub.lu.se/student-papers/search/publication/7766859 (visited on 05/04/2026).

[13] Marin Lujak and Sascha Ossowski. "On Avoiding Panic by Pedestrian Route Recommendation in Smart Spaces". In: *2016 IEEE International Black Sea Conference on Communications and Networking (BlackSeaCom)*. IEEE, 2016, pp. 1–5. DOI: 10.1109/BlackSeaCom.2016.7901576. URL: https://ieeexplore.ieee.org/abstract/document/7901576/ (visited on 05/04/2026).

[14] Marin Lujak and Sascha Ossowski. "Evacuation Route Optimization Architecture Considering Human Factor". In: *AI Communications* 30.1 (Mar. 27, 2017), pp. 53–66. ISSN: 18758452, 09217126. DOI: 10.3233/AIC-170721. URL: https://journals.sagepub.com/doi/full/10.3233/AIC-170721 (visited on 05/04/2026).

[15] Hongrak Pak and Ali Mostafavi. *Situational Awareness as the Imperative Capability for Disaster Resilience in the Era of Complex Hazards and Artificial Intelligence*. Aug. 21, 2025. DOI: 10.48550/arXiv.2508.16669. arXiv: 2508.16669 [cs]. URL: http://arxiv.org/abs/2508.16669 (visited on 04/23/2026). Pre-published.

[16] Vivekananda Reddy Uppaluri. "Real-Time AI-driven Hazard Detection: Integrating Computer Vision and Sensor Networks for Enhanced Mining Safety". In: *International Journal of Scientific Research in Computer Science, Engineering and Information Technology* 11.1 (Jan. 3, 2025), pp. 195–202. ISSN: 2456-3307. DOI: 10.32628/CSEIT25111228. URL: https://ijsrcseit.com/index.php/home/article/view/CSEIT25111228 (visited on 05/06/2026).

[17] Gabriele Bernardini et al. "Can Active and Passive Wayfinding Systems Support Fire Evacuation in Buildings? Insights from a Virtual Reality-Based Experiment". In: *Journal of Building Engineering* 74 (Sept. 1, 2023), p. 106778. ISSN: 2352-7102. DOI: 10.1016/j.jobe.2023.106778. URL: https://www.sciencedirect.com/science/article/pii/S2352710223009579 (visited on 04/29/2026).

[18] Huibo Bi and Erol Gelenbe. "A Survey of Algorithms and Systems for Evacuating People in Confined Spaces". In: *Electronics* 8.6 (June 2019), pp. 711–737. ISSN: 2079-9292. DOI: 10.3390/electronics8060711. URL: https://www.mdpi.com/2079-9292/8/6/711 (visited on 04/29/2026).

[19] Adriana Balboa et al. "Testing a Real-Time Intelligent Evacuation Guiding System for Complex Buildings". In: *Safety Science* 132 (Dec. 1, 2020), p. 104970. ISSN: 0925-7535. DOI: 10.1016/j.ssci.2020.104970. URL: https://www.sciencedirect.com/science/article/pii/S0925753520303672 (visited on 04/29/2026).

[20] Fabrizio Fornari et al. "Digital Twins of Business Processes: A Research Manifesto". In: *Internet of Things* 30 (Mar. 1, 2025), p. 101477. ISSN: 2542-6605. DOI: 10.1016/j.iot.2024.101477. URL: https://www.sciencedirect.com/science/article/pii/S2542660524004189 (visited on 04/28/2026).

[21] Mohammed Eunus Ali et al. "Enabling Spatial Digital Twins: Technologies, Challenges, and Future Research Directions". In: *PFG – Journal of Photogrammetry, Remote Sensing and Geoinformation Science* 92.6 (Dec. 2024), pp. 761–778. ISSN: 2512-2789, 2512-2819. DOI: 10.1007/s41064-024-00301-2. URL: https://link.springer.com/10.1007/s41064-024-00301-2 (visited on 05/08/2026).

[22] Lucia Cristina Tolve. "Management of Unexpected Events in Emergency Scenarios". In: (2019), pp. 1–154. URL: https://iris.univpm.it/handle/11566/263498 (visited on 05/11/2026).

[23] Xiaoshu Wang et al. "Digital Twin for Multi-Scenario Emergency of Railway Passenger Stations". In: *Frontiers in Physics* 11 (2024), p. 1291785. DOI: 10.3389/fphy.2023.1291785. URL: https://www.frontiersin.org/journals/physics/articles/10.3389/fphy.2023.1291785/full (visited on 05/11/2026).

[24] Ivan Compagnucci. "Adaptive Digital Twin Supporting IoT-enhanced Business Process". In: (2024). URL: https://tesidottorato.depositolegale.it/handle/20.500.14242/210562 (visited on 05/18/2026).

[25] Flavio Corradini et al. "Modeling, Formalizing, and Animating Environment-Aware BPMN Collaborations". In: (). DOI: 10.1007/978-3-032-02867-9_8.

[26] Zhiheng Xi et al. *The Rise and Potential of Large Language Model Based Agents: A Survey*. Sept. 19, 2023. DOI: 10.48550/arXiv.2309.07864. arXiv: 2309.07864 [cs]. URL: http://arxiv.org/abs/2309.07864 (visited on 04/08/2026). Pre-published.

[27] Peter Belcak et al. *Small Language Models Are the Future of Agentic AI*. Sept. 15, 2025. DOI: 10.48550/arXiv.2506.02153. arXiv: 2506.02153 [cs]. URL: http://arxiv.org/abs/2506.02153 (visited on 04/08/2026). Pre-published.

[28] Jiacheng Wang et al. "SLMQuant: Benchmarking Small Language Model Quantization for Practical Deployment". In: *Proceedings of the 3rd International Workshop on Rich Media With Generative AI*. RichMediaGAI '25. New York, NY, USA: Association for Computing Machinery, Oct. 26, 2025, pp. 2–10. ISBN: 979-8-4007-2044-4. DOI: 10.1145/3746262.3761973. URL: https://dl.acm.org/doi/10.1145/3746262.3761973 (visited on 04/08/2026).

[29] Johannes Schneider. "Generative to Agentic AI: Survey, Conceptualization, and Challenges". Version 1. In: (2025). DOI: 10.48550/ARXIV.2504.18875. URL: https://arxiv.org/abs/2504.18875 (visited on 02/04/2026).

[30] Zeyu Zhang et al. "A Survey on the Memory Mechanism of Large Language Model-Based Agents". In: *ACM Trans. Inf. Syst.* 43.6 (Sept. 10, 2025), 155:1–155:47. ISSN: 1046-8188. DOI: 10.1145/3748302. URL: https://dl.acm.org/doi/10.1145/3748302 (visited on 04/09/2026).

[31] Shunyu Yao et al. *ReAct: Synergizing Reasoning and Acting in Language Models*. Mar. 10, 2023. DOI: 10.48550/arXiv.2210.03629. arXiv: 2210.03629 [cs]. URL: http://arxiv.org/abs/2210.03629 (visited on 04/10/2026). Pre-published.

[32] Shishir G. Patil et al. *Gorilla: Large Language Model Connected with Massive APIs*. May 24, 2023. DOI: 10.48550/arXiv.2305.15334. arXiv: 2305.15334 [cs]. URL: http://arxiv.org/abs/2305.15334 (visited on 04/10/2026). Pre-published.

[33] Qingyun Wu et al. *AutoGen: Enabling next-Gen LLM Applications via Multi-Agent Conversation*. Oct. 3, 2023. DOI: 10.48550/arXiv.2308.08155. arXiv: 2308.08155 [cs]. URL: http://arxiv.org/abs/2308.08155 (visited on 04/12/2026). Pre-published.

[34] Taicheng Guo et al. *Large Language Model Based Multi-Agents: A Survey of Progress and Challenges*. Apr. 19, 2024. DOI: 10.48550/arXiv.2402.01680. arXiv: 2402.01680 [cs]. URL: http://arxiv.org/abs/2402.01680 (visited on 04/12/2026). Pre-published.

[35] Ahmed M. Darwish, Essam A. Rashed, and Ghada Khoriba. "Mitigating LLM Hallucinations Using a Multi-Agent Framework". In: *Information-an International Interdisciplinary Journal* 16.7 (June 21, 2025), pp. 1–17. ISSN: 2078-2489. DOI: 10.3390/info16070517. URL: https://www.mdpi.com/2078-2489/16/7/517 (visited on 03/26/2026).

[36] Ranjan Sapkota, Konstantinos I. Roumeliotis, and Manoj Karkee. "AI Agents vs. Agentic AI: A Conceptual Taxonomy, Applications and Challenges". In: *Information Fusion* 126 (Feb. 2026), p. 103599. ISSN: 15662535. DOI: 10.1016/j.inffus.2025.103599. URL: https://linkinghub.elsevier.com/retrieve/pii/S1566253525006712 (visited on 02/04/2026).

[37] Linhao Luo et al. *Reasoning on Graphs: Faithful and Interpretable Large Language Model Reasoning*. Feb. 24, 2024. DOI: 10.48550/arXiv.2310.01061. arXiv: 2310.01061 [cs]. URL: http://arxiv.org/abs/2310.01061 (visited on 04/14/2026). Pre-published.

[38] Shaoxiong Ji et al. "A Survey on Knowledge Graphs: Representation, Acquisition, and Applications". In: *IEEE Transactions on Neural Networks and Learning Systems* 33.2 (Feb. 2022), pp. 494–514. ISSN: 2162-2388. DOI: 10.1109/TNNLS.2021.3070843. URL: https://ieeexplore.ieee.org/document/9416312 (visited on 04/20/2026).

[39] Aidan Hogan et al. "Knowledge Graphs". In: *ACM Computing Surveys* 54.4 (July 2, 2021), pp. 1–37. DOI: 10.1145/3447772. URL: https://dl.acm.org/doi/10.1145/3447772 (visited on 04/20/2026).

[40] William Bakst. *How to Build a Knowledge Graph from Unstructured Information — Mirascope*. Nov. 19, 2024. URL: https://mirascope.com/blog/how-to-build-a-knowledge-graph (visited on 04/29/2026).

[41] Zoi Kelepiri, Athanasios Lentzas, and Dimitrios Vrakas. "Intelligent Emergency Evacuation Systems for Buildings". In: *Proceedings of the 26th Pan-Hellenic Conference on Informatics*. PCI '22. New York, NY, USA: Association for Computing Machinery, Mar. 29, 2023, pp. 52–57. ISBN: 978-1-4503-9854-1. DOI: 10.1145/3575879.3575966. URL: https://dl.acm.org/doi/10.1145/3575879.3575966 (visited on 06/07/2026).

[42] Ya-Hong Lin et al. "The IFC-based Path Planning for 3D Indoor Spaces". In: *Advanced Engineering Informatics* 27.2 (Apr. 1, 2013), pp. 189–205. ISSN: 1474-0346. DOI: 10.1016/j.aei.2012.10.001. URL: https://www.sciencedirect.com/science/article/pii/S1474034612000948 (visited on 06/08/2026).

[43] Will Y. Lin and Pao H. Lin. "Intelligent Generation of Indoor Topology (i-GIT) for Human Indoor Pathfinding Based on IFC Models and 3D GIS Technology". In: *Automation in Construction* 94 (Oct. 1, 2018), pp. 340–359. ISSN: 0926-5805. DOI: 10.1016/j.autcon.2018.07.016. URL: https://www.sciencedirect.com/science/article/pii/S092658051731138X (visited on 06/08/2026).

[44] Meiqing Fu et al. "Generating Straight Skeleton-Based Navigation Networks with Industry Foundation Classes for Indoor Way-Finding". In: *Automation in Construction* 112 (Apr. 1, 2020), p. 103057. ISSN: 0926-5805. DOI: 10.1016/j.autcon.2019.103057. URL: https://www.sciencedirect.com/science/article/pii/S0926580519307769 (visited on 06/08/2026).

[45] Farid Mirahadi and Brenda Y. McCabe. "EvacuSafe: A Real-Time Model for Building Evacuation Based on Dijkstra's Algorithm". In: *Journal of Building Engineering* 34 (Feb. 2021), p. 101687. ISSN: 23527102. DOI: 10.1016/j.jobe.2020.101687. URL: https://linkinghub.elsevier.com/retrieve/pii/S2352710219324982 (visited on 06/09/2026).

[46] Hui Deng et al. "BIM and Computer Vision-Based Framework for Fire Emergency Evacuation Considering Local Safety Performance". In: *Sensors* 21.11 (June 2, 2021). ISSN: 1424-8220. DOI: 10.3390/s21113851. URL: https://www.mdpi.com/1424-8220/21/11/3851 (visited on 06/08/2026).

[47] Shaqayeq Omidbeyk and Seyed Morsal Ghavami. "Multi-Agent Simulation of Population Evacuation during Dynamic Fire Using Reinforcement Learning Based on Integration of Geographic Information Systems and Building Information Modeling". In: *Journal of Building Engineering* 111 (2025), p. 113035. ISSN: 2352-7102. DOI: 10.1016/j.jobe.2025.113035. URL: https://www.sciencedirect.com/science/article/pii/S2352710225012720 (visited on 06/01/2026).

[48] Jun Zeng et al. "Simulation Method for Disaster Mechanism Caused by Panic Emotion of Evacuation Crowds". In: *Egyptian Informatics Journal* 32 (Dec. 2025), p. 100811. ISSN: 11108665. DOI: 10.1016/j.eij.2025.100811. URL: https://linkinghub.elsevier.com/retrieve/pii/S111086652500204X (visited on 05/30/2026).

[49] Aokun Yu et al. "Integrated Multi-Agent-Based Outpatient Building Fire Response Modeling for Risk-Driven Resource Use and Retrofitting Strategies: A Case Study". In: *Reliability Engineering & System Safety* 260 (Aug. 2025), p. 110970. ISSN: 09518320. DOI: 10.1016/j.ress.2025.110970. URL: https://linkinghub.elsevier.com/retrieve/pii/S0951832025001735 (visited on 05/30/2026).

[50] Helena G. Theodoropoulou et al. "From Real-World Images to Agent-Based Crowd Simulations: An End-to-End Pipeline". In: *Proceedings of the 25th International Conference on Autonomous Agents and Multiagent Systems*. AAMAS '26. Richland, SC: International Foundation for Autonomous Agents and Multiagent Systems, May 24, 2026, pp. 4143–4145. ISBN: 979-8-4007-2317-9. DOI: 10.65109/TLYB5574. URL: https://dl.acm.org/doi/10.65109/TLYB5574 (visited on 06/07/2026).

[51] Sen Yang et al. "When Agents Learn to Think: Large Language Model-Enhanced Agent-Based Modeling for Crowd Evacuation in Disaster Scenarios". In: *Reliability Engineering & System Safety* 269 (May 2026), p. 112056. ISSN: 09518320. DOI: 10.1016/j.ress.2025.112056. URL: https://linkinghub.elsevier.com/retrieve/pii/S0951832025012554 (visited on 05/30/2026).

[52] Yiming Xiao, Kai Yin, and Ali Mostafavi. "CrisiSense-RAG: Crisis Sensing Multimodal Retrieval-Augmented Generation for Rapid Disaster Impact Assessment". In: *Computer-Aided Civil and Infrastructure Engineering* 48 (Aug. 2026), p. 100096. ISSN: 10939687. DOI: 10.1016/j.cacaie.2026.100096. URL: https://linkinghub.elsevier.com/retrieve/pii/S1093968726030823 (visited on 05/30/2026).

[53] Jinghong Hu, Ligang Sun, and Xiliang Liu. "GeoAgent: An Agentic AI Framework for Spatial Query Understanding and Interactive Geospatial Intelligence". In: *Proceedings of the 1st ACM SIGSPATIAL International Workshop on Generative and Agentic AI for Multi-Modality Space-Time Intelligence*. GeoGenAgent '25. New York, NY, USA: Association for Computing Machinery, Nov. 2, 2025, pp. 54–60. ISBN: 979-8-4007-2261-5. DOI: 10.1145/3764915.3770719. URL: https://dl.acm.org/doi/10.1145/3764915.3770719 (visited on 06/07/2026).

[54] Bo Li et al. *Disaster Management in the Era of Agentic AI Systems: A Vision for Collective Human-Machine Intelligence for Augmented Resilience*. Oct. 21, 2025. DOI: 10.48550/arXiv.2510.16034. arXiv: 2510.16034 [cs]. URL: http://arxiv.org/abs/2510.16034 (visited on 03/17/2026). Pre-published.

[55] Jiawei Cui et al. "Large Language Models and AI Agents in Disaster-Resilient Infrastructure: Concepts, Applications, Pathways, and Challenges". In: *Reliability Engineering & System Safety* 272 (Aug. 2026), p. 112497. ISSN: 09518320. DOI: 10.1016/j.ress.2026.112497. URL: https://linkinghub.elsevier.com/retrieve/pii/S0951832026003133 (visited on 05/30/2026).

[56] Haowen Xu et al. "Towards Fully Automated City Operations: Integrating Agentic AI with Urban Digital Twins". In: *Computers, Environment and Urban Systems* 128 (Sept. 1, 2026), p. 102449. ISSN: 0198-9715. DOI: 10.1016/j.compenvurbsys.2026.102449. URL: https://www.sciencedirect.com/science/article/pii/S0198971526000517 (visited on 05/30/2026).

[57] Farshad Firouzi et al. "Generative IoT (GIoT): Advancing IoT with Generative AI and Large Language Models". In: *Digital Communications and Networks* (Mar. 2026), S235286482600026X. ISSN: 23528648. DOI: 10.1016/j.dcan.2026.03.010. URL: https://linkinghub.elsevier.com/retrieve/pii/S235286482600026X (visited on 05/30/2026).

[58] Botao Zhang et al. "Enhancing Fire Evacuation Safety in Metro Stations: Multi-Agent Reinforcement Learning-Guided Dynamic Signage Coordination". In: *Computers & Industrial Engineering* 217 (July 2026), p. 112024. ISSN: 03608352. DOI: 10.1016/j.cie.2026.112024. URL: https://linkinghub.elsevier.com/retrieve/pii/S0360835226002251 (visited on 05/30/2026).

[59] *Langgraph*. URL: https://reference.langchain.com/python/langgraph (visited on 07/27/2026).

[60] Tom Taulli and Gaurav Deshmukh. *Building Generative AI Agents: Using LangGraph, AutoGen, and CrewAI*. Berkeley, CA: Apress, 2025. ISBN: 979-8-8688-1133-3 979-8-8688-1134-0. DOI: 10.1007/979-8-8688-1134-0. URL: https://link.springer.com/10.1007/979-8-8688-1134-0 (visited on 07/27/2026).

[61] Mohammad M. Ahmadpanah et al. "Securing Node-RED Applications". In: *Protocols, Strands, and Logic: Essays Dedicated to Joshua Guttman on the Occasion of His 66.66th Birthday*. Ed. by Daniel Dougherty et al. Cham: Springer International Publishing, 2021, pp. 1–21. ISBN: 978-3-030-91631-2. DOI: 10.1007/978-3-030-91631-2_1. URL: https://doi.org/10.1007/978-3-030-91631-2_1 (visited on 07/29/2026).

[62] Innocent Uzougbo Onwuegbuzie. "Node-RED and IoT Analytics: A Real-Time Data Processing and Visualization Platform". In: (Sept. 17, 2024), pp. 1–12. DOI: 10.5281/zenodo.13856860. URL: https://zenodo.org/records/13856860 (visited on 08/03/2026).

[63] Jakob Freund and Bernd Rücker. *Real-Life BPMN: Using BPMN and DMN to Analyze, Improve, and Automate Processes in Your Company*. Trans. by James Venis, Kristen Hannum, and Jalynn Venis. 4th edition. Berlin: Camunda, 2019. 212 pp. ISBN: 978-1-0863-0209-7.

[64] *Agentic Orchestration Platform for Enterprise AI*. URL: https://camunda.com/platform/ (visited on 08/17/2026).